MONITORING WITH PROMETHEUS

With Real Examples

MetricFire

Monitoring with Prometheus: with Real Examples

Credits

Chapter 1 - Shevaun Frazier, Daryna Galata

Chapter 2 - Shevaun Frazier

Chapter 3 - Aymen El Amri

Chapter 4 - Rodrigue Chadoke

Chapter 5 - Ryan Tendonge

Chapter 6 - Madhur Ahjua

Chapter 7 - Aymen El Amri

Chapter 8 - Daryna Galata

Chapter 9 - Giedrius Statkevičius

Chapter 10 - Giedrius Statkevičius

Chapter 11 - Cian Synnott

Chapter 12 - Cian Synnott

Chapter 13 - Vaibhav Thakur

Chapter 14 - Parker Janke

Chapter 15 - Vaibhav Thakur

Editor: Lindsey Rogerson and the MetricFire team

Design: Garth Humbert, Ivan Tolmachev, Amanda Trimble and the MetricFire team

Publish location: Las Vegas, USA

ISBN:

Table of Contents

Getting started with PromQL

Top 5 Prometheus Alertmanager gotchas

Understanding Prometheus rate()

Prometheus remote storage

Example 1: Monitoring a Python web app with Prometheus

Example 2: HA Kubernetes monitoring using Prometheus and Thanos

Example 3: monitoring Redis clusters with Prometheus

Example 4: Prometheus metrics based autoscaling in Kubernetes

Foreword

This book has been produced by the MetricFire team as a resource for our community. We want to inspire our users to apply Prometheus to its fullest, and to set the foundations for great communication surrounding Prometheus. Our users need better and better monitoring every year, and we know this will stem from partnerships built on good communication. This book has been compiled from the work of our engineers, writers, users and the whole MetricFire team.

If you have any questions or comments about this ebook, please reach out to our editor through the address lindsey@metricfire.com. We are looking forward to hearing your ideas and feedback!

Introduction

Prometheus is an increasingly popular tool in the world of SREs and operational monitoring. Based on ideas from Google's internal monitoring service (Borgmon), and with native support from services like Docker and Kubernetes, Prometheus is designed for a cloud-based, containerised world. As a result, it's quite different from existing services like Graphite. It was developed in 2012, and has recently been donated to the CNCF as one of the first cloud-native open-source projects.

Prometheus is a time-series monitoring tool that scrapes and stores time-series data. The Prometheus ecosystem has many components, which enable you to scrape, store, visualize and alert on data.

Starting out, it can be tricky to know where to begin with the official Prometheus docs and the wave of recent Prometheus content. This book acts as a high level overview of the significant information relating to Prometheus, as well as a solid collection of tips and tricks for using Prometheus. We also take a look at where MetricFire fits into the world of monitoring solutions you could choose from. While going through this book, we recommend using the MetricFire 14-day free trial of MetricFire's Hosted Prometheus, so you can send metrics and learn to use Prometheus with no delay for the set up.

MetricFire is a hosted Prometheus and Grafana service, that offers a complete infrastructure and application monitoring platform. We help users collect, store and visualize time series data from any source. Our platform runs on-premises or on cloud, with support directly from engineers for alerting design, analytics and overall monitoring. Our users include large multinational coffee brewers, game companies, and other data science/SaaS companies.

1.1 What is Prometheus and how does it work?

Prometheus can be thought of as a set of tools for the following data operations:

- storage
- aggregations
- visualization
- alerts

The advantage of Prometheus is that it provides the functionality to control many systems and servers from just one place. Prometheus achieves this with a decentralized and self-managed architecture. At the same time, individual commands can be used for individual servers.

If we compare it with the existing solutions, we can say that Prometheus is a time-series database, that also has the option of adding a variety of tools that extend the functionality to monitoring and data analysis.

Let's discuss the general structure of the Prometheus architecture to better understand the principles behind how it functions.

Prometheus includes these components:

- Prometheus server is a server that processes the data requests (metrics) and stores them in a database.
- Exporters – systems or services where the monitoring process is performed. In most cases, they periodically send data to the Prometheus server. They export data in a format that will be understandable to the Prometheus server.
- Pushgateway – a component that processes metrics for short term jobs.
- Dashboard – metrics visualization using the native web interface or Grafana.

- Client Libraries – to connect different programming languages and data export tools.
- Alertmanager – the manager for sending notifications.

The core of this architecture is the **Prometheus server**, which processes data independently and stores it locally or on the selected resource. It scrapes objects to receive the essential information needed for the metrics. As a result, we need to configure the monitoring process only on the Prometheus side instead of the individual systems. This approach simplifies the deployment of the monitoring system: all you need to do is install the server and define the monitoring parameters.

Metrics collection is implemented with pull and push mechanisms. In the second case, metrics are pushed using a special **pushgateway component**, which is necessary to export (collect) metrics from protected systems or if the connection process needs to take place in a short time. Prometheus provides a ready-to-use collection of exporters for different technologies.

Visualization is done with Grafana. Grafana comes in the package that you can install from Prometheus.io. You can display your metrics on graphs, histograms, gauges, and more. You can query your metrics and set up alerts directly in Grafana. Alerts and visualizations can be set up with the Prometheus expression browser, which lets you search the stored metrics, apply functions and preview graphs. See chapter 5 for more information on visualizations.

The native query language **PromQL** allows us to select the required information from the received data set and perform analysis and visualizations with that data using Grafana dashboards. PromQL is used to query in Expression Browser as well as in Grafana. The **Alertmanager** generates notifications from this data and sends them to us using methods such as e-mail, PagerDuty, and more. See chapter 8 for more information about PromQL.

Prometheus supports configuring 2 kinds of rules – recording rules and alerting rules.

- Recording rules allow you to specify a PromQL-style rule to create new metrics from incoming data by applying transformations and functions to the data. This can be great if, for example, you have a large number of metrics to view at once, and they're taking a long time to retrieve. Instead you can create a sum () metric on the fly, and you'll only need to retrieve one metric in the future.

- Alerting rules instruct Prometheus to look at one or more metrics being collected and go into an alerting state if specified criteria are breached. The state of alerts is checked just by going to the alerts page in the Prometheus UI; Prom doesn't have the capacity to send notifications. AlertManager is a service that adds that ability, and monitors alerts separately in case the platform that a Prometheus server is running on has errors.

Applications can provide metrics endpoints to Prometheus using client libraries available for various languages. You can also use separate exporters which gather metrics from specific applications and make them available to Prometheus. Each application or exporter endpoint serves up metrics (plus tags) and appropriate metadata whenever Prometheus requests them.

Exporters The role of the exporter is extracting information at regular intervals then converting it into the Prometheus format. Sources should expose endpoints to give the server the ability to scrape the collected metrics.

Official and unofficial exporters exist for dozens of services. A popular one is node_exporter, which collects system metrics for Linux and other Unix servers. See chapter 4 for more information on exporters.

Metrics are stored locally on disk, and by default they're only retained for 15 days, providing a sliding window of data instead of a long term storage solution. Prometheus doesn't have the capability to store the metrics in more than one location. However, since the metrics aren't consumed when requested, it's possible to run more than one Prometheus for the same services in order to have redundancy. Federation also allows one Prometheus server to scrape another for data, consolidating related or aggregated data into one location.

Remote storage is another option: Prometheus can be configured with remote_write and remote_read endpoints. Prometheus will regularly forward its data to the remote_write endpoint. When

queried, it will request data via the remote_read endpoint and add it to the local data. This can produce graphs that display a much longer timeframe of metrics. **MetricFire** provides these remote storage endpoints for your Prometheus installations. See chapter 11 for more information about remote storage.

1.2 Benefits of using Prometheus

- **Service discovery** – a big plus for Prometheus. Large scale deployments can change all the time, and service discovery allows Prometheus to keep track of all the current endpoints effortlessly. Service discovery can be achieved via support from various resource management services, such as Kubernetes, Openstack, AWS EC2 and others. There are also generic options for DNS and file-based service discovery.
- **Outage detection** – since Prometheus knows what it should be monitoring, outages are very quickly detected when the request fails.
- **PromQL** is an incredibly flexible, Turing-complete, query language. It can apply functions and operators to your metric queries, filter and group by labels and use regular expressions for improved matching and filtering.
- **Low load on the services being monitored** – metrics are stored in-memory as they are generated, and are only converted into a readable format when requested. This uses up fewer resources than converting every metric into a string to send as soon as it's created (as you would for a service like Graphite). Also metrics are batched and sent all at once via HTTP, so the per-metric load is lower than sending the equivalent, even by UDP.

- **Control of traffic volumes** – push metric services can be overwhelmed by large volumes of datapoints, but Prometheus only receives metrics when it asks for them – even if the service itself is very busy. Any user of Jenkins has probably seen their metric volumes spike when a batch is processed, however with a Prometheus exporter in place, metrics would still be queried every 15s regardless of how many events are being generated. That keeps your monitoring service safe.
- **Metrics in the browser** – You can look at the metrics endpoint directly to see what's being generated at any given time, e.g. Prometheus' own metrics can be viewed on http://localhost:9090/metrics
- **Easy reconfiguration** – since Prometheus has the responsibility of obtaining metrics, if there's a change to the configuration required it only needs to be done in Prometheus instead of changing the configuration for all the monitored services.

1.3 Challenges of using Prometheus

- **Storage** – the standard storage method uses space and resources on your server. This might not be dedicated to Prometheus, and could be expensive depending on what platform you're using (AWS high-IO EBS volumes seem affordable but the costs do mount up!). The actual amount of space taken up by storage is reasonably simple to calculate, and the more services you monitor the more data you store.
- **Redundancy** – Prometheus saves to one location only, unlike Graphite for example, which can save data to storage clusters. Running multiple Prometheus instances for the same metrics

is the given solution, but it's arguably awkward and a little messy.

- **No event-driven metrics** – each value is a point-in-time retrieved on request, which doesn't work for short lived jobs and batches, since metrics will only be available for a short period of time, or intermittently. Prometheus works around this by providing a Pushgateway.
- **Limited network access** – access restrictions to the resources being monitored may mean that multiple Prometheus services have to be run, since there may not be external access to those resources. That requires connecting to different instances of Prom to view different metrics, or using Federation to scrape metrics from multiple Proms into one central server.

1.4 What is MetricFire?

At **MetricFire** we provide a hosted version of Prometheus. This includes long term, scalable, storage for Prometheus, in the form of remote_read and remote_write destinations for your existing Prometheus installations. That means off-disk storage, with redundancy built in, and extended retention of up to 2 years.

It comes with a hosted Grafana service, which lets you configure your Prometheus installations as data sources. Alternatively, you can use the **MetricFire** data source to view stored data from all your Prometheus servers together, in one place. Each Prometheus server may generate metrics with the same names and will often consider it's own hostname to be localhost:9090. You can use the 'external labels' option in the global configuration to ensure that similar metrics from different Prometheus servers can be differentiated.

To make the most of this ebook, get on to the MetricFire free trial and try it out! You can use our hosted Prometheus service within minutes of signing up, and you can try Grafana, PromQL and our add-ons directly in the platform.

Deploying Prometheus to a Minikube Cluster with One Node

2.1 Introduction

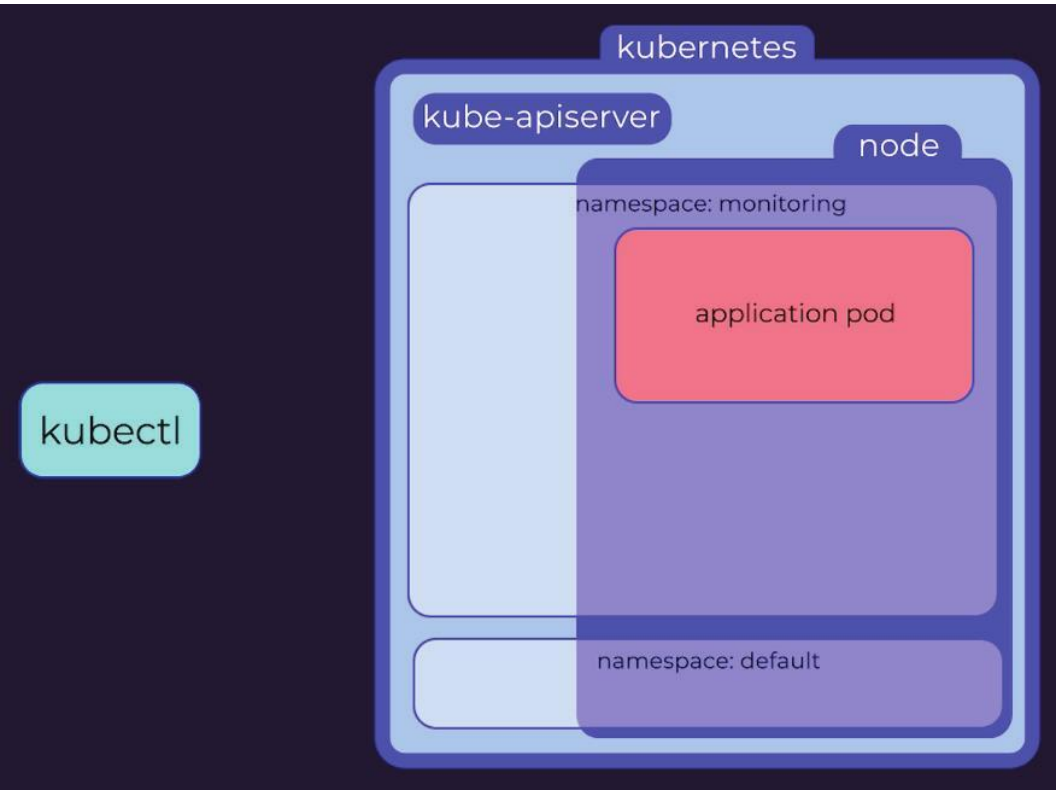
In this chapter we will look at how to deploy Prometheus to a Minikube cluster. This chapter is a full tutorial, and it includes the configuration for remote storage on MetricFire. This tutorial uses a Minikube cluster with one node, but these instructions should work for any Kubernetes cluster. Also, there is a video that walks through all the steps, if that format is better.

We'll be using YAML files to create resources since this means we can keep a record of what we've done and reuse the files whenever we need to make changes. You can find versions of the files here with space for your own details.

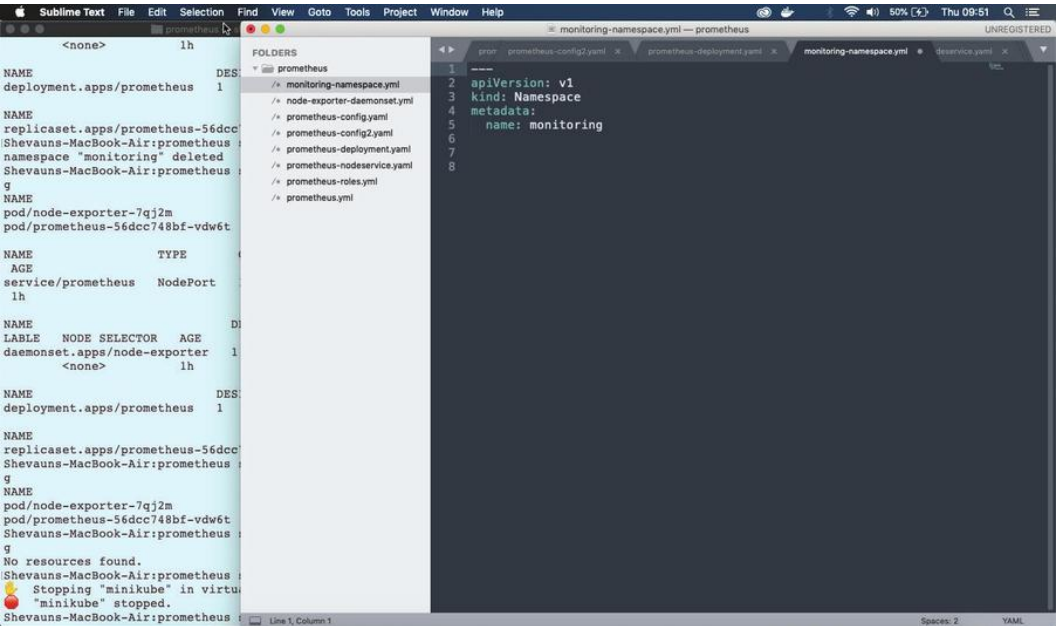
We'll go over what the YAML files contain and what they do as we go, though we won't go too deep into how Kubernetes works. It should give you a good start for tackling the rest of this book. Each of these YAML files instructs Kubectl to submit a request to the Kubernetes API server, and creates resources based on those instructions.

2.2 Creating the monitoring namespace

All resources in Kubernetes are launched in a namespace, and if no namespace is specified, then the 'default' namespace is used. To give us finer control over our monitoring setup, we'll follow best practice and create a separate namespace called "monitoring".



This is a very simple command to run manually, but we'll stick with using the files instead for speed, accuracy, and accurate reproduction later. Looking at the file we can see that it's submitted to the apiversion called **v1**, it's a kind of resource called a **Namespace**, and its name is **monitoring**.



The command to apply this is:

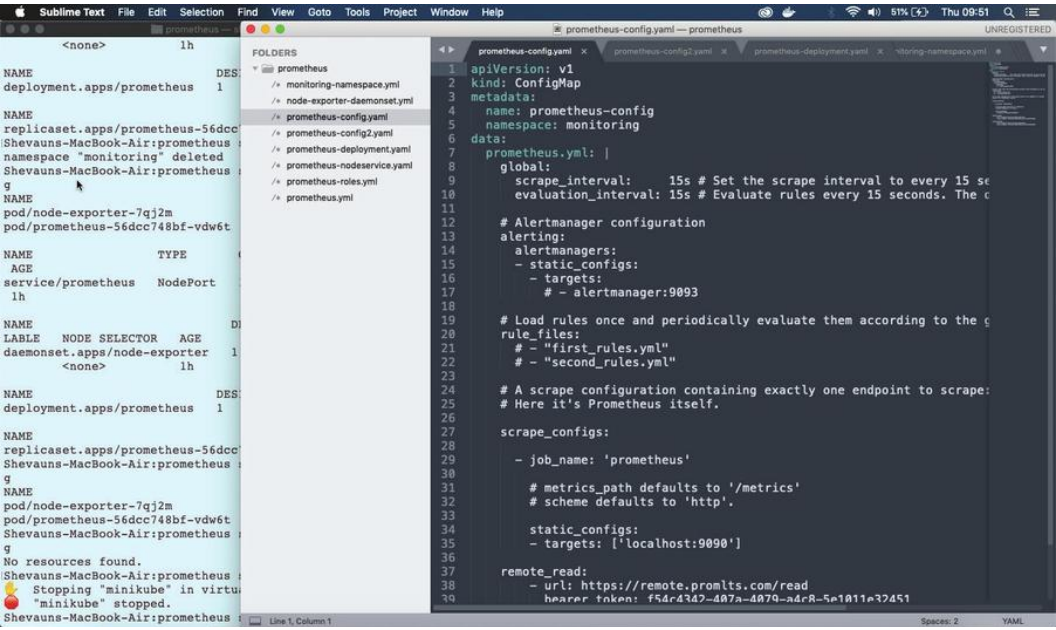
```
kubectl apply -f monitoring-namespace.yaml
```

Once this is applied we can view the available namespaces with the command:

```
kubectl get namespaces
```

2.3 Setting up the ConfigMap

The next step is to set up the configuration map. A **ConfigMap** in Kubernetes provides configuration data to all of the pods in a deployment.

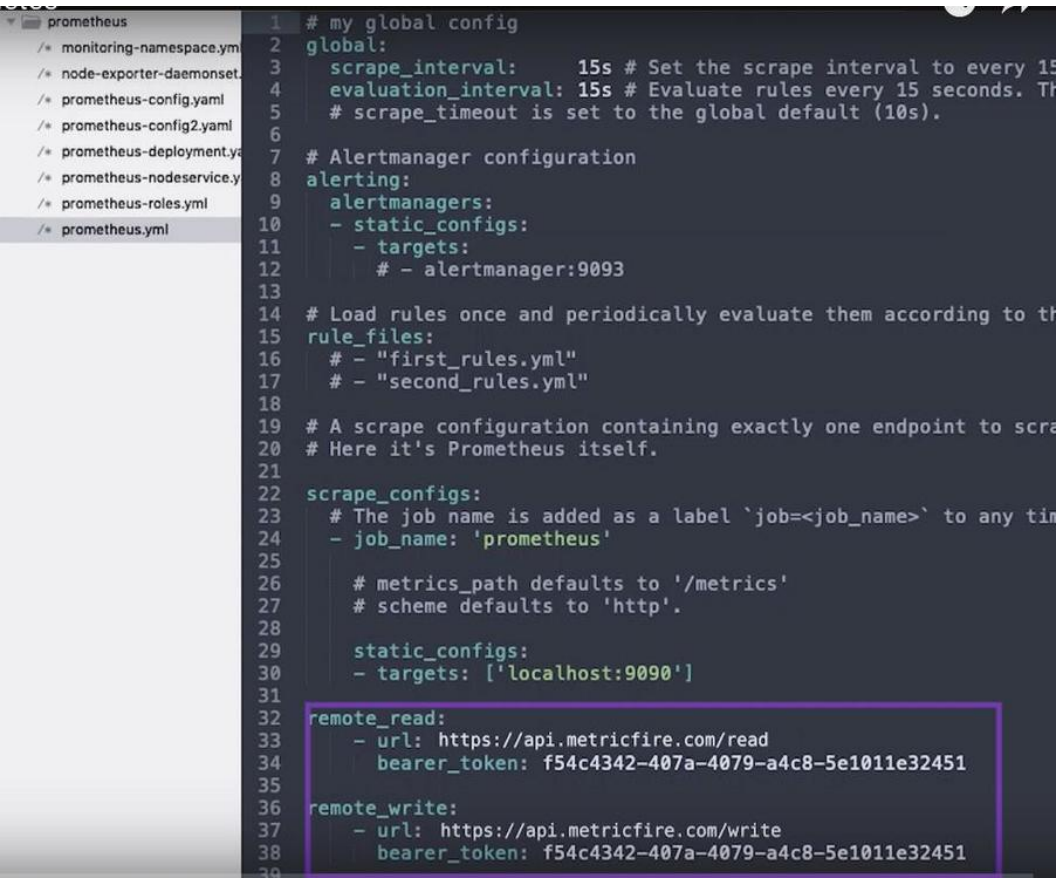


In this file we can see:

1. the apiversion, which is v1 again
2. the kind, which is now **ConfigMap**
3. in the metadata we can see the name, "`prometheus-config`", and the namespace "`monitoring`", which will place this ConfigMap into the **monitoring** namespace

Below that in the data section, there's a very simple **prometheus.yml** file. Looking at it separately we can see it contains some simple interval settings, nothing set up for alerts or rules, and just one **scrape job**, to get metrics from Prometheus about itself.

It also contains remote storage details for MetricFire, so as soon as this Prometheus instance is up and running it's going to start sending data to the remote-write location; we're just providing an endpoint and an API key for both remote_read and remote_write.

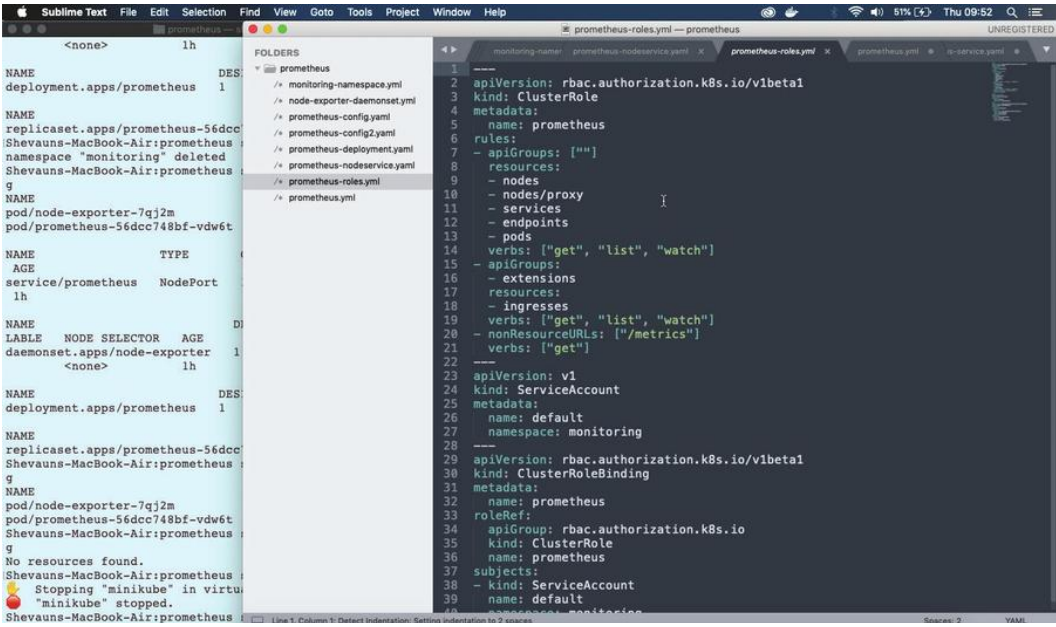


The configMap doesn't do anything by itself, but we'll apply it so it's available when we deploy prometheus later in this chapter:

```
kubectl apply -f prometheus-config.yaml
```

2.4 Setting up roles

Next, we're going to set up a **role** to give access to all the Kubernetes resources, and a **service account** to apply the role to, both in the monitoring namespace. Specifically, we'll set up a **ClusterRole**: a normal role only gives access to resources within the same namespace, and Prometheus will need access to nodes and pods from across the cluster to get all the metrics we're going to provide.



The ClusterRole's rules can be applied to groups of Kubernetes APIs (which are the same APIs kubectl uses to apply these yaml files) or to non-resource URLs - in this case "/metrics", the endpoint for scraping Prometheus metrics. The verbs for each rule determine what actions can be taken on those APIs or URLs.

The **ServiceAccount** is an identifier which can be applied to running resources and pods. If no ServiceAccount is specified then the **default** service account is applied, so we're going to make a default service account for the Monitoring namespace. That means Prometheus will use this service account by default.

Finally, we're applying a ClusterRoleBinding to bind the role to the service account.

We're creating all three of these in one file, and you could bundle them in with the deployment as well if you like. We'll keep them separate for clarity.

```
kubectl apply -f prometheus-roles.yml
```

Remember, when you apply this to your own Kubernetes cluster you may see an error message at this point about only using `kubectl apply` for resources already created by kubectl in specific ways, but the command works just fine.

2.5 Deployment

So now we're ready! We have a namespace to put everything in, we have the configuration, and we have a default service account with a cluster role bound to it. We're ready to deploy Prometheus itself.

The deployment file contains details for a **ReplicaSet**, including a **PodTemplate** to apply to all the pods in the set. The ReplicaSet data is contained in the first "**spec**" section of the file.

```
 9  spec:
10    replicas: 1
11    strategy:
12      rollingUpdate:
13        maxSurge: 1
14        maxUnavailable: 1
15      type: RollingUpdate
16    selector:
17      matchLabels:
18        app: prometheus
19    template:
20      metadata:
21        labels:
22          app: prometheus
23        annotations:
24          prometheus.io/scrape: "true"
25          prometheus.io/port: "9090"
```

Replicas is the number of desired replicas in the set. For this example we're only launching one.

Selector details how the ReplicaSet will know which pods it's controlling. This is a common way for one resource to target another.

Strategy is how updates will be performed.

The **Template** section is the pod template, which is applied to each pod in the set.

A **Namespace** isn't needed this time, since that's determined by the ReplicaSet.

A **Label** is required as per the selector rules, above, and will be used by any Services we launch to find the pod to apply to.

Values in **annotations** are very important later on, when we start scraping pods for metrics instead of just setting Prometheus up to scrape a set endpoint. They are converted into labels which can be used to set values for a job before it runs, for example an alternative port to use or a value to filter metrics by. We won't use this immediately, but we can see that we've annotated a port as 9090, which we can also view farther down.

```
26    spec:
27      containers:
28      - name: prometheus
29        image: quay.io/prometheus/prometheus:v2.0.0
30        imagePullPolicy: IfNotPresent
31        args:
32          - '--storage.tsdb.retention=6h'
33          - '--storage.tsdb.path=/prometheus'
34          - '--config.file=/etc/prometheus/prometheus.yml'
35        command:
36        - /bin/prometheus
37        ports:
38        - name: web
39          containerPort: 9090
40        volumeMounts:
41        - name: config-volume
42          mountPath: /etc/prometheus
43        - name: data
44          mountPath: /prometheus
45      restartPolicy: Always
46      securityContext: {}
47      terminationGracePeriodSeconds: 30
48      volumes:
49      - name: config-volume
50        configMap:
51          name: prometheus-config
52      - name: data
53        emptyDir: {}
```

The second **Spec** section within the template contains the specification for how each container will run. This is very involved, so we'll only go into detail about the options specific to Prometheus.

- **Image** is the docker image which will be used, in this case the prometheus image hosted on quay.io.
- **Command** is the command to run in the container when it's launched.
- **Args** are the arguments to pass to that command, including the location of the configuration file which we'll set up below.

- **Ports** is where we specify that port 9090 should be open for web traffic.
- **volumeMounts** is where external volumes or directories are mounted into the containers. They're indicated here by name and given a path - you can see here the config volume is mounted in the location specified in the arguments passed to prometheus on startup.

The **volumes** and their names are configured separately to the containers, and there are two volumes defined here.

First is the **ConfigMap**, which is considered a type of volume so that it can be referenced by processes in the container. The second is an emptyDir volume, a type of storage which exists for as long as the pod exists. If the containers are deleted the volume remains, but if the whole pod is removed, this data will be lost.

Ideally the data should be stored somewhere more permanent; we're only using temporary storage for the tutorial, but also since we've configured remote_read and remote_write details, Prometheus will be sending all the data it receives offsite to MetricFire.

We'll apply that deployment file now:

```
kubectl apply -f prometheus-deployment.yaml
```

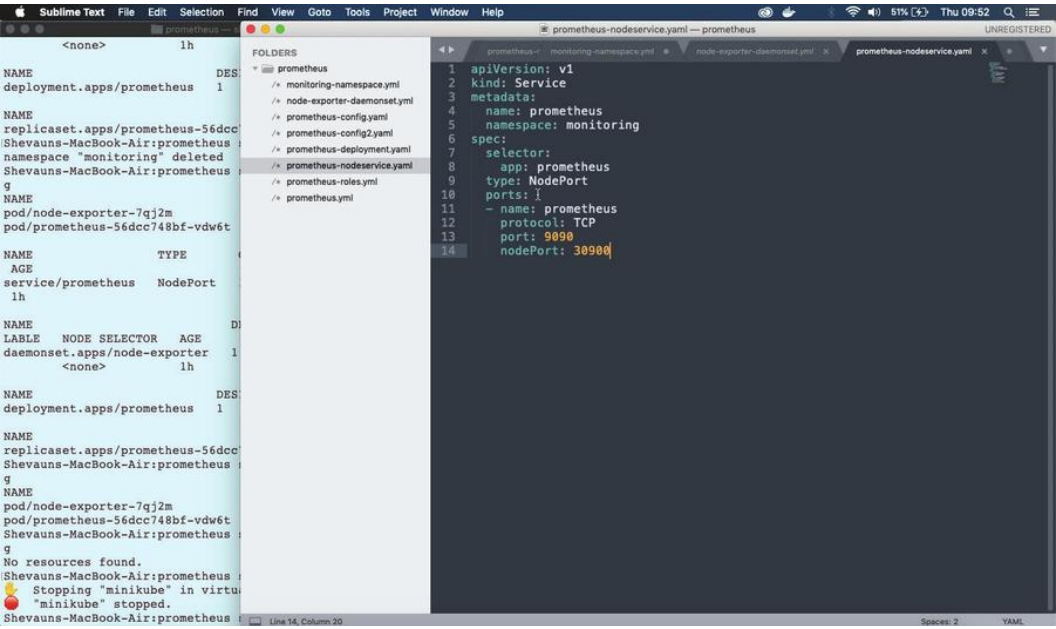
And we'll take a look at the status of the resources in our monitoring namespace:

```
Kubectl get all --namespace=monitoring
```

```
Shevauns-MacBook-Air:prometheus shevaun$ kubectl apply -f monitoring-namespace
.yml
namespace "monitoring" configured
Shevauns-MacBook-Air:prometheus shevaun$ kubectl apply -f prometheus-config.ya
ml
configmap "prometheus-config" created
Shevauns-MacBook-Air:prometheus shevaun$ kubectl apply -f prometheus-roles.yml

clusterrole.rbac.authorization.k8s.io "prometheus" configured
Warning: kubectl apply should be used on resource created by either kubectl cr
eate --save-config or kubectl apply
serviceaccount "default" configured
clusterrolebinding.rbac.authorization.k8s.io "prometheus" configured
Shevauns-MacBook-Air:prometheus shevaun$ kubectl apply -f prometheus-deploymen
t.yaml
deployment.extensions "prometheus" created
Shevauns-MacBook-Air:prometheus shevaun$ kubectl get all --namespace=monitorin
g
NAME                                 READY     STATUS    RESTARTS   AGE
pod/prometheus-6fbf5846fc-bj5vr      1/1       Running   0          14s

NAME                            DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/prometheus      1         1         1            1           14s

NAME                                       DESIRED   CURRENT   READY     AGE
replicaset.apps/prometheus-6fbf5846fc      1         1         1         14s
Shevauns-MacBook-Air:prometheus shevaun$
```

2.6 NodePort

There's one thing left to do before we can start looking at our metrics in Prometheus. At the moment we don't have access to Prometheus, since it's running in a cluster. We can set up a service called a NodePort which will allow access to Prometheus via the node IP address.



The file is very simple, stating a namespace, a selector (so it can apply itself to the correct pods), and the ports to use.

```
kubectl apply -f prometheus-nodeservice.yaml
```

Once we apply this, we can take a look at our running Prometheus on port 30900 on any node. Getting the node IP address differs for each Kubernetes setup, but luckily Minikube has a simple way to get the node url. We can see all the services using:

```
minikube service list
```

Or we can directly open the URL for Prometheus on our default browser using:

```
minikube service --namespace=monitoring
prometheus
```

The metrics available are all coming from Prometheus itself via that one scrape job in the configuration. We can bring up all the metrics for that job by searching for the label “job” with the value “prometheus” `{job="prometheus"}`.

2.7 Node Exporter and a new ConfigMap

Now we need to get some useful metrics about our cluster. We're going to use an application called Node Exporter to get metrics about the cluster node, and then change the Prometheus configmap to include jobs for the nodes and pods in the cluster. We'll be using Kubernetes service discovery to get the endpoints and metadata for these new jobs.

Node Exporter is deployed using a special kind of ReplicaSet called a DaemonSet. Where a ReplicaSet controls any number of pods running on one or more nodes, a DaemonSet runs exactly one pod per node. It’s perfect for a node monitoring application.

A file for creating a DaemonSet looks a lot like the file for a normal deployment. There’s no number of replicas however, since that’s fixed by the DaemonSet, but there is a PodTemplate as before, including metadata with annotations, and the spec for the container.

The volumes for Node Exporter are quite different though. There’s no configmap volume, but instead we can see system directories

from the node are mapped as volumes into the container. That's how Node Exporter accesses metric values. Node Exporter has permission to access those values because of the **securityContext** setting, "`privileged: true`"

We'll apply that now, and then look to see the DaemonSet running:

```
kubectl apply -f node-exporter-daemonset.yml
kubectl get all --namespace=monitoring
```

In the new configMap file the `prometheus` job has been commented out because we're going to get the metrics in a different way. Instead, two new jobs have been added in: `kubernetes-nodes` and `kubernetes-pods`.

Kubernetes-pods will request metrics from each pod in the cluster, including Node Exporter and Prometheus, while kubernetes-nodes will use service discovery to get names for all the nodes, and then request information about them from Kubernetes itself.

In the nodes job you can see we've added details for a secure connection using credentials provided by Kubernetes. There are also a number of relabelling rules. These act on the **labelset** for the job, which consists of standard labels created by Prometheus, and metadata labels provided by service discovery. These rules can create new labels or change the settings of the job itself before it runs.

In this case the rules are doing three things:

- First, creating labels for the job based on any labels applied to the node.
- Second, changing the address used for the job from the one provided by service discovery, to a specific endpoint for accessing node metrics.
- And third, changing the metric path from /metrics, to a specific API path which includes the node name.

In the second job, we're accessing the annotations set on the pods. The annotation called `prometheus.io/scrape` is being used to clarify which pods should be scraped for metrics, and the annotation `prometheus.io/port` is being used along with the `__address__` tag to ensure that the right port is used for the scrape job for each pod.

Replacing the configMap is a 2-step process for Prometheus. First, we give Kubernetes the replacement map with the replace command:

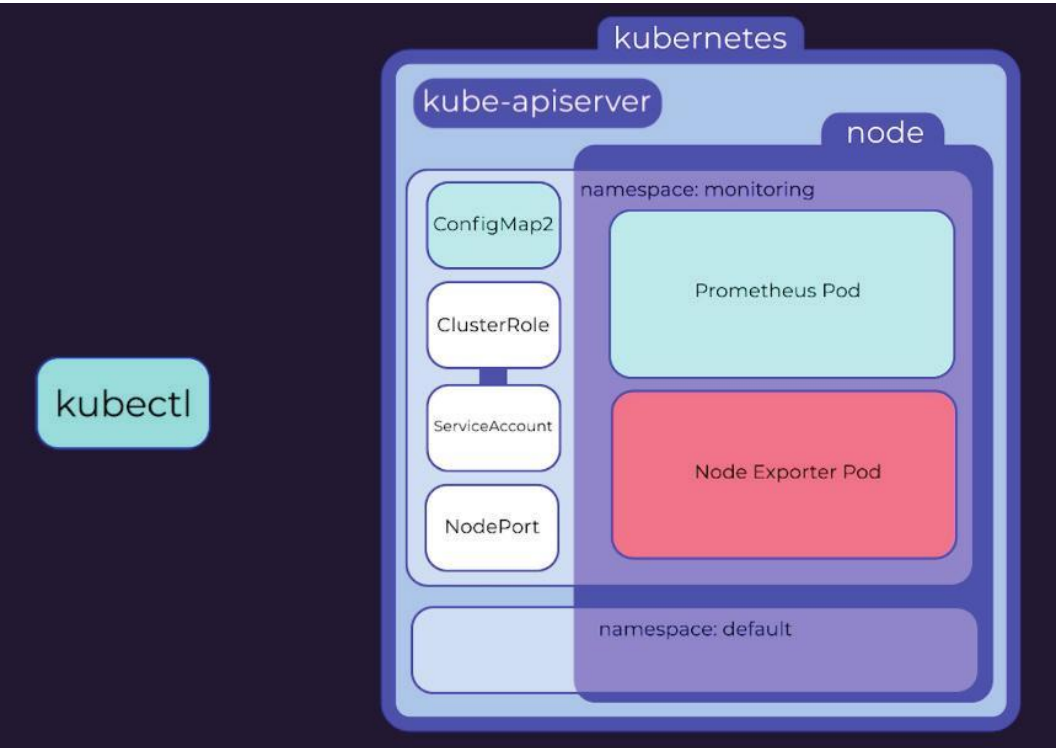
```
kubectl replace -f prometheus-config2.yaml
```

The configMap will be rolled out to every container which is using it. However, Prometheus doesn't automatically load the new configuration - you can see the old configuration and jobs if you look in the Prometheus UI - prometheus:30900/config

The quickest way to load the new config is to scale the number of replicas down to 0 and then back up to one, causing a new pod to be created. This will lose the existing data, but of course, it's all been sent to MetricFire.

If we refresh the configuration page we can now see the new jobs. If we check the targets page, the targets and metadata are visible as well. Metrics can be found under the kubernetes-pods job, with the node prefix.

If we flip over to MetricFire, I've already set up a dashboard for Node Exporter metrics. If I refresh the dashboard, you can see these new metrics are now visible via the MetricFire Datasource.



To summarize then, we have:

- Created a namespace
- Created a configMap
- Created a ClusterRole, a default ServiceAccount, and bound them together.
- Deployed Prometheus
- Created a nodeport service to expose the Prometheus UI
- Deployed the node-exporter daemonset
- Updated the configMap with new jobs for the node exporter
- And we've reloaded Prometheus by scaling to 0 and back up to 1

Once you're comfortable with this setup, you can add other services like cAdvisor for monitoring your containers, and jobs to get metrics about other parts of Kubernetes. For each new service, simply configure a new scrape job, update the configMap, and reload the configuration in Prometheus. Easy!

All good tutorials should end by telling you how to clean up your environment. In this case, it's really easy: removing the namespace will remove everything inside of it! So we'll just run

```
kubectl delete namespace monitoring
```

And then confirm that everything is either gone, or shutting down:

```
kubectl get all --namespace=monitoring
```

After a few moments, everything has been cleaned up.

Deploying Prometheus to Kubernetes on GKE with Helm

3.1 Overview

In this chapter, we are going to see how to use Prometheus on GKE by doing a Helm install. We'll take a look at the components that get installed through the Helm chart that we use, and also how to install Grafana.

3.2 Installation and Configuration

You will need to run a Kubernetes cluster first. You can use a Minikube cluster like in Chapter 2, or deploy a cloud-managed solution like GKE. In this chapter we'll use GKE.

We are also going to use Helm to deploy Grafana and Prometheus. If you don't know Helm, it's a package manager for Kubernetes - it is just like APT for Debian. The CNCF maintains the project in collaboration with Microsoft, Google, Bitnami, and the Helm contributor community.

With the help of Helm, we can manage Kubernetes applications using "Helm Charts". The role of Helm Charts is defining, installing, and upgrading Kubernetes applications.

The Helm community develops and shares charts on Helm hub. From web servers, CI/CD tools, databases, and security tools to web apps, Helm hub hosts a distributed repositories of Kubernetes-ready apps.

To install Helm, start by downloading the last version, unpack it and move "helm" binary to "/usr/local/bin/helm":

```
mv linux-amd64/helm /usr/local/bin/helm
```

MacOS users can use brew install helm, Windows users can use Chocolatey choco install kubernetes-helm. Linux users (and MacOS users as well), can use the following script:

```
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/
master/scripts/get-helm-3 chmod 700 get_helm.
sh ./get_helm.sh
```

Using Helm, we are going to install Prometheus operator in a separate namespace.

It is a good practice to run your Prometheus containers in a separate namespace, so let's create one:

```
kubectl create ns monitor
```

Then, proceed with the installation of the Prometheus operator:

```
helm install prometheus-operator stable/
```

```
prometheus-operator --namespace monitor
```

- “prometheus-operator” is the name of the release. You can change this if you want.
- “stable/prometheus-operator” is the name of the chart.
- “monitor” is the name of the namespace where we are going to deploy the operator.

You can verify your installation using:

```
kubectl get pods -n monitor
```

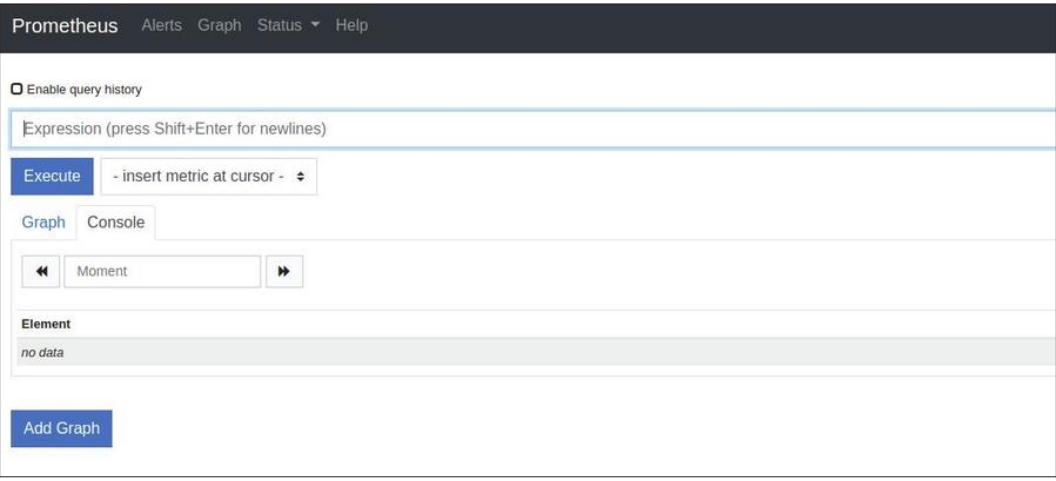
You should be able to see the Prometheus Alertmanager, Grafana, kube-state-metrics pod, Prometheus node exporters and Prometheus pods.

```
NAME                                                  READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-operator-alertmanager-0       2/2     Running   0          49s
prometheus-operator-grafana-5bd6cbc556-w9lds          2/2     Running   0          59s
prometheus-operator-kube-state-metrics-746dc6ccc-gk2p8 1/1    Running   0          59s
prometheus-operator-operator-7d69d686f6-wpjtd         2/2     Running   0          59s
prometheus-operator-prometheus-node-exporter-4nwbf    1/1     Running   0          59s
prometheus-operator-prometheus-node-exporter-jrw69    1/1     Running   0          59s
prometheus-operator-prometheus-node-exporter-rnqfc    1/1     Running   0          60s
prometheus-prometheus-operator-prometheus-0           3/3     Running   1          39s
```

Now that our pods are running, we have the option to use the Prometheus dashboard right from our local machine. This is done by using the following command:

```
kubectl port-forward -n monitor
prometheus-prometheus-operator-prometheus-0 9090
```

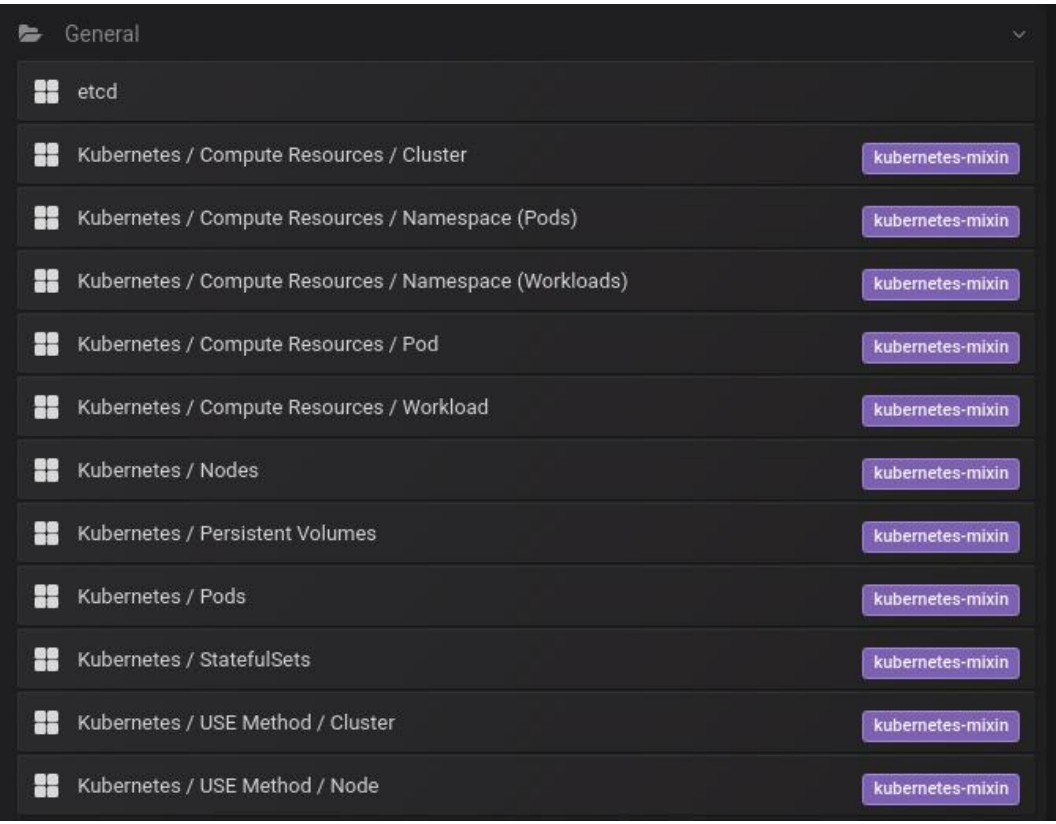
Now visit http://127.0.0.1:9090 to access the Prometheus dashboard.



Same as Prometheus, we can use this command to make the Grafana dashboard accessible:

```
kubectl port-forward $(kubectl get
pods --selector=app=grafana -n monitor
--output=jsonpath="{.items..metadata.name}")
-n monitor 3000
```

After visiting http://127.0.0.1:3000 you will be able to discover that there are some default preconfigured dashboards:



Note that you should use "admin" as the login and "prom-operator" as the password. Both can be found in a Kubernetes Secret object:

```
kubectl get secret --namespace monitor
grafana-credentials -o yaml
```

You should get a YAML description of the encoded login and password:

```
apiVersion: v1 data: password:
cHJvbS1vcGVyYXRvcgo= user: YWRtaW4=
```

You need to decode the username and the password using:

```
echo "YWRtaW4=" | base64 --decode echo
"cHJvbS1vcGVyYXRvcgo=" | base64 --decode
```

Using the "base64 --decode", you will be able to see the clear credentials.

3.3 Preconfigured Grafana dashboards for Kubernetes

By default, Prometheus operator ships with a preconfigured Grafana - some dashboards are available by default, like the one below:

Some of these default dashboards, are "Kubernetes / Nodes", "Kubernetes / Pods", "Kubernetes / Compute Resources / Cluster", "Kubernetes / Networking / Namespace (Pods)" and "Kubernetes / Networking / Namespace (Workload)", etc.

If you are curious, you can find more details about these dashboards here. For example, if you want to see how the "Kubernetes / Compute Resources / Namespace (Pods)" dashboard works, you should view this ConfigMap. For more on visualizations with Grafana, see chapter 5.

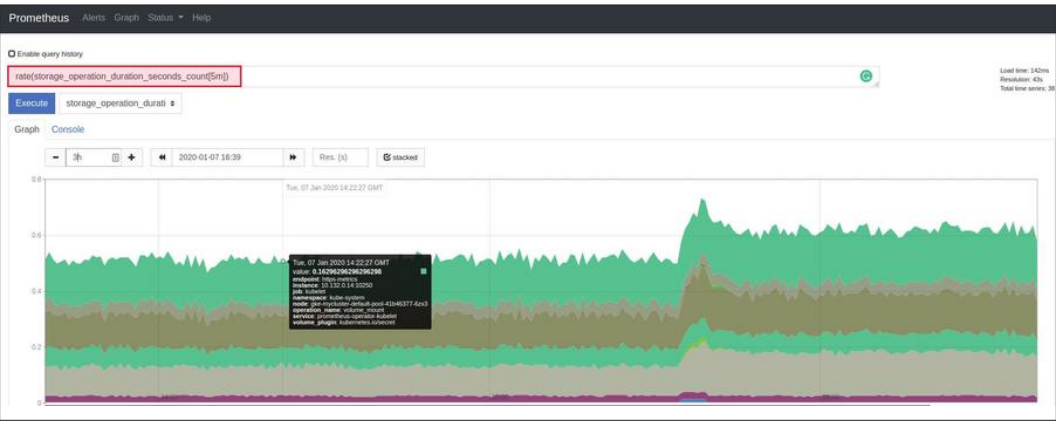
3.4 The metrics that are available as a result of the Helm install

To understand how Prometheus works, let's access the Prometheus dashboard. Use this port forwarding command:

```
kubectl port-forward -n monitor
prometheus-prometheus-operator-prometheus-0 9090
```

Then visit "http://127.0.0.1:9090/metrics".
You will be able to see a long list of metrics.

Prometheus uses PromQL (Prometheus Query Language), a functional query language, that lets the user select and aggregate time series data in real time. PromQL can be complicated especially if you start to comprehensively learn it, but you can start with these examples from the official documentation and you will be able to understand a good part of it. Also, check out chapter 8 for more information about PromQL.



When we deployed Prometheus using Helm, we used this chart, and it actually deployed not just Prometheus but also:

- prometheus-operator
- prometheus
- alertmanager
- node-exporter
- kube-state-metrics
- grafana
- service monitors to scrape internal kubernetes components
- kube-apiserver
- kube-scheduler
- kube-controller-manager
- etcd
- kube-dns/coredns
- kube-proxy

So in addition to Prometheus, we included tools like the service monitors that scrape internal system metrics and other tools like "kube-state-metrics".

kube-state-metrics will also export information that Prometheus server can read. We can see a list of these metrics by visiting "http://127.0.0.1:8080/metrics" after running:

```
kubectl port-forward -n monitor
prometheus-operator-kube-state-metrics-xxxx-xxx
8080
```

First Contact with Prometheus Exporters

4.1 Introduction

Creating Prometheus exporters can be complicated, but it doesn't have to be. In this chapter, we will learn the basics of exporters and we will walk you through two step-by-step guides showing implementations of exporters based on Python. The first part of this chapter is about third party exporters that expose metrics in a standalone way regarding the application they monitor. The second part is about exporters that expose built-in application metrics. Let's begin!

4.2 Quick overview of exporter-related Prometheus concepts

4.2.1 Pull approach of data collection

The pull approach of data collection consists of having the server component (Prometheus server) periodically retrieve metrics from client components. This pulling is commonly referred to as "scrape" in the Prometheus world. Because of scrape, the client components are only responsible for producing metrics and making them available for scraping.

Tools like Graphique, InfluxDB, and many others, use a push approach where the client component has to produce metrics and push them to the server component. Therefore, the client determines when to push the data regardless of whether the server needs it or whether it is ready to collect it.

The Prometheus pull approach is innovative because by requiring the server -- not the client -- to scrape, it collects metrics only when the server is up and running and when the data is ready. This approach requires that each client component enables a specific capability called Prometheus exporter.

4.2.2 Prometheus exporters

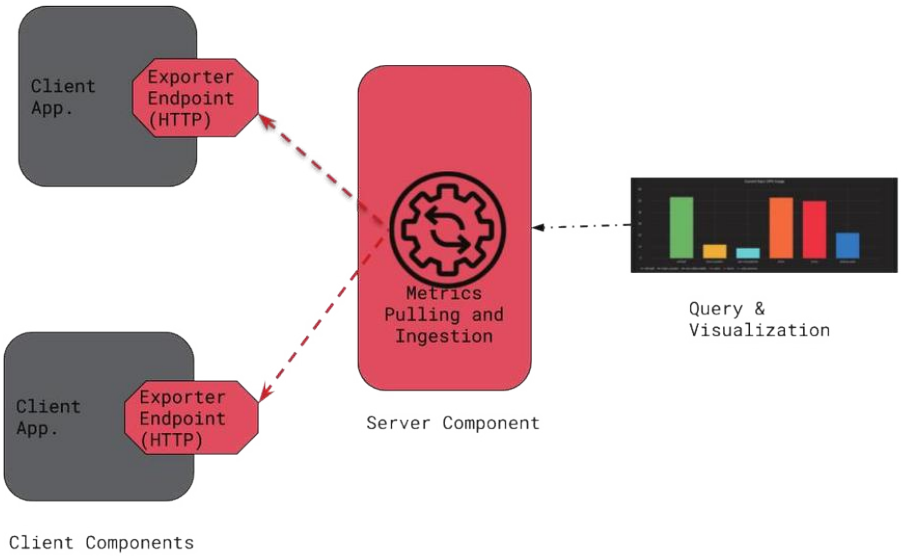
Exporters are essential pieces within a Prometheus monitoring environment. Each program acting as a Prometheus client holds an exporter at its core. An exporter is comprised of software features that produce metrics data, and an HTTP server that exposes the generated metrics available via a given endpoint. Metrics are exposed according to a specific format that the Prometheus server can read and ingest (scraping). We will discuss how to produce metrics, their format, and how to make them available for scraping later in this chapter.

4.2.3 Flexible visualization

Once metrics have been scraped and stored by a Prometheus server, there are various means to visualize them. The first and easiest approach is to use Prometheus Expression Browser.

However, due to its basic visualization capabilities, the Expression Browser is mainly helpful for debugging purposes (to check the availability or last values of certain metrics). For better and more advanced visualization, users often opt for other tools like Grafana. Furthermore, in some contexts users may have custom-made visualization systems that directly query Prometheus API to retrieve the metrics that need to be visualized.

Basic Architecture of a Prometheus environment with a server component, two client components and an external visualization system. The client components each have an Exporter Endpoint.



4.3 Implementing a Prometheus exporter

From an application perspective, there are two kinds of situations where you can implement a Prometheus exporter: for exporting

built-in application metrics, and exporting metrics from a standalone or third-party tool.

4.3.1 Application built-in exporter

This is typically the case when a system or an application exposes its key metrics natively. The most interesting example is when an application is built from scratch, since all the requirements that it needs to act as a Prometheus client can be studied and integrated through the design. Sometimes, we may need to integrate an exporter to an existing application. This requires updating the code -- and even the design -- to add the required capabilities to act as a Prometheus client. Integrating to an existing application can be risky because, if not done carefully, those changes may introduce regressions on the application's core functions. If you must do this, be sure to have sufficient tests in place in order not to introduce regressions into the application (e.g. bugs or performance overhead due to changes in code and/or in design).

4.3.2 Standalone/third-party exporter

Sometimes the desired metrics can be collected or computed externally. An example of this is when the application provides APIs or logs where metric data can be retrieved. This data can then be used as is, or it may need further processing to generate metrics (an example of this is the MySQL exporter).

You may also require an external exporter if the metrics need to be computed throughout an aggregation process by a dedicated system. As an example, think of a Kubernetes cluster where you

need to have metrics that show CPU resources being used by sets of pods grouped by labels. Such an exporter may rely on Kubernetes API, and works as follows:

(i) retrieve the current CPU usage along with the label of each individual pod
(ii) sum up the usage based on pod labels
(iii) make the results available for scraping

4.4 Examples of exporter implementation using Python

In this section we'll show step-by-step how to implement Prometheus exporters using Python. We'll demonstrate two examples covering the following metric types:

- **Counter**: represents a metric where the value can only increase over time; this value is reset to zero on restart. Such a metric can be used to export a system uptime (time elapsed since the last reboot of that system).

- **Gauge**: represents a metric where value can arbitrarily go up and down over time. It can be used to expose memory and CPU usage over time.

We will go through two scenarios: In the first one, we consider a standalone exporter exposing CPU and memory usage in a system. The second scenario is a Flask web application that exposes its request response time and also its uptime.

4.4.1 Standalone/third-party exporter exposing CPU and memory usage

This scenario demonstrates a dedicated Python exporter that periodically collects and exposes CPU and memory usage on a system.

For this program, we'll need to install the Prometheus client library for Python.

```
$ pip install prometheus_client
```

We'll also need to install psutil, a powerful library, to extract system resource consumption.

```
$ pip install psutil
```

Our final exporter code looks like this: (see source gist)

rchakode / prometheus_exporter_system_resource_usage.py Secret

Edit Delete Unsubscribe Star 0

Last active now

Code Revisions 5

Embed <script src="https://gist. Download ZIP

Sample Python program providing a Prometheus exporter that exposes system CPU and memory usage every 5 minutes.

prometheus_exporter_system_resource_usage.py Raw

```
import prometheus_client
import time
import psutil

UPDATE_PERIOD = 300
SYSTEM_USAGE = prometheus_client.Gauge('system_usage',
                                       'Hold current system resource usage',
                                       ['resource_type'])
if __name__ == '__main__':
  prometheus_client.start_http_server(9999)

while True:
  SYSTEM_USAGE.labels('CPU').set(psutil.cpu_percent())
  SYSTEM_USAGE.labels('Memory').set(psutil.virtual_memory()[2])
  time.sleep(UPDATE_PERIOD)
```

The code can be downloaded and saved in a file:

```
$ curl -o prometheus_exporter_cpu_memory_
usage.py \
-s -L https://git.io/Jesvq
```

The following command then allows you to start the exporter:

```
$ python ./prometheus_exporter_cpu_memory_
usage.py
```

We can check exposed metrics through a local browser: http://127.0.0.1:9999. The following metrics, among other built-in metrics enabled by the Prometheus library, should be provided by our exporter (values may be different according to the load on your computer).

```
# HELP response_time_last Hold the last request response time
# TYPE response_time_last gauge
response_time_last 0.0012016449999805445
# HELP service_uptime Hold the time elasted since service startup
# TYPE service_uptime gauge
service_uptime 4.0
```

Simple, isn't it? This is in part due to the magic of Prometheus client libraries, which are officially available for Golang, Java, Python and Ruby. They hide boilerplates and make it easy to implement an exporter. The fundamentals of our exporter can be summarized by the following entries (where the line numbers refer to the code in the source Gist linked/imaged above):

- Import the Prometheus client Python library (line 1).
- Instantiate an HTTP server to expose metrics on port 9999 (line 10).
- Declare a gauge metric and name it system_usage (line 6).
- Set values for metrics (lines 13 and 14).
- The metric is declared with a label (resource_type, line 6), leveraging the concept of multi-dimensional data model. This lets you hold a single metric name and use labels to differentiate CPU and memory metrics. You may also choose to declare two metrics instead of using a label. Either way, we highly recommend that you read the best practices about metrics naming and labels.

4.4.2 Exporter for a Flask application

This scenario demonstrates a Prometheus exporter for a Flask web application. Unlike with a standalone, an exporter for a Flask web application has a WSGI dispatching application that works as a gateway to route requests to both Flask and Prometheus clients. This happens because the HTTP server enabled by Flask cannot be used consistently to also serve as Prometheus client. Also, the HTTP server enabled by the Prometheus client library would not serve Flask requests.

To enable integration with a WSGI wrapper application, Prometheus provides a specific library method (make_wsgi_app) to create a WSGI application to serve metrics.

The following example (source gist) -- a Flask hello-world application slightly modified to process requests with random response times -- shows a Prometheus exporter working alongside

a Flask application. (see hello method at line 18). The Flask application is accessible via the root context (/ endpoint), while the Prometheus exporter is enabled through /metrics endpoint (see line 23, where the WSGI dispatching application is created). Concerning the Prometheus exporter, it exposes two metrics:

- Last request response time: this is a gauge (line 10) in which, instead of using the set method like in our former example, we introduced a Prometheus decorator function (line 17) that does the same job while keeping the business code clean.

- Service uptime: this is a counter (line 8) that exposes the time elapsed since the last startup of the application. The counter is updated every second thanks to a dedicated thread (line 33).

```
1  import prometheus_client
2  import werkzeug.wsgi
3  import flask
4  import random
5  import time
6  import threading
7
8  SERVICE_UPTIME = prometheus_client.Gauge('service_uptime',
9                                           'Hold the time elasted since service startup')
10 RESPONSE_TIME = prometheus_client.Gauge('response_time_last',
11                                          'Hold the last request response time')
12
13 # Create Flask app
14 app = flask.Flask(__name__)
15
16 @app.route('/')
17 @RESPONSE_TIME.time()
18 def hello():
19     time.sleep(random.random())
20     return "Hello World!"
21
```

```
22  # Prometheus wsgi middleware to route /metrics requests
23  app_dispatch = werkzeug.wsgi.DispatcherMiddleware(app, {
24    '/metrics': prometheus_client.make_wsgi_app()
25  })
26
27  def update_uptime():
28    while True:
29      SERVICE_UPTIME.inc(1)
30      time.sleep(1)
31
32  SERVICE_UPTIME.set(0)
33  uptime_updater = threading.Thread(target=update_uptime)
34  uptime_updater.start()
```

So that the program works, we need to install additional dependencies:

```
$ pip install uwsgi
```

Then, the program needs to be launched using WGSI:

```
$ curl -o prometheus_exporter_flask.py \
-s -L https://git.io/Jesvh
```

Now we need to start the service as a WSGI application:

```
$ uwsgi --http 127.0.0.1:9999 \
--wsgi-file prometheus_exporter_flask.py \
--callable app_dispatch
```

Note that the --wsgi-file shall point to the Python program file while the value of the --callable option must match the name of the WSGI application declared in our program (line 23).

Once again, you can check exposed metrics through a local browser: http://127.0.0.1:9999/metrics. Among other built-in metrics exposed by the Prometheus library, we should find the following ones exposed by our exporter (values may be different according to the load on your computer):

```
# HELP response_time_last Hold the last request response time
# TYPE response_time_last gauge
response_time_last 0.0012016449999805445
# HELP service_uptime Hold the time elasted since service startup
# TYPE service_uptime gauge
service_uptime 4.0
```

Here we go! Our different exporters are now ready to be scraped by a Prometheus server. You can learn more about this here.

4.5. Conclusion

In this chapter we first discussed the basic concepts of Prometheus exporters and then went through two documented examples of implementation using Python. Those examples leverage Prometheus's best practices so that you can use them as a starting point to build your own exporters according to your specific application needs.

Visualizations: An Overview

5.1 Introduction

When it comes to dashboarding and visualizations with Prometheus, you have three options. Prometheus Expression browser, Prometheus Console Templates, and Grafana. Both come packaged with Prometheus in the initial download. Expression Browser is used for quick queries and flexible investigations into the data. For graphs and dashboards, Grafana is the best available dashboard software for time series data. We'll take a look at each one below.

5.2 Prometheus Expression Browser

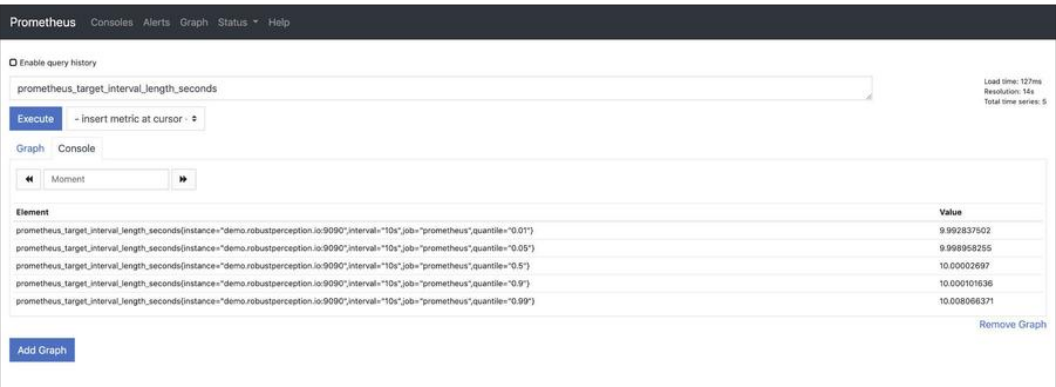
The Prometheus Expression Browser provides an efficient way for you to display time series metrics collected by the Prometheus server. It is part of the Prometheus suite and can be accessed using the endpoint */graph*. Using this tool, you can efficiently visualize time series metrics by either displaying them on a graph or in a tabular manner depending on your preferences.

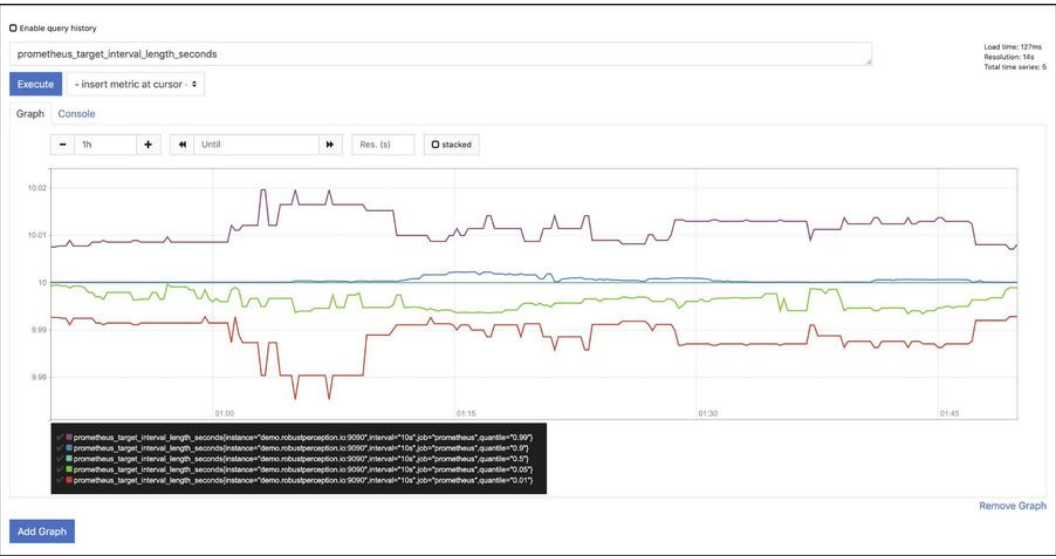
Given the sheer amount of data collected by Prometheus, it can be confusing to know what metrics you should keep an eye on, with some people trying to represent every piece of data they have,

cramping up the console. The Expression Browser allows you to specify which exact metric you wish to display by typing in the metric name into the *Expression* field.

You can also control the Prometheus Expression Browser using PromQL. PromQL is the language with which you query in Prometheus Expression Browser, and you can also manipulate visualizations using PromQL. Find out more about PromQL in chapter 8.

When you hit the *Execute* button, data relating to this particular metric, provided it exists, is provided both in a table (on the Console Tab) and a graph (on the Graph Tab), allowing you to switch between these two with just a single click. For example, say we want to pull up data on the `prometheus_target_interval_length_seconds` metric. This metric measures the amount of time between target scrapes. In other words, the amount of time between data collection from a prometheus target. Entering this metric into the Expression Browser yields the following results:





You can visit the */metrics* endpoint to get a list of all the time series data metrics being monitored by Prometheus. You could have multiple graphs open on the Expression Browser at a time, but it's best practice to keep it at a bare minimum, only monitoring data that is essential to accomplish your goal. Flooding the page with irrelevant graphs/lines on a graph can cause you to lose focus on what is important and thus, miss important signals.

5.3 Prometheus console templates

There exists another option called Prometheus Console templates. Prometheus console templates allow for the creation of arbitrary consoles using the Go templating language. Basically you can build your dashboard console ahead of time by specifying code instructions on how/what you want your console to look like and what functionalities it should carry out. These are then served from the Prometheus server.

5.4 Grafana

Using Prometheus and Grafana together is a great combination of tools for monitoring an infrastructure. In this article, we will discuss how Prometheus can be connected with Grafana and what makes Prometheus different from the rest of the tools in the market.

Grafana is a very versatile visualization tool. It is able to read data from a variety of data sources, and plot with visualization options such as graphs, gauges, world maps, heatmaps etc. The next chapter will go into further detail about setting up Grafana with Prometheus. For information about designing the best Grafana dashboards, check out the MetricFire blog.

Connecting Prometheus and Grafana

6.1 Introduction

In this chapter we will visualize information from cAdvisor and Redis by processing the data using Prometheus, and then visualize it on Grafana. We will use docker to set up a test environment for Grafana and Prometheus. We will use the official docker images for Grafana and Prometheus available on Docker Hub. We will also need to use the docker images for cAdvisor and Redis. cAdvisor is a tool by google which collects metrics about running containers and exposes the metrics in various formats, including Prometheus formatting.

6.2 Configuring cAdvisor to collect metrics from Redis

First, we will configure cAdvisor to collect metrics from the Redis container and visualize it in Grafana.

```
version: '3.2'
services:
 prometheus:
   image: prom/prometheus
```

```
  ports:
    - "9090:9090"
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml

grafana:
  image: grafana/grafana
  ports:
  - "3000:3000"

cadvisor:
  image: google/cadvisor:latest
  container_name: cadvisor
  ports:
  - 8080:8080
  volumes:
  - /:/rootfs:ro
  - /var/run:/var/run:rw
  - /sys:/sys:ro
  - /var/lib/docker/:/var/lib/docker:ro
  depends_on:
  - redis

redis:
  image: redis:latest
  container_name: redis
  ports:
  - 6379:6379
```

We will also create a default prometheus.yml file along with docker-compose.yml. This configuration file contains all the configuration related to Prometheus. The config below is the default configuration which comes with Prometheus.

```
# my global config
global:
 scrape_interval:     15s # Set the scrape interval to
every 15 seconds. Default is every 1 minute.
 evaluation_interval: 15s # Evaluate rules every 15
seconds. The default is every 1 minute.
 # scrape_timeout is set to the global default (10s).

# Alertmanager configuration
alerting:
 alertmanagers:
 - static_configs:
   - targets:
     # - alertmanager:9093

# Load rules once and periodically evaluate them according
to the global 'evaluation_interval'.
rule_files:
 # - "first_rules.yml"
 # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to
scrape:
# Here it's Prometheus itself.
```

```
scrape_configs:
 # The job name is added as a label `job=<job_name>` to
any timeseries scraped from this config.
 - job_name: 'prometheus'

   # metrics_path defaults to '/metrics'
   # scheme defaults to 'http'.

   static_configs:
   - targets: ['localhost:9090']
```

We can see the metrics of the Redis container by going to http://localhost:8080/docker/redis

The screenshot below shows the information that cAdvisor is able to collect from Redis.

redis (/docker/ea5dd03e5e13d4ff28c26310b5525308b92f37a0f8142998bd8d96015a1248e7)

Docker Containers / redis (/docker/ea5dd03e5e13d4ff28c26310b5525308b92f37a0f8142998bd8d96015a1248e7)

Isolation

CPU
Shares 1024 *shares*
Allowed Cores 0 1
Memory
Reservation unlimited
Limit unlimited
Swap Limit unlimited

Now these metrics from cAdvisor need to be fed into Prometheus. To do this we will modify the prometheus.yml as below:

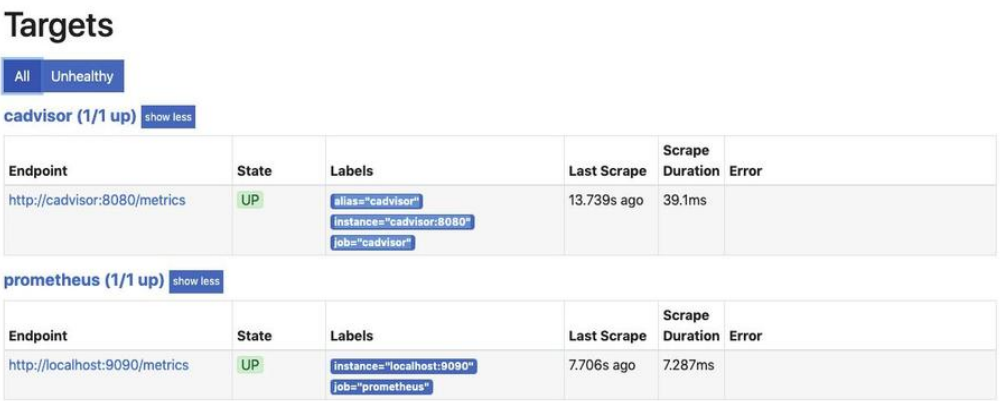
```
global:
 scrape_interval:     15s # Set the scrape interval to
every 15 seconds. Default is every 1 minute.
 evaluation_interval: 15s # Evaluate rules every 15
seconds. The default is every 1 minute.
 # scrape_timeout is set to the global default (10s).
```

```
# Alertmanager configuration
alerting:
 alertmanagers:
 - static_configs:
   - targets:
     # - alertmanager:9093

# Load rules once and periodically evaluate them according
to the global 'evaluation_interval'.
rule_files:
 # - "first_rules.yml"
 # - "second_rules.yml"

# A scrape configuration containing exactly one endpoint to
scrape:
# Here it's Prometheus itself.
scrape_configs:
 # The job name is added as a label `job=<job_name>` to
any timeseries scraped from this config.
 - job_name: 'prometheus'

   # metrics_path defaults to '/metrics'
   # scheme defaults to 'http'.

   static_configs:
   - targets: ['localhost:9090']
  - job_name: 'cadvisor'
   static_configs:
   - targets: ['cadvisor:8080']
```

```
    labels:
      alias: 'cadvisor'
```

Note that we have added a new job called cAdvisor. Prometheus will now periodically pull the metrics from the cAdvisor. To reflect the changes in the Prometheus configuration file, we need to restart it with "docker-compose restart prometheus".

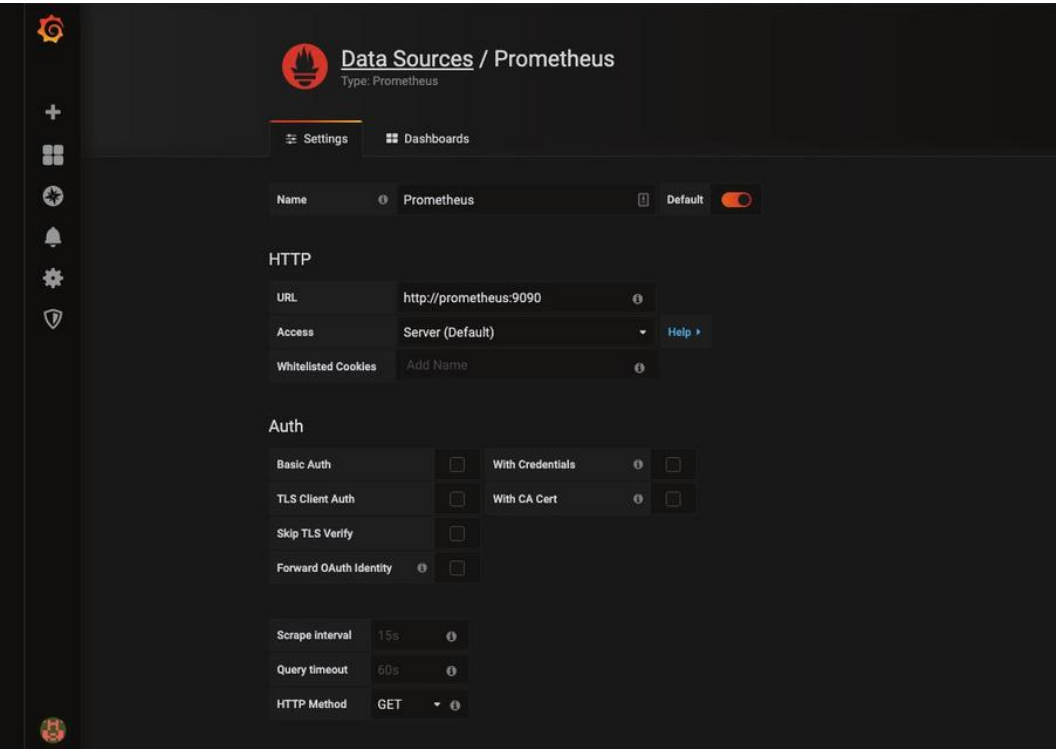
We should be able to see two jobs in Prometheus web ui at http://localhost:9090/targets. The screenshot below shows the two jobs, one for cAdvisor and other for Prometheus itself.



6.3 Making Prometheus a datasource in Grafana

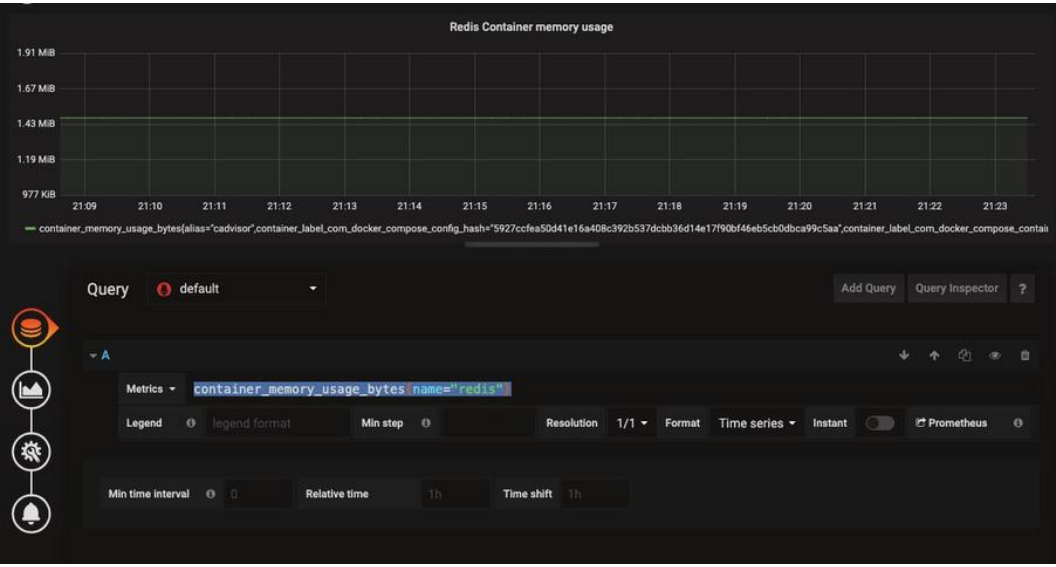
Now that we are able to feed our container metrics into Prometheus, it's time to visualize it in Grafana. Browse to http://localhost:3000 and login using admin/admin and add the datasource for Prometheus as shown below:

Note: the URL will be http://prometheus:9090 if you are using docker as described in this article. This is because we want Grafana to connect to Prometheus from the backend (where it says Access: Server) rather than the browser frontend. For the Grafana container, the location of Prometheus is http://prometheus:9090 and not http://127.0.0.1:9090 as you might expect.



Now let's create a simple Grafana dashboard and add a simple graph. This is fairly straightforward. The tricky part is configuring the data source and providing the query.

We will make a visualization of the Redis container memory usage from the Prometheus data source. In the query dropdown box, choose Prometheus as the data source and we will use container_memory_usage_bytes{name="redis"} as the metric as shown below:



6.4 Conclusion

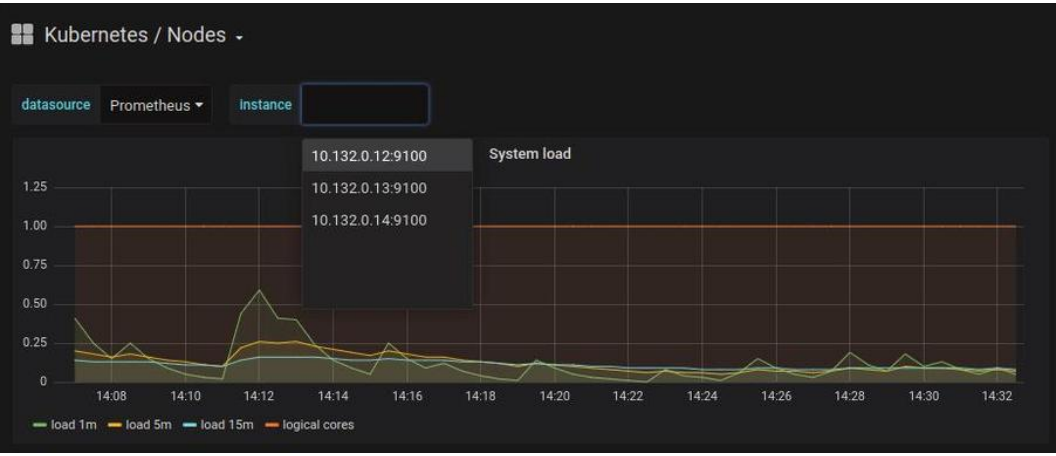
We have seen that Grafana provides a seamless way to connect to the Prometheus data source and it provides great visualization through queries.

Important Metrics to Watch in Production

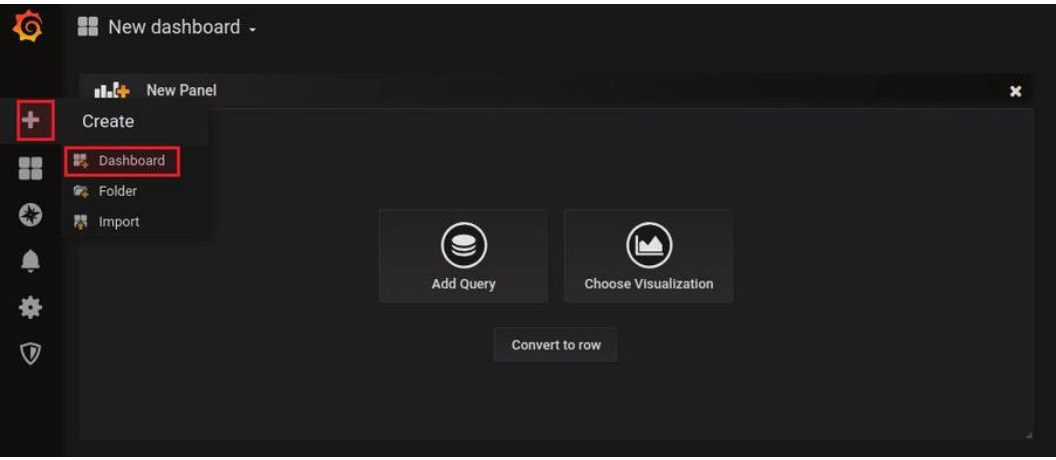
Now that we have Prometheus set up as a Datasource for Grafana - what metrics should we watch, and how do we watch them?

There is a large selection of default dashboards available in Grafana. Default dashboard names are self-explanatory, so if you want to see metrics about your cluster nodes, you should use "Kubernetes / Nodes". The dashboard below is a default dashboard:

Unless you are using minikube or one of its alternatives, a Kubernetes cluster usually runs more than one node. You must make sure that you are monitoring all of your nodes by selecting them one at a time:



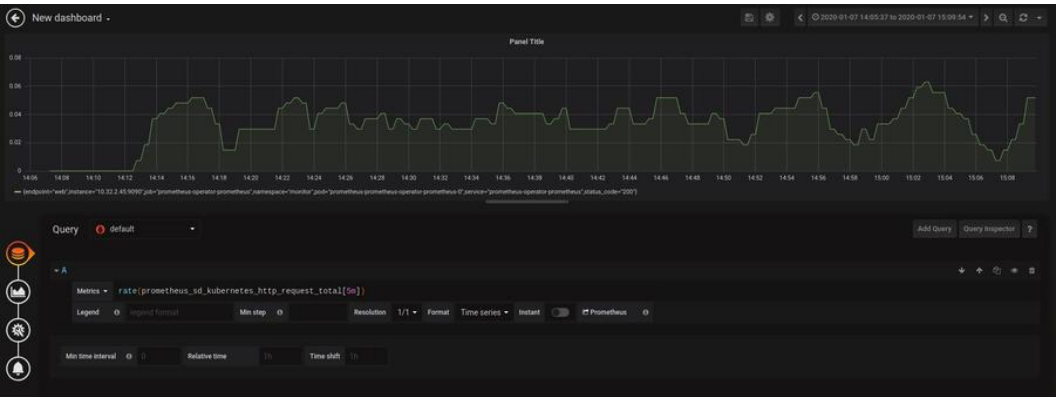
It is possible to add your own dashboards using a similar ConfigMap manifest, or directly using the Grafana dashboard interface. You can see below the “create dashboard” UI:



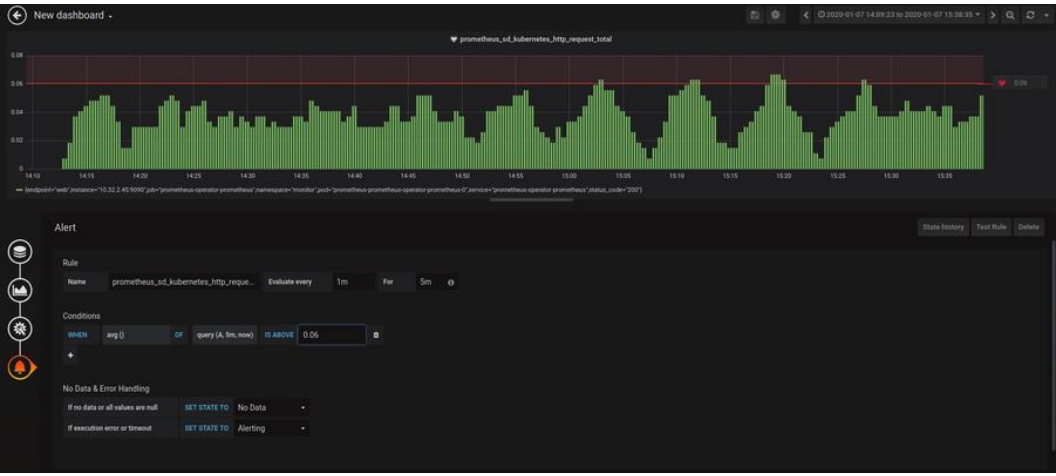
When creating a new custom dashboard, you will be asked whether you want to add a query or use visualization. If you choose visualization, you will be asked to choose a type of graph and a datasource. We can choose Prometheus as our data source. We can also choose to add a query using Prometheus as a data source.

As an example, we can set up a graph to watch the prometheus_sd_kubernetes_http_request_total metric, which gives us information about the number of HTTP requests to the Kubernetes API by endpoint.

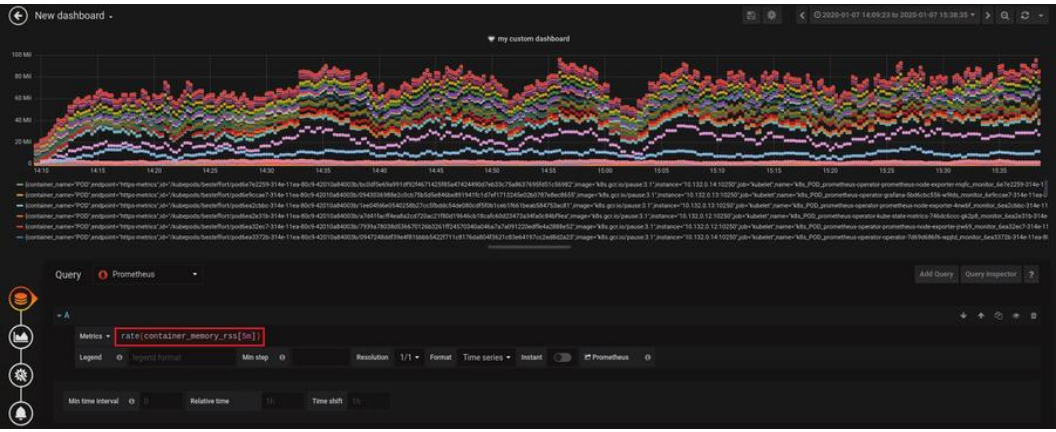
Since this metric is cumulative, Grafana asked us if we would rather use the function rate(), which gives us better information about our metric.



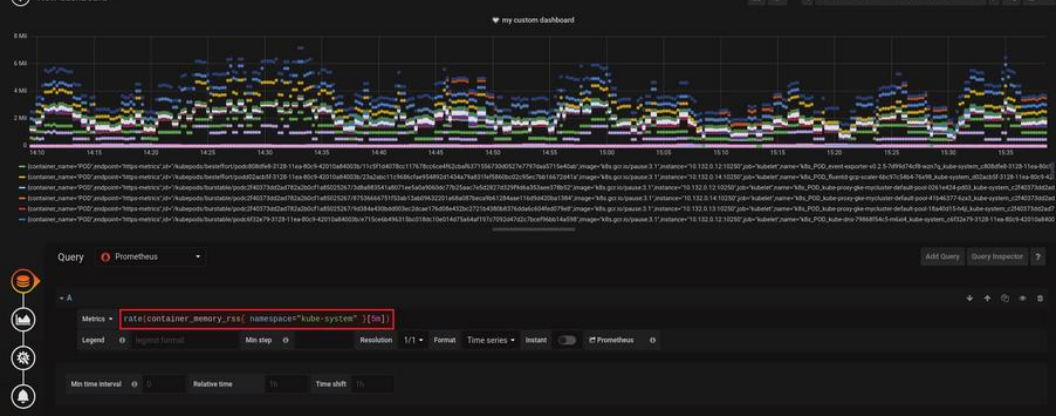
After setting up the name of the visualization, you can set alerts. Let's say that the average of the metric we chose should not exceed "0.06":



Let's choose a more significant metric, like the Resident Set Size (RSS) of Kubernetes system containers (RSS is the portion of memory occupied by a process that is held in main memory (RAM)). See the dashboard below:



Sometimes, some views are overpopulated, and in our example, since we want to monitor the system containers, we can refine our dashboard by filtering by namespace:



Let's try a third important metric to watch, like the total number of restarts of our container. You can access this information using kube_pod_container_status_restarts_total or kube_pod_container_status_restarts_total{namespace="<namespace>"} for a specific namespace.

There are many other important metrics to watch in production. Some of them are common and related to Nodes, Pods, Kube API, CPU, memory, storage and resources usage in general. These are some examples:

- kube_node_status_capacity_cpu_cores : describes nodes CPU capacity
- kube_node_status_capacity_memory_bytes : describes nodes memory capacity
- kubelet_running_container_count : returns the number of currently running containers
- cloudprovider_*_api_request_errors : returns the cloud provider API request errors (GKE, AKS, EKS, etc.)
- cloudprovider_*_api_request_duration_seconds: returns the request duration of the cloud provider API call in seconds

In addition to the default metrics, our Prometheus instance is scraping data from kube-apiserver, kube-scheduler, kube-controller-manager, etcd, kube-dns/coredns, and kube-proxy, as well as all the metrics exported by "kube-state-metrics". The amount of data we can get is huge, and that's not necessarily a good thing.

The list is long but the important metrics can vary from one context to another, you may consider that Etcd metrics are not important for your production environment while it can be for someone else.

However, some abstractions and observability good practices like Google Golden Signals or the USE method, can help us to choose

what metrics we should watch. Take a look at this article on the MetricFire blog for a guide on how to adhere to Google's four golden rules of observability when monitoring with Prometheus.

Getting Started with PromQL

8.1 Introduction

This chapter will focus on how to use PromQL. Prometheus uses Golang and allows simultaneous monitoring of many services and systems. In order to enable better monitoring of these multi-component systems, Prometheus has strong built in data storage and tagging functionalities. To use PromQL to query these metrics you need to understand the architecture of data storage in Prometheus, and how the metric naming and tagging works.

The native query language PromQL allows us to select the required information from the received data set and perform analysis and visualizations with that data using Grafana dashboards. The Alertmanager generates notifications from this data and sends them to us using methods such as e-mail, PagerDuty, and more.

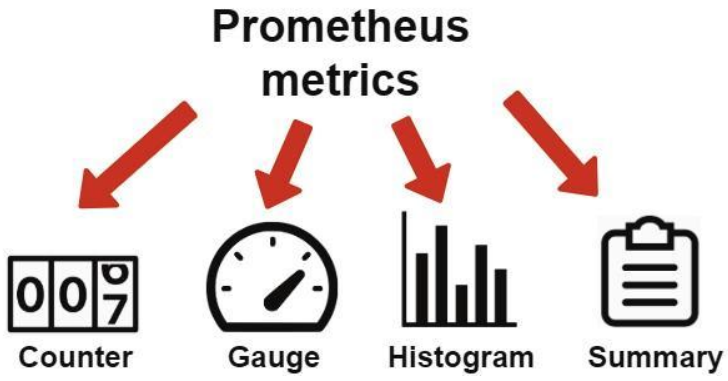
This article will go through the the data storage architecture, and then outline 10 examples of how to use PromQL. The examples will be routed in the theory laid out in parts 8.2 and 8.3.

8.2 Prometheus data architecture

Data representation in Prometheus depends on two factors: the model of data representation and the metrics type. To understand the processes of monitoring and the correct programming of PromQL commands, we'll briefly review these terms and their application in Prometheus.

Let's start with the Prometheus metrics. Their calculation and processing is an important and essential stage of monitoring. In general, metrics are numerical indicators which describe the state of a monitored object. A simple example of metrics for monitoring a remote server would be resource monitoring, such as the processor, memory, disk, network, etc. Similarly, it is possible to monitor web resource statistics such as visitor numbers, server response time or key requests.

Let's take a look at the different kinds of Prometheus Metrics:



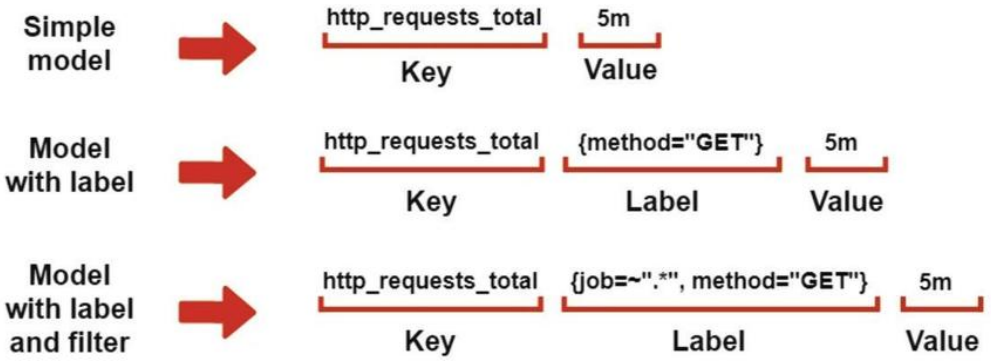
- **Counter** – accepts and stores only those values that will increase with time.
- **Gauge** – stores the values where the value can both increase and decrease.
- **Histogram** – samples observations (usually things like request durations or response sizes) and counts them in configurable buckets. It also provides a sum of all observed values, allowing you to calculate averages.
- **Summary** – histogram with a more detailed data representation using additional statistics (quantiles).

An important requirement for the proper and accurate display of metrics is the selection of the corresponding data type. For example, Gauge does not always correctly show the evolution of the metrics over the period. For this purpose, you should use the Histogram metrics. And the choice between Histogram and Summary comes down to the data presentation needs: either for a time period, or a continuous evolution in time.

Now let's proceed to Prometheus's metrics structuring system. Each metric has a Metric Name, which should express the aspect of the system that's being measured. Optionally, we can also assign a Label to each metric. Adding a Label creates a subset of time-series within the larger group defined by the Metric Name. Each Metric Name and label is paired with a Value, where the Value is the actual numerical data point sent from the system. Metric Names, Labels, and Values are stored as a set, and we use this information to query the time-series info.

- the Metric Name describes the system feature that we will measure
- the Label represents the more specific group within the time-series data
- the Value is the numerical data point sent from the feature we're monitoring

Below we will present this model as a chart.



So, we can configure the model with greater flexibility by using Labels and filtering the data. Filtering is the process of choosing which Labels you want to include in your query. Generally, this allows for fast and simple data aggregation in Prometheus.

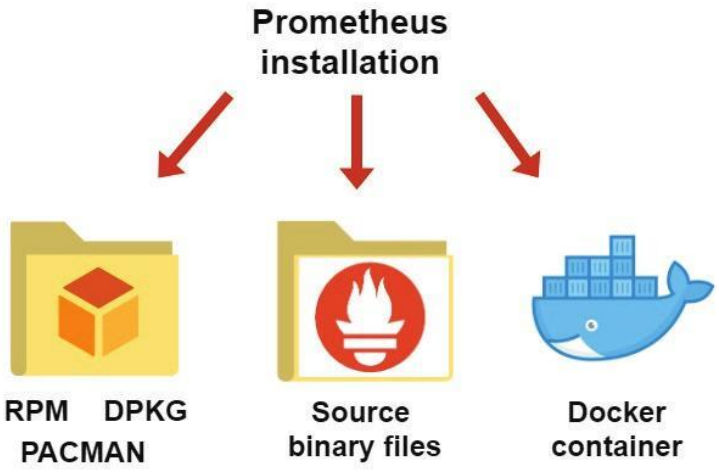
More detailed information about Prometheus is available in the official documentation.

8.3 Installation and configuration aspects affecting PromQL queries

Prometheus' architecture allows us to integrate it into almost any platform quite simply. The basic installation procedure consists of deploying the Prometheus server, configuring exporters, and configuring client libraries. The configuration affects how we use PromQL and what we are able to monitor.

For a brief overview, we'll take a look at the most important points in this section. The Prometheus components have ready-to-compile binary files and require only internal program dependencies and libc support. Therefore, the Prometheus monitoring system will run on almost any platform: on different server systems, as well as on user computers with Linux or even Windows.

There are three ways of installation:



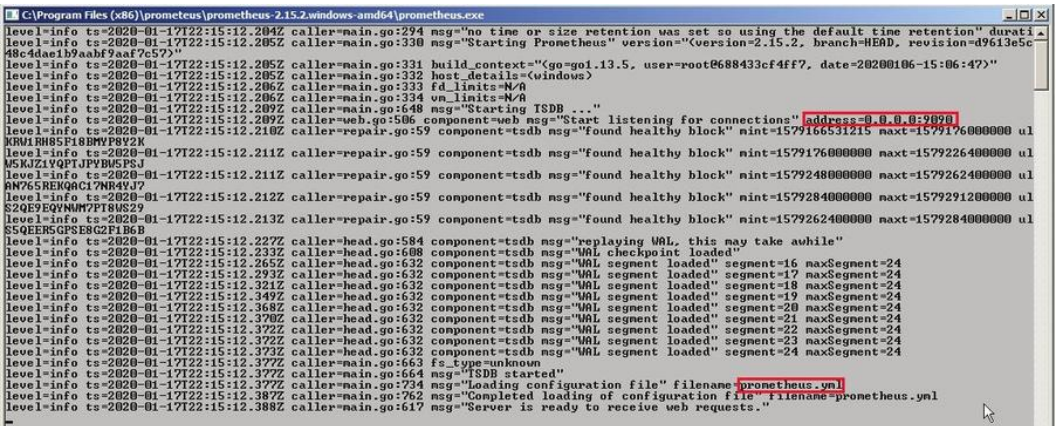
- With a package manager (only for Linux).
- Using binary source files from the official website (for all available platforms).
- Deploying the installed system in the Docker container.

The first method is the simplest - find the package, and start the installation. However, we should remember that Linux repositories often don't contain the latest software versions. The second way is the most complicated from the user's side, but it allows us to customize all the components of the monitoring system. The method using Docker containers is convenient at the stage of deployment on a remote server or some cloud platform.

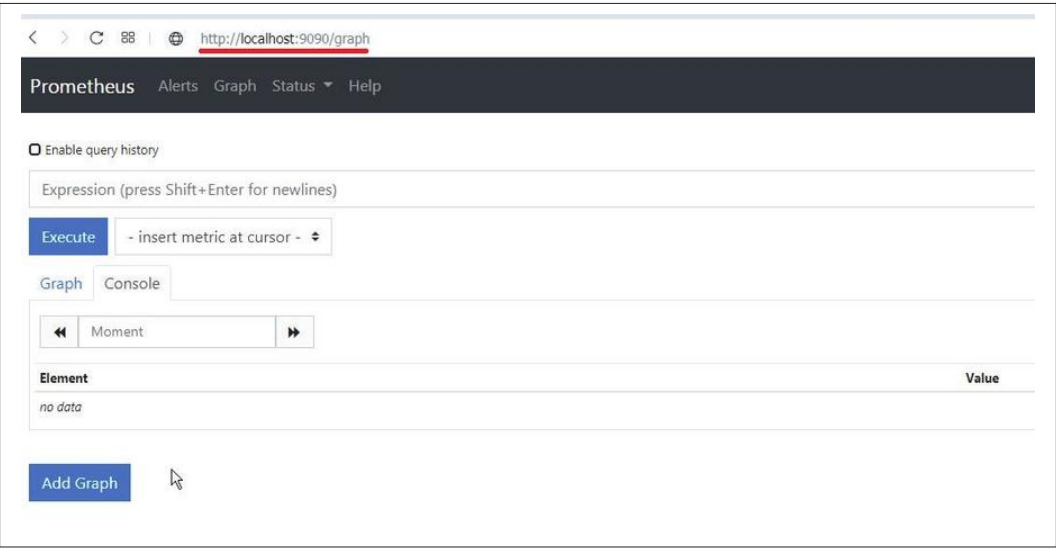
After the installation procedure, we need to configure Prometheus by defining the options in the prometheus.yml file. The main settings are presented below:

- scrape interval – time interval for scrapping
- evaluation interval – a time interval to compare with rules
- scrape configs – define and set monitoring targets
- rule files – path to monitoring rules

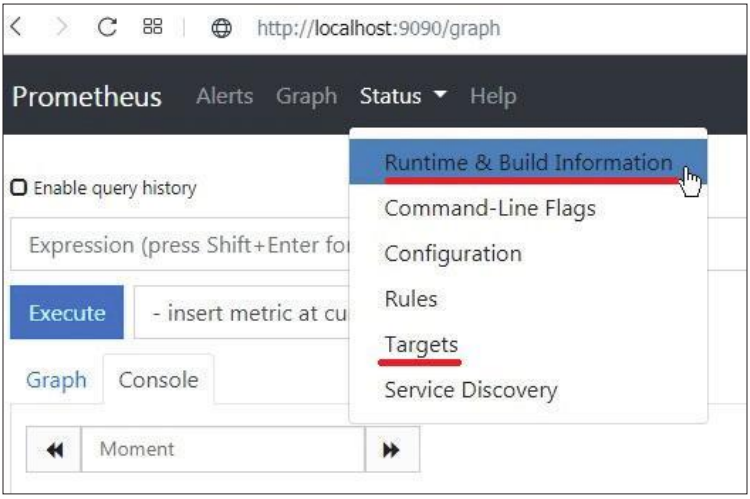
The specific values of these parameters depend on the purpose and targets of the monitoring. In this article, we will use Prometheus to monitor local computer resources on Windows. Next, let's choose a source binary file that matches the system requirements and run Prometheus with the version 2.15.2.

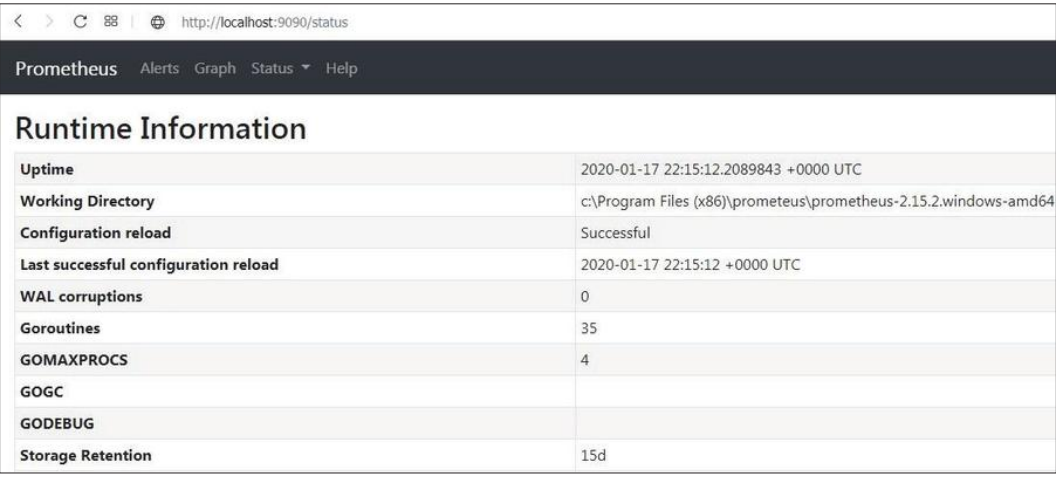


The terminal provided us with logs for the successful running of the monitoring system, along with the address (in this case, the localhost) and access port to the Prometheus server web interface. Then we will use the browser to monitor the local system.

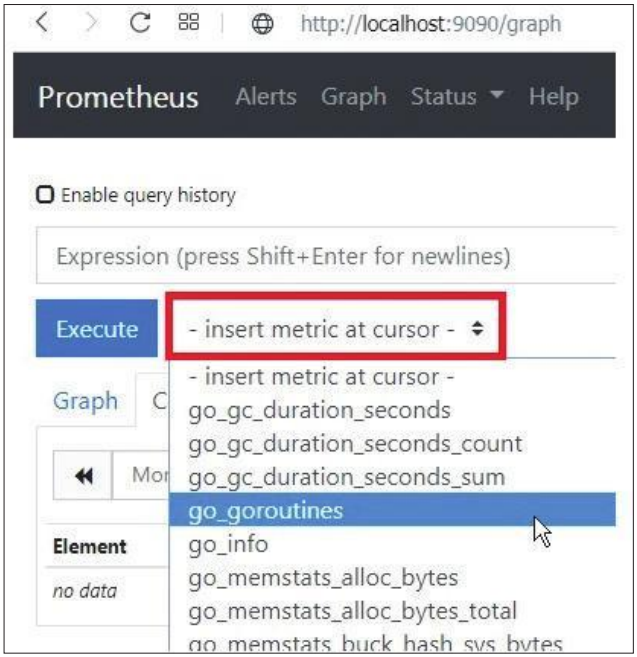


The Web interface provides access to the local Prometheus server and the associated parameters and monitoring options. It allows us to see the connected modules and the overall status of the server.





Also, this interface supports using predefined metrics and custom queries with PromQL.

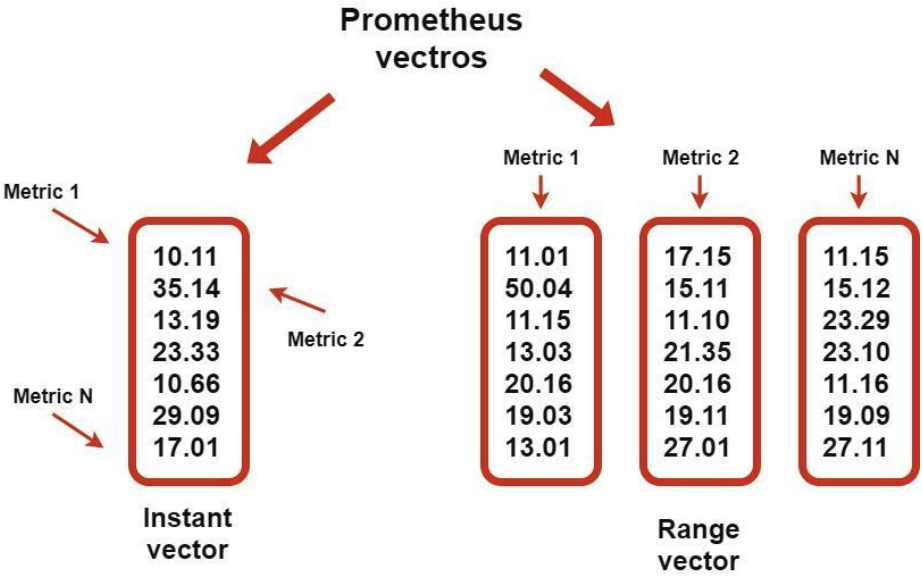


Let's explore the methods of writing queries using PromQL below.

8.4 Prometheus querying with PromQL: 10 examples

PromQL is a language for creating queries and extracting or filtering data from the Prometheus server. It uses the Prometheus data representation model in the form of "Key&Value" and returns the results as vectors.

Prometheus can return vectors of two types:

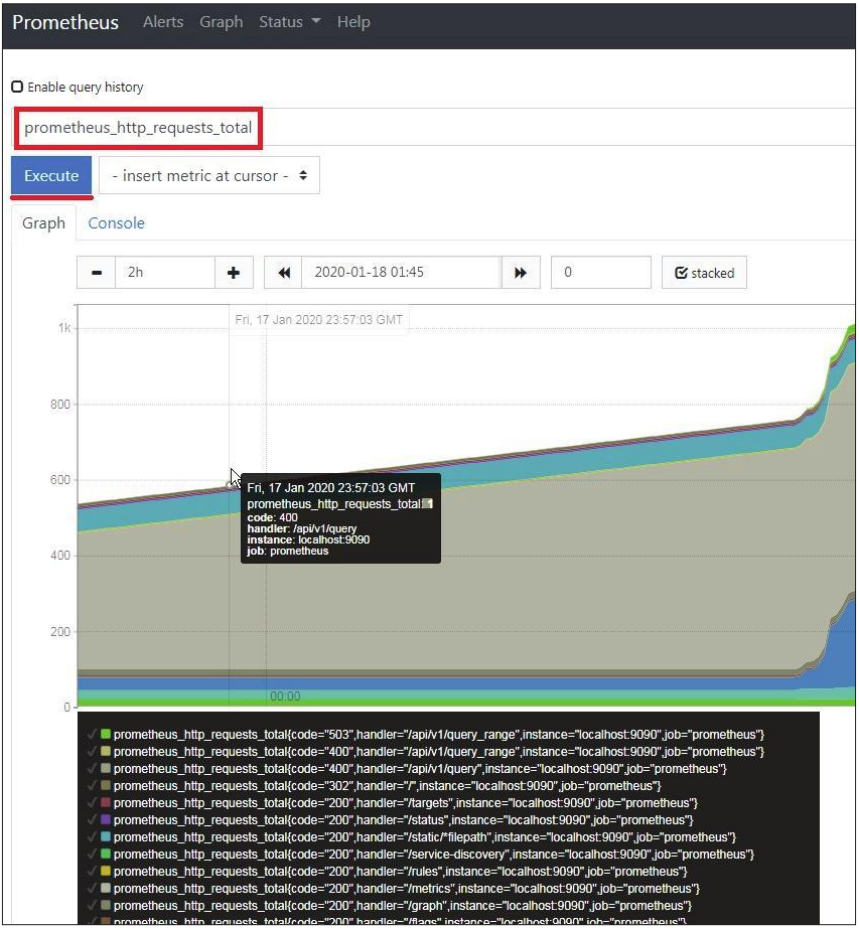


- Instant vectors – output of the requested values of all metrics from within the last time interval.
- Range vectors – represents values of several metrics as a set of vectors that is calculated for the selected period of time.

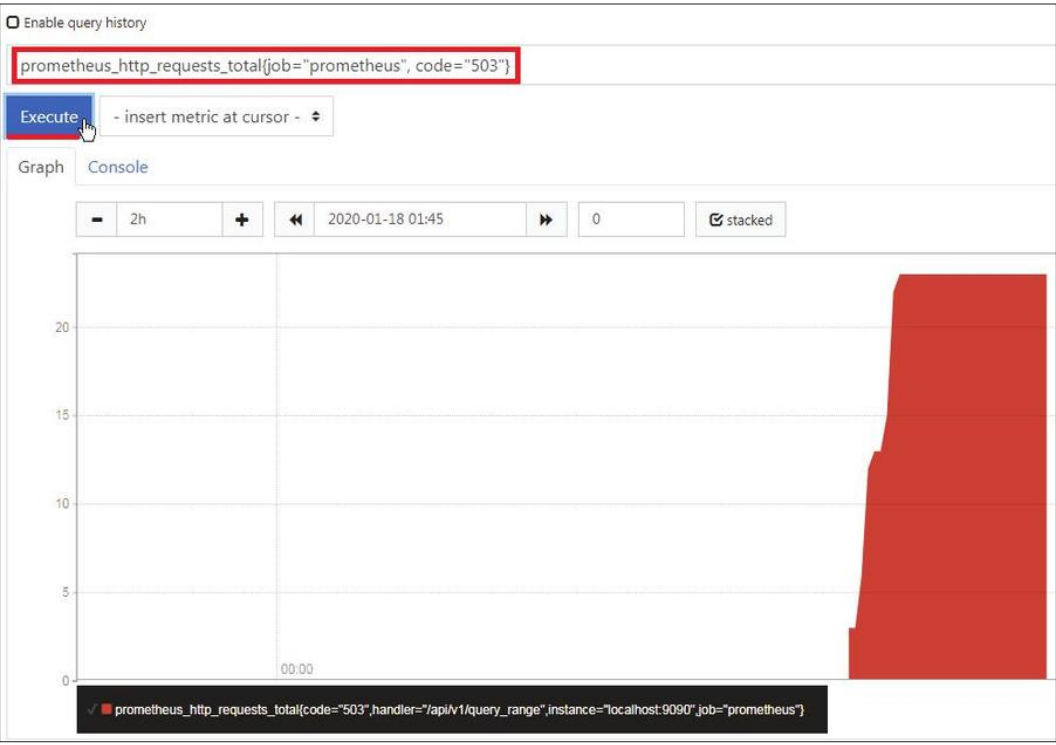
Choosing which type of vector to use depends on the metric you are requesting and the result you are looking for. Once you have

requested to see the metric, the value of these vectors appears as graphics in the built-in web interface. We'll demonstrate this with ten simple examples of such requests.

1. Let's show the value of the all-time time series with the counter **metric prometheus_http_requests_total**. This will show us all of the data denoted by this metric name, or "key".

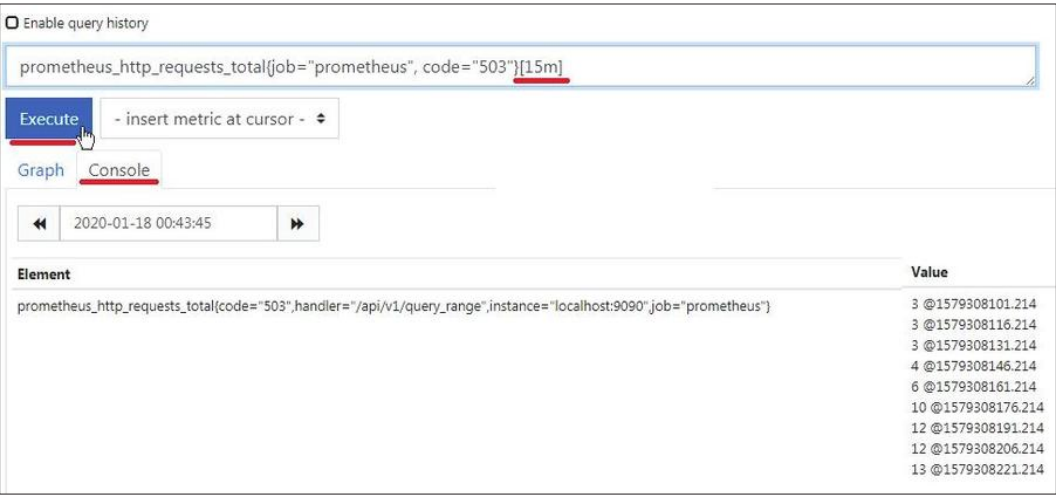


2. Now, let's take a look at the same metric as in example 1, but add some labels. Let's add the values for the labels "job" and "code".



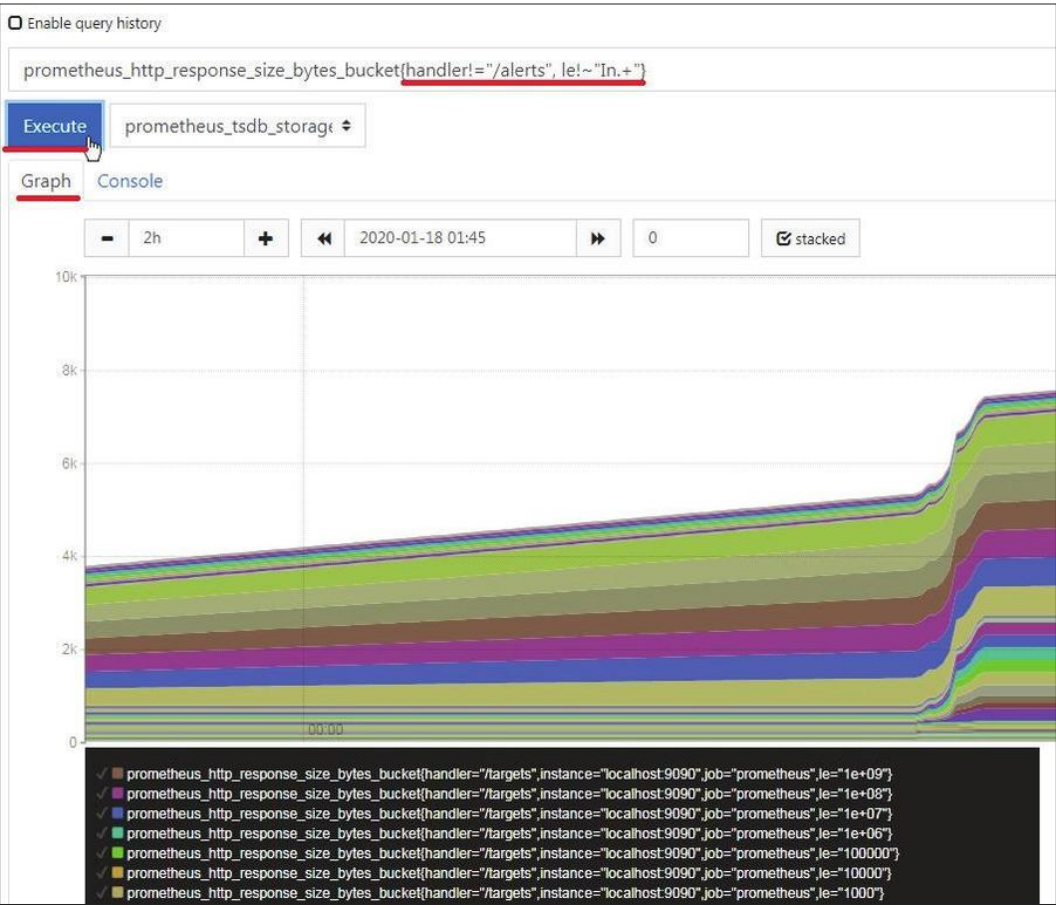
The result of this query is a chart or dashboard. We can also filter the essential data by using the time and date criteria on the graph interface.

3. Let's take a look at the metric from example 1 again, but let's add the time interval component. In the example below, we add the interval component by writing [15m] at the end of the query. The result will be displayed on the console tab in the web interface, and not on the graph tab. You will see the result as a list of values.



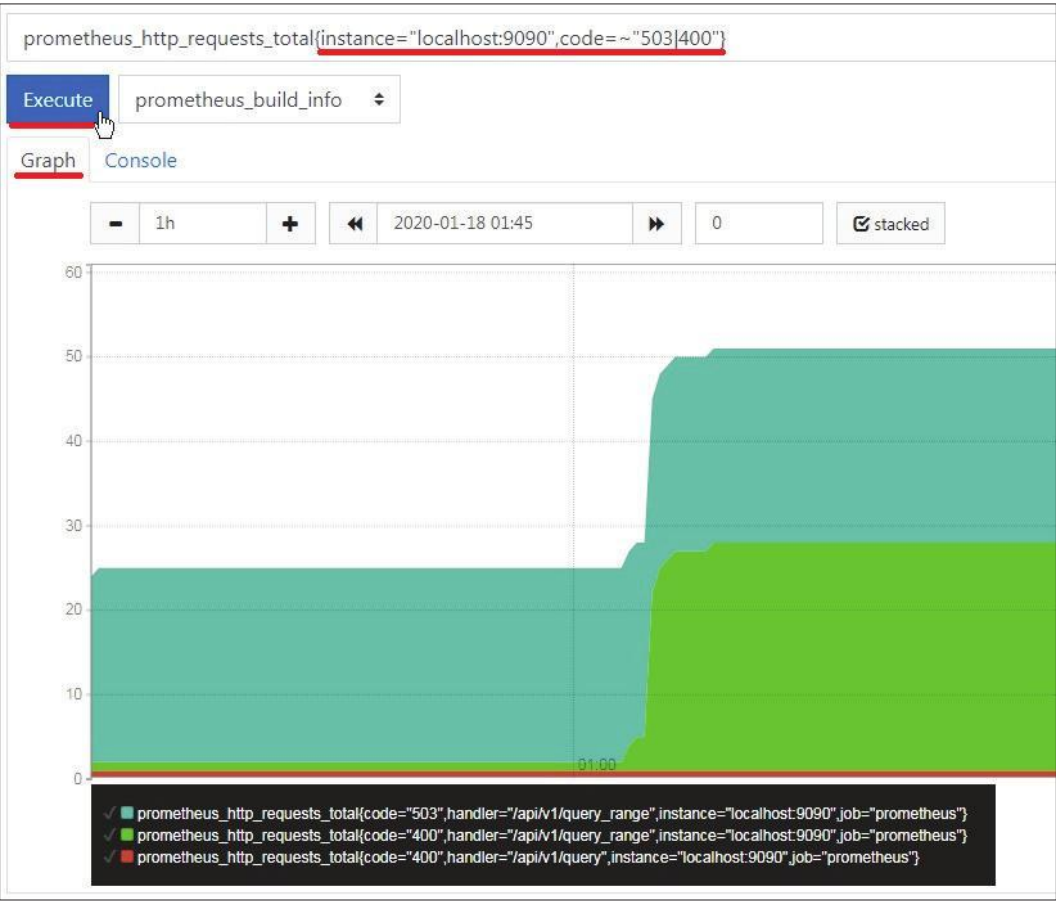
Note that a chart cannot be drawn because the vectors have multiple values for each timestamp.

4. To build complex filters, we will use regular expressions for Go – RE2 syntax. Let's create a filter for a histogram metric that will exclude all "/alerts" values for "handler" field and for "le" fields that starts with "In" symbols.



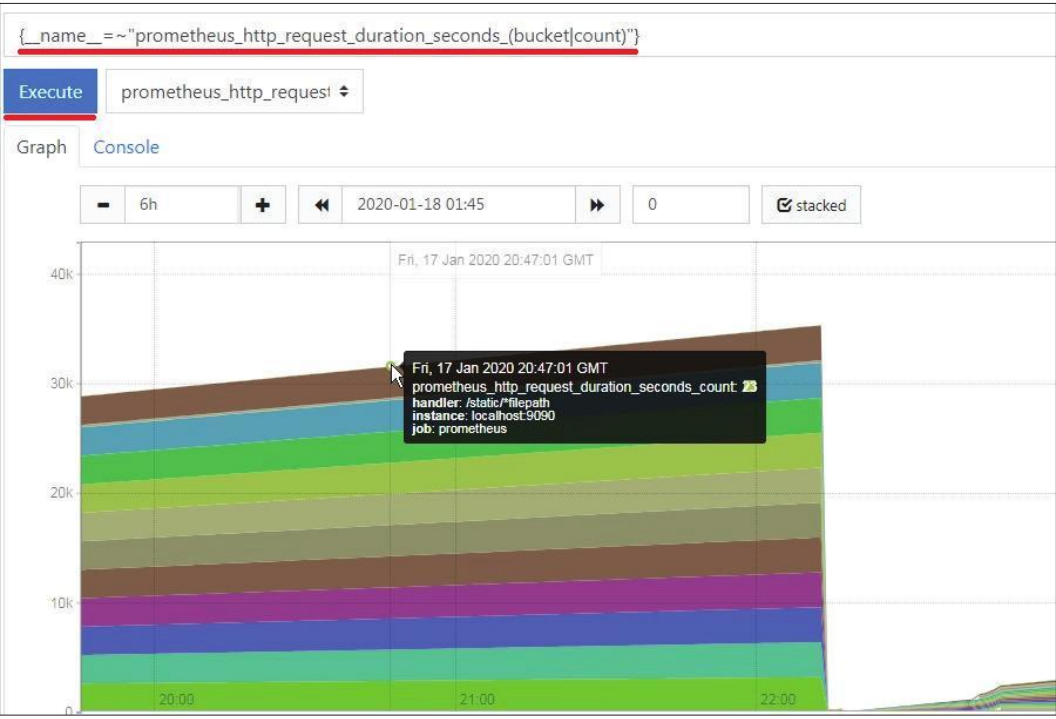
The result of the query execution is a dashboard that excludes filtered values of the specified metrics. In this way, we can exclude unnecessary data from the visualizations.

5. We can make even more complex queries by using other filtering rules. In the following example we filter by an instance name, and also for two specific values of “code”. We want to see only instances named “Localhost:9090”, that have “code” equal to either 503 or 400.



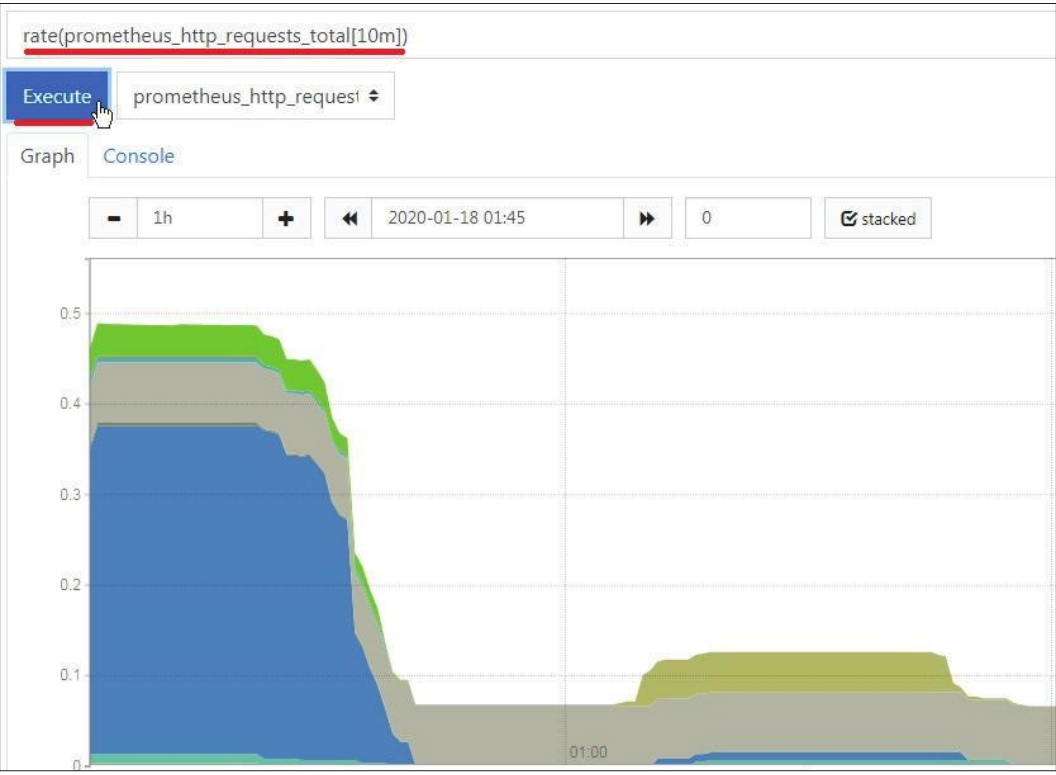
Note that when we query for two values of “code” it will automatically visualize for both the "and" and "or" logical functions.

6. The PromQL query function also allows you to create filters that combine different metrics using their names. Let's demonstrate it for histogram and counter metrics with similar names.



This approach allows for simple metrics value aggregation with similar names.

7. Usually, calculation functions are required for detailed analysis. One of them is the rate function that calculates the per-second rate for time series. Take a look at our article on Prometheus rate() for more detailed information. Let's build a rate() query for the counter metric from example 1, with a [10m] time value. The [10m] time value indicates the span of time over which the per-second rate is calculated for each point on the graph.



8. PromQL operates with basic comparison, logic, and arithmetic operations. Let's explore them below.

Comparison and Logical operations:

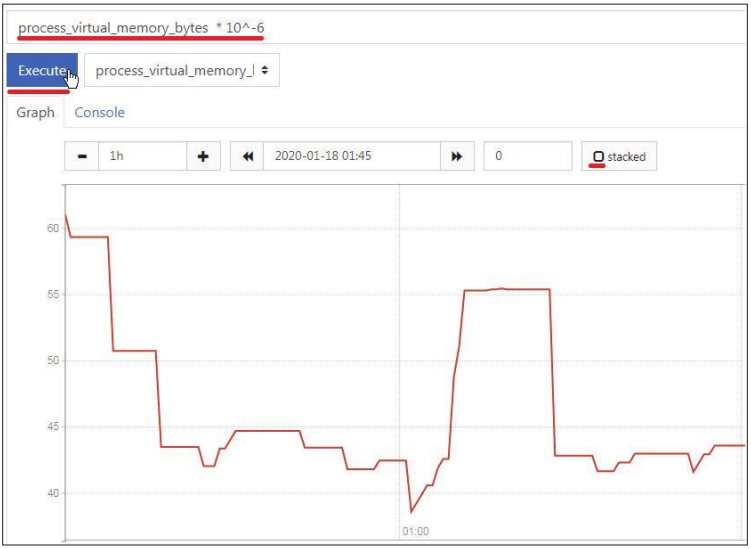
- greater (>)
- greater-or-equal (>=)
- less (<)
- less-or-equal (<=)
- equal (==)
- not equal (!=)
- and (intersection)

- or (union)
- unless (complement)

Arithmetic operations:

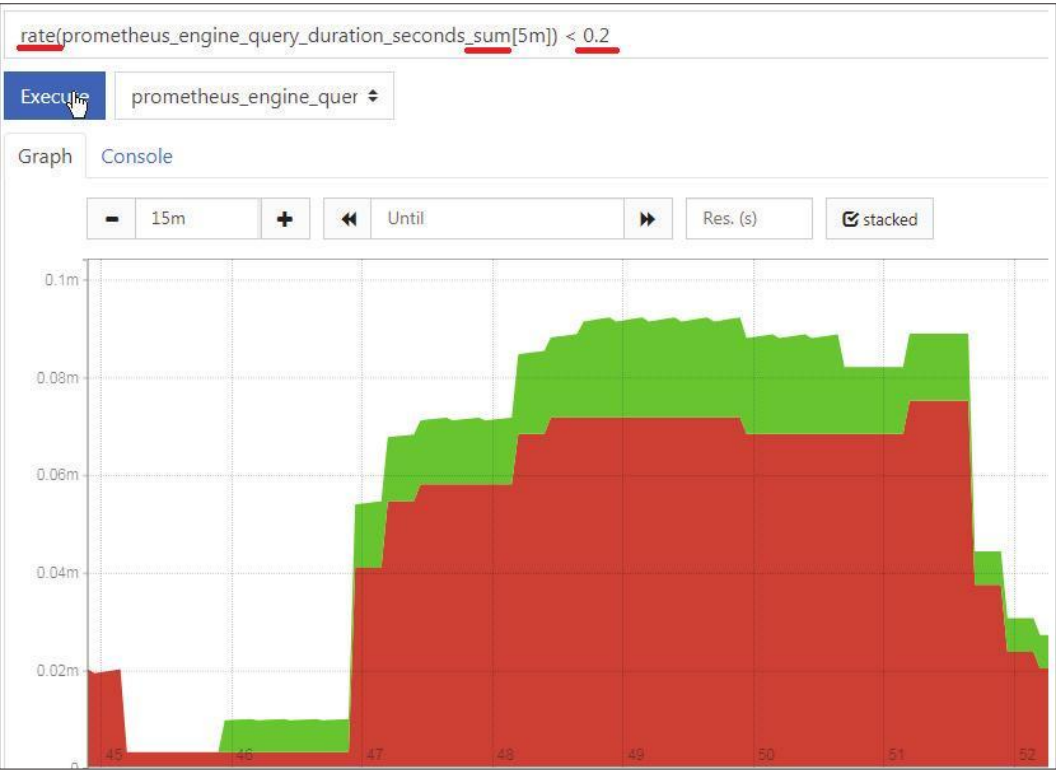
- addition (+)
- subtraction (-)
- division (/)
- multiplication (*)
- power (^)
- modulo (%)

This functionality provides the basic calculations for additional data analysis. As an example, they can be used to convert measurement units to the required format. Next, we demonstrate how to convert virtual memory size data recorded in bytes into megabytes for a gauge metric.

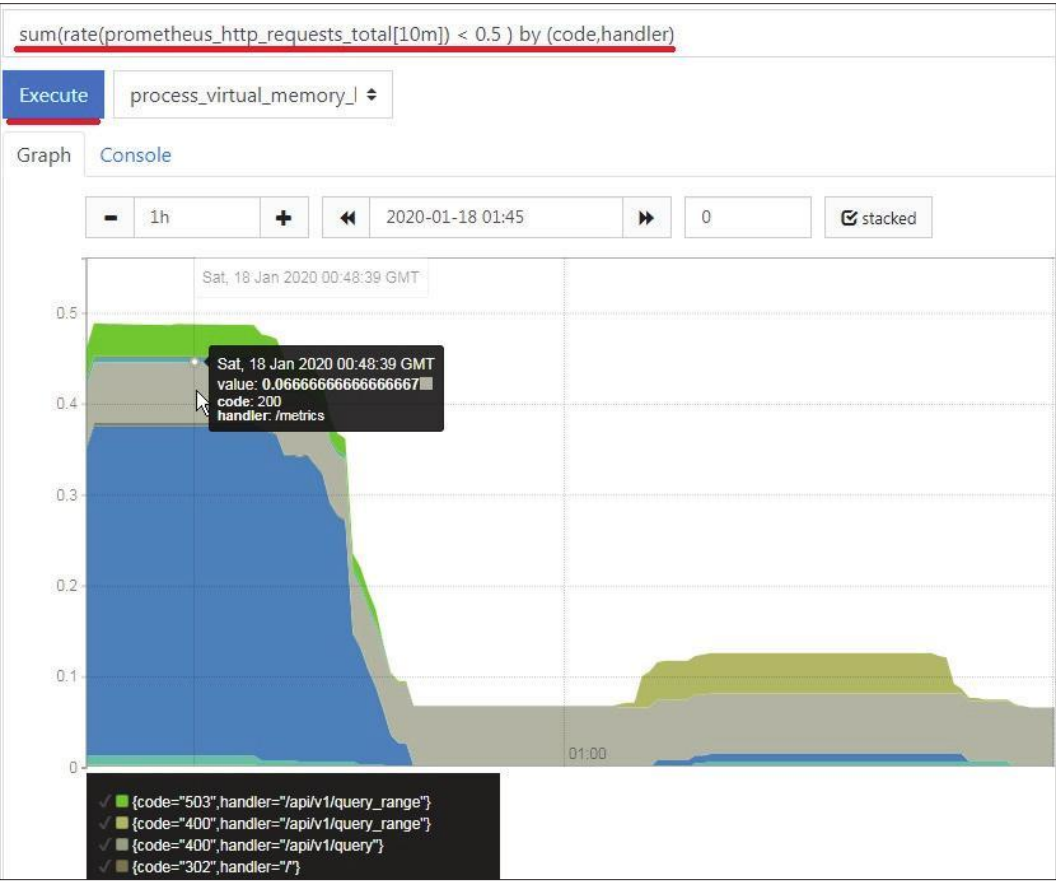


The arithmetic operations can be performed with time series, and these operations will be possible only for the corresponding values of these series.

We will also show an example for comparison operations with summary metric and limiting parameters to some limits.

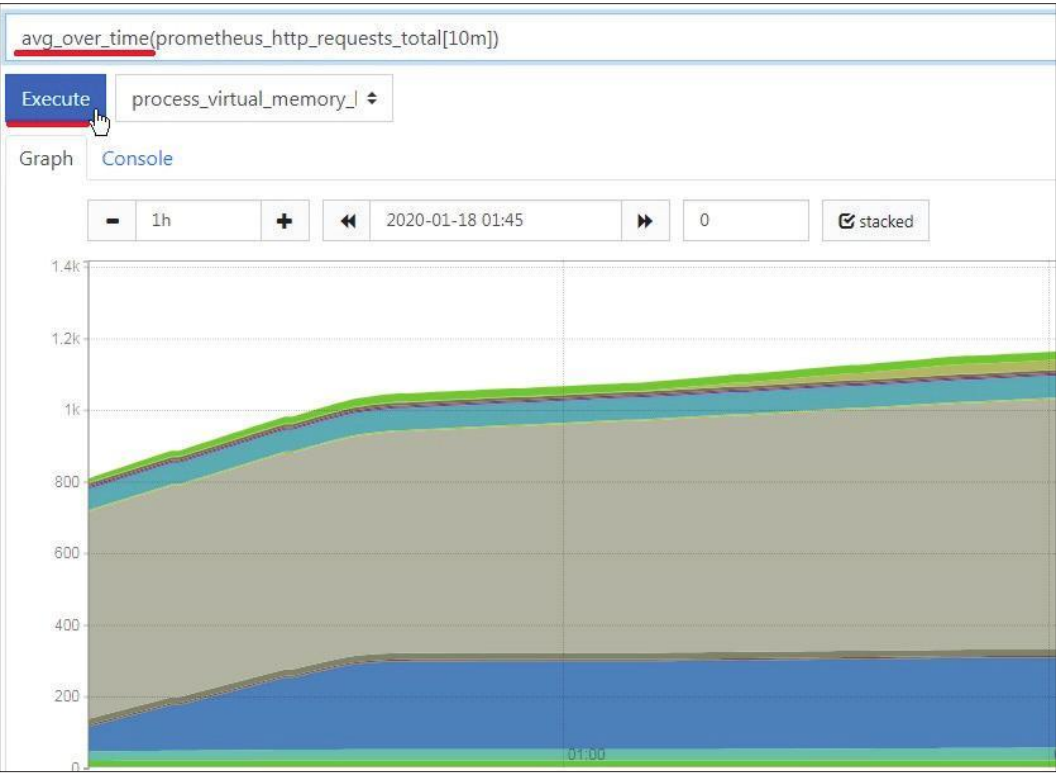


9. Frequently, in the case of many labels, it is necessary to correctly group and aggregate them. PromQL has built-in functionality for this issue. Let's demonstrate it using the same metric and rate() calculation as in the previous example.



As we can see, the resulting data is sorted by the corresponding categories in the query. Such functionality will be useful in preparing the monitoring data for further analysis.

10. Usually, if we work with gauge type metrics, we will need to limit, scale, or convert the metric values. To implement this type of query, we should use the <aggregation>_over_time PromQL functionality. As a demonstration, we will use it to calculate the statistical data for the average number of requests per defined time range.



We' ve shown only a small part of the PromQL functionality that allows flexible customization of queries with the Prometheus monitoring system. The full list and description of PromQL functionality are available on the official website.

8.5 Conclusion

This chapter introduced the basic functionality of the Prometheus monitoring system, with a focus on PromQL. We have discussed the basic principles and features of its operation, which can help you use it for your tasks in the future. We have also demonstrated the features and capabilities of building queries with PromQL.

Top 5 Alertmanager Gotchas

9.1 Introduction

Prometheus is very commonly deployed with another component called Alertmanager which provides numerous features such as the deduplication of alerts, silencing, grouping, inhibition, and routing (source). In fact, Prometheus and Alertmanager are almost inseparable because Prometheus has strong support for it - there is a top-level key called alerting in the configuration of Prometheus. This key is solely for specifying the Alertmanager nodes, as well as the mangling rules of alerts before firing them.

However, this might not be as straight-forward as it seems on first glance. Experience shows that some issues come up again and again. This chapter will shed light on them and show you how to avoid and solve these common problems.

9.2 Annotations vs. labels

One of the first things that you will run into while defining alerts are these two things called annotations and labels. Here is what a simple alerting rule looks like:

File: ExampleAlert.yaml

```yaml
groups:
- name: ExampleAlert
  rules:
  - alert: UnusualDiskWriteLatency
    expr: rate(node_disk_write_time_seconds_total[1m]) / rate(node_disk_writes_completed_total[1m]) > 100
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "Unusual disk write latency (instance {{ $labels.instance }})"
      description: "Disk latency is growing (write operations > 100ms)\n  VALUE = {{ $value }}\n  LABELS: {{ $labels }}"
```

As you can see, annotations and labels are seemingly used for the same thing: adding extra data to the alert above what is already there.

1. Labels are something that identifies the alert, and they automatically get written based on the alert's expression by default. But, they can be overridden by the alerting rule's definition, in the labels section.
2. Annotations also add information about the alert but they do not get automatically pre-filled by using the alerting rule's data. Instead, you are supposed to create your own annotations which enrich the data that comes from the labels.

Also, you can use the available templating system. For instance, in this example you can see things such as {{ $value }} which gets substituted with the value of the alert's expression. This is not possible with labels which are ordinary string values. You can find more information about the different possibilities for the

templating engine by looking into Go's documentation or here in Prometheus' documentation.

There is one more crucial difference: labels are used to group related alerts together. The key group_by in a route's configuration is used to set what labels are used to lump alerts together into one, which means that the end receiver that you had configured in Alertmanager will receive them in one batch.

The variable CommonLabels in your notification template will contain all of the labels that you have specified. But, each separate alert in the Alerts variable might contain extra labels beyond what's contained in CommonLabels. The separate alerts in the Alerts variable might also contain extra labels besides the ones that you have specified in group_by.

Therefore please pay attention to this small difference when making alerting rules because it is easy to make a mistake here, especially if you are new to Prometheus.

9.3 Avoid flapping alerts

After writing some alerting rules, have you ever noticed that they are continuously firing and becoming resolved time and time again. This may be happening because your alerting rule fires the alert to your configured receivers too quickly, without waiting to see if it gets solved naturally. How do we solve this?

The issue might come from:

- The non-existence (or it being too small) of the "for" clause in your alerting rule and/or
- Aggregation operations in your alerting rule

First of all, in your alerting rules you ought to almost always have some kind of time component in them that indicates how long the alert should wait before sending the notification. This is important because failures are inevitable due to the network connection being imperfect so we might sometimes fail to scrape a target. We want to make sure the alert has been active for a designated amount of time before we send the notification, that way, if the alert gets resolved within 1 - 2 minutes, we aren't notified at all. Also, any service level indicators, objectives, or agreements that you might have for your services are typically defined in terms of an error budget that you can "spend" over some time or the percent that your service was available over the last, lets say, one month. In turn, monitoring systems such as Prometheus are designed to alert on trends - when something goes really haywire - but not on exact facts. Here is a useful generator for SLOs that generates these things automatically for you according to Google's book.

When trying to solve this issue using the Aggregation Operations option listed above, we see that this is the harder way to solve the problem because it involves the expression itself, so it might be harder to change. Please feel free to use any of the aggregation functions described in this page, and then increase the time ranges used in your range vectors gradually as described here. However,

this is not always applicable because you might want to alert if a specific metric becomes 1, for example. That's where the former method comes into play.

As you have seen before, the definition of an alerting rule contains a field called "for":

```
groups:
- name: example
  rules:
  - alert: HighRequestLatency
    expr: job:request_latency_seconds:mean5m{job="myjob"} > 0.5
    for: 10m
    labels:
      severity: page
    annotations:
      summary: High request latency
```

In this picture the "for" value is equal to 10m or, in other words, 10 minutes. It means that Prometheus will check that the alert has been active for 10 minutes before firing the alert to your configured receivers. Also, note that by default alerting rules are evaluated every 1 minute and you can change that via the evaluation_interval parameter in your Prometheus configuration.

It is important to recognize that it probably does not make much sense to make the "for" clause smaller than the evaluation_interval because your alerting rule will be evaluated two times either way. Even if you make the "for" clause smaller than the evaluation_interval, your "for" clause will be "equal" to evaluation_interval in practice.

So please always consider one of those two options (or both) so that you will not get unnecessarily spammed with notifications. Notifications should always be actionable.

9.4 Be careful with PromQL expressions

Now let's talk about the dangers that lie in the PromQL expressions that you might use for alerting rules.

9.4.1 Missing metrics

First of all, you should keep in mind that the metric that you have written in the *expr* field might not exist at some point in time. In such a case, alerting rules silently start failing i.e. they do not become "firing". To solve this problem, you should add extra alerts which would alert you on missing metrics with the absent function. Some even coalesce this with the original alert by using the "or" binary operator:

```
checker_upload_last_succeeded{instance="foo.
bar"} != 1 or absent(checker_upload_last_
succeeded{instance="foo.bar"}) == 1
```

However, this is mostly useful in cases where you do not use any aggregation functions, and only use a single metric, since it quickly becomes unwieldy.

9.4.2 Cardinality explosions

Prometheus creates new metrics called ALERTS and ALERTS_FOR_STATE for every alert defined in your system. They contain all of the labels that were generated from the alert's expression and the extra labels that you have defined as discussed previously. The purpose of ALERTS and ALERTS_FOR_STATE is to allow you to see what kind of alerts were firing historically. But, there is one small problem - if your original expression does not contain any specific label selectors, Prometheus could potentially create thousands of new time series for each of the matched series. Each of the series has a set of labels that was generated by joining the labels of the original expression and the extra labels that you have defined in your alerting rule.

The labels defined in the alerting rule are static, however the labels on the metric are not - this is where the cardinality problem comes from. This could potentially significantly slow down your Prometheus instance because each new time series is equal to a new entry in the index which leads to increased look-up times when you are querying something. This is one of the ways to get what is called a "cardinality explosion" in your Prometheus instance. You should always, always validate that your alerting expression will not touch too many different time series.

For example, it is not unimaginable that you will be scraping something like kube-state-metrics which could have thousands of metrics. Let's say you have numerous pods running in your Kubernetes cluster and then you might have lots of metrics like kube_pod_created. The metric's value shows when the pod

has been created. Perhaps, you will have an alerting rule like: kube_pod_created > 1575763200 to know if any pods have been created after 12/08/2019 @ 12:00am (UTC) which could be the start of your Kubernetes cluster's maintenance window. Alas, your users would continue creating thousands of new pods each day. In this case, ALERTS and ALERTS_FOR_STATE would match all of the pods' information (the pod's name and its namespace, to be more exact) that you have in your Kubernetes cluster thus leading to a multiplication of the original time series.

This is a fictitious example but nevertheless it shows you the danger lying in this aspect of PromQL expressions. In conclusion, you should be conscious of the decisions that you are making in terms of the label selectors in your alert expressions.

9.4.3 Huge queries

Last but not least, let's talk about how alerting rules might start utilizing all of the resources of your Prometheus instance. You might unwittingly write an alert which would load hundreds of thousands samples into memory. This is where --query.max-samples jumps in. By default, it forbids you from loading more than 50 million samples into memory with one query. You should adjust it accordingly.
If it hits that limit then you will see errors such as this in your Prometheus logs: "query processing will load too many samples into memory in query execution". This is a very helpful notification!

But, typically your queries will go through and you will not notice anything wrong until one day your Prometheus instance might start responding slower. Fortunately Prometheus has a lot of nice

metrics itself that show what might be taking longer than usual. For instance, prometheus_rule_group_last_duration_seconds will show, by alerting rule group, how long it took to evaluate them the last time in seconds. You will most likely want to create or import a Grafana dashboard which will visualize this data for you, or you could actually write another alert which will notify you in case it starts taking more than a specified threshold. The alerting expression could look like this:

```
avg_over_time(prometheus_rule_group_last_
duration_seconds{instance="my.prometheus.
metricfire.com"}[5m]) > 20
```

You will most likely want to add the calculation of the average here since the duration of the last evaluation will naturally have some level of jitter because the load of your Prometheus will inevitably differ over time.

9.5 Conclusion

PromQL and the Prometheus alerting engine provides lots of different capabilities to users but careful use of it is key to the long-term stability and performance of your Prometheus instance. You should keep in mind these tips while writing alerting rules and everything should be fine.

At MetricFire, we have users who prefer to write their own alerting rules, as well as users who prefer to have us write them. We're always happy to guide users and build good alerting design based on the situation.

Understanding Prometheus Rate()

10.1 Introduction

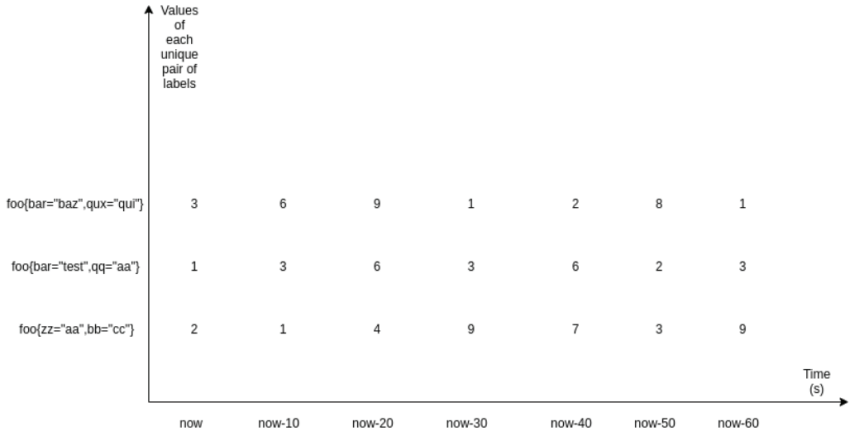
Both Prometheus and its querying language PromQL have quite a few functions for performing various calculations on the data they have. One of the most widely used functions is rate(), however it is also one of the most misunderstood.

One of the essential functions of rate() is predicting trends. As the name suggests, it lets you calculate the per-second average rate of how a value is increasing over a period of time. It is the function to use if you want, for instance, to calculate how the number of requests coming into your server changes over time, or the CPU usage of your servers. But first, let's talk about its internals. We need to understand how it works under-the-hood, so that we can build up our knowledge from there.

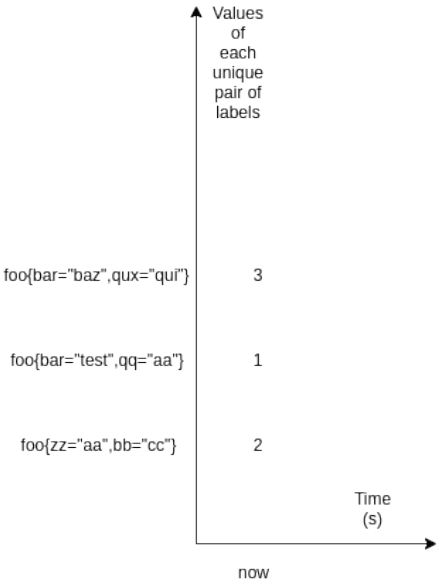
10.2 How it works

10.2.1 Types of arguments

There are two types of arguments in PromQL: range and instant vectors. Here is how it would look if we looked at these two types graphically:



This is a matrix of three range vectors, where each one encompasses one minute of data that has been scraped every 10 seconds. As you can see, it is a set of data that is defined by a unique set of label pairs. Range vectors also have a time dimension - in this case it is one minute - whereas instant vectors do not. Here is what instant vectors would look like:



As you can see, instant vectors only define the value that has been most recently scraped. Rate() and its cousins take an argument of the range type since to calculate any kind of change, you need at least two points of data. They do not return any result at all if there are less than two samples available. PromQL indicates range vectors by writing a time range in square brackets next to a selector which says how much time into the past it should go.

10.2.2 Choosing the time range for range vectors

What time range should we choose? There is no silver bullet here: at the very minimum it should be two times the scrape interval. However, in this case the result will be very "sharp": all of the changes in the value would reflect in the results of the function faster than any other time range. Thereafter, the result would become 0 again swiftly. Increasing the time range would achieve the opposite - the resulting line (if you plotted the results) would become "smoother" and it would be harder to spot the spikes. Thus, the recommendation is to put the time range into a different variable (let's say 1m, 5m, 15m, 2h) in Grafana, then you are able to choose whichever value fits your case the best at the time when you are trying to spot something - such as a spike or a trend.

One could also use the special variable in Grafana called $__interval - it is defined to be equal to the time range divided by the step's size. It could seem like the perfect solution as it looks like all of the data points between each step would be considered, but it has the same problems as mentioned previously. It is impossible to see both very detailed graphs and broad trends at the same time. Also your time interval becomes tied to your query step, so if your

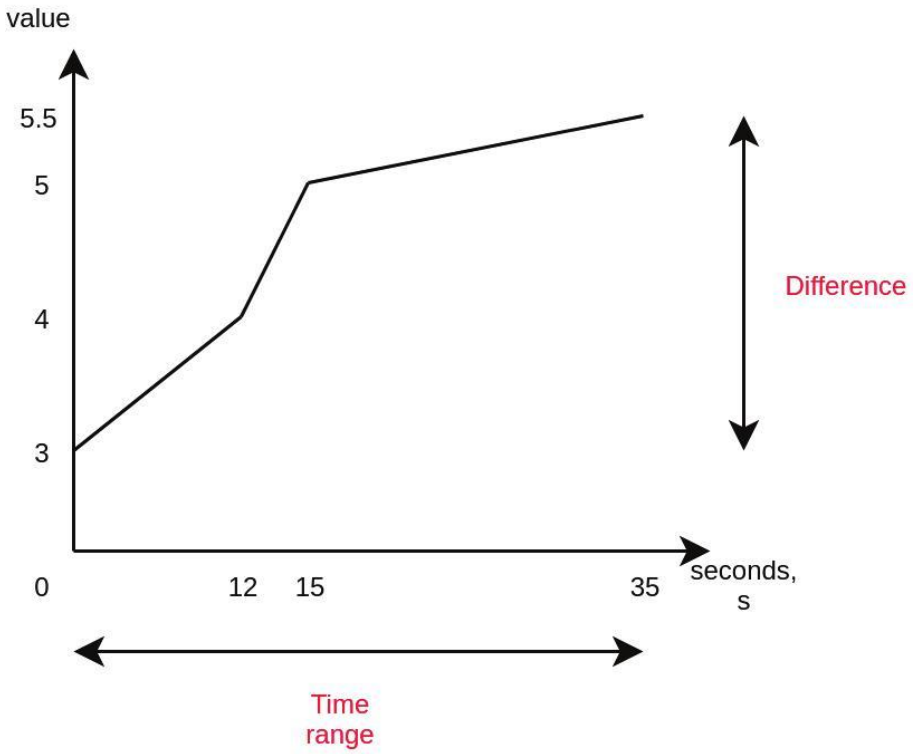
scrape interval ever changes then you might have problems with very small time ranges.

10.2.3 Calculation

Just like everything else, the function gets evaluated on each step. But, how does it work?

It roughly calculates the following:

rate(x[35s]) = difference in value over 35 seconds / 35s



The nice thing about the rate() function is that it takes into account all of the data points, not just the first one and the last one. There is another function, irate, which uses only the first and last data points.

You might now say… why not delta()?

Well, rate() that we have just described has this nice characteristic: it automatically adjusts for resets. What this means is that it is only suitable for metrics which are constantly increasing, a.k.a. the metric type that is called a "counter". It's not suitable for a "gauge". Also, a keen reader would have noticed that using rate() is a hack to work around the limitation that floating-point numbers are used for metrics' values and that they cannot go up indefinitely so they are "rolled over" once a limit is reached. This logic prevents us from losing old data, so using rate() is a good idea when you need this feature.

Note: because of this automatic adjustment for resets, if you want to use any other aggregation together with rate() then you must apply rate() first, otherwise the counter resets will not be caught and you will get weird results.

Either way, PromQL currently will not prevent you from using rate() with a gauge, so this is a very important thing to realize when choosing which metric should be passed to this function. It is incorrect to use rate() with gauges because the reset detection logic will mistakenly catch the values going down as a "counter reset" and you will get wrong results.

All in all, let's say you have a counter metric which is changing like this:

```
0
4
6
10
2
```

The reset between "10" and "2" would be caught by irate() and rate() and it would be taken as if the value after that were "12" i.e. it has increased by "2" (from zero). Let's say that we were trying to calculate the rate with rate() over 60 seconds and we got these 6 samples on ideal timestamps. So the resulting average rate of increase per second would be:

12-0/60 = 0.2. Because everything is perfectly ideal in our situation, the opposite calculation is also true: **0.2 * 60 = 12**. However, this opposite calculation is not always true in the cases where some samples are not covering the full range ideally, or when samples do not line up perfectly due to random delays introduced between scrapes. Let me explain this in more detail in the following section.

10.2.4 Extrapolation: what rate() does when it's missing information

Last but not least, it's important to understand that rate() performs extrapolation. Knowing this will save you from headaches in the long-term. Sometimes when rate() is executed in a point in time, there might be some data missing if some of the scrapes had failed. What's more, the scrape interval due to added randomness

might not align perfectly with the range vector, even if it is a multiple of the range vector's time range.

In such a case, rate() calculates the rate with the data that it has and then, if there is any information missing, extrapolates the beginning or the end of the selected window using either the first or last two data points. This means that you might get uneven results even if all of the data points are integers, so this function is suited **only for** spotting trends, spikes, and for alerting if something happens.

10.2.5 Aggregation

Optionally, you apply rate() only to certain dimensions just like with other functions. For example, rate(foo) by (bar) will calculate the rate of change of foo for every bar (label's name). This can be useful if you have, for example, haproxy running and you want to calculate the rate of change of the number of errors by different backends so you can write something like rate(haproxy_connection_errors_total[5m]) by (backend).

10.3 Examples

10.3.1 Alerting rules

Just like described previously, rate() works perfectly in the cases where you want to get an alert when the amount of errors jumps up. So, you could write an alert like this:

```
groups:
```

```
- name: Errors
  rules:
  - alert: ErrorsCountIncreased
    expr: rate(haproxy_connection_errors_
total[5m]) by (backend) > 0.5
    for: 10m
    labels:
      severity: page
    Annotations:
      Summary: High connection error count in
{{ $labels.backend }}
```

This would inform you if any of the backends have an increased amount of connection errors. As you can see, rate() is perfect for this use case. Feel free to implement similar alerts for your services that you monitor with MetricFire, or Prometheus.

10.3.2 SLO calculation

Another common use-case for the rate() function is calculating SLIs, and seeing if you do not violate your SLO/SLA. Google has recently released a popular book for site-reliability engineers. Here is how they calculate the availability of the services:

Calculating the SLIs

Using the preceding metrics, we can calculate our current SLIs over the previous seven days, as shown in Table 2-2.

Table 2-2. Calculations for SLIs over the previous seven days

Availability	`sum(rate(http_requests_total{host="api", status!~"5.."}[7d]))`
	`/`
	`sum(rate(http_requests_total{host="api"}[7d])`

As you can see, they calculate the rate of change of the amount of all of the requests that were not 5xx and then divides by the rate of change of the **total** amount of requests. If there are any 5xx responses then the resulting value would be less than one. You can, again, use this formula in your alerting rules with some kind of specified threshold - then you would get an alert if it is violated or you could predict the near future with predict_linear and avoid any SLA/SLO problems.

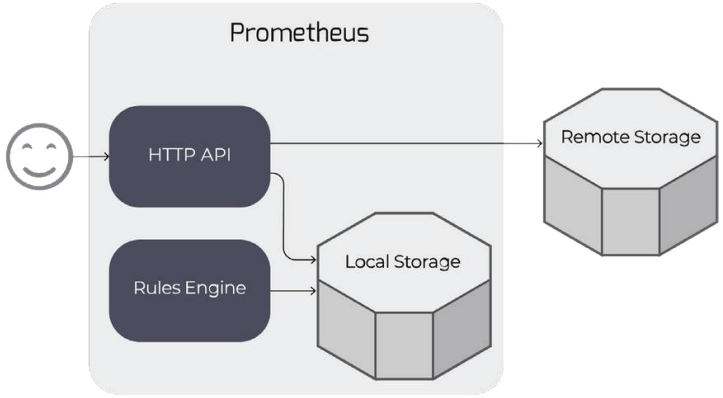
Prometheus Remote Storage

11.1 Introduction

Prometheus can be configured to read from and write to remote storage, in addition to its local time series database. This enables long-term storage of monitoring data, as the local database only stores data for up to two weeks.

11.2 Remote read

When configured, Prometheus storage queries (e.g. via the HTTP API) are sent to both local and remote storage, and results are merged.



Note that to maintain reliability in the face of remote storage issues, alerting and recording rule evaluation use only the local TSDB.

11.3 Configuration

You configure the remote storage read path in the `remote_read` section of the Prometheus configuration file.

At its simplest, you will just specify the read endpoint URL for your remote storage, plus an authentication method. You can use either HTTP basic or bearer token authentication.

You might want to use the `read_recent` flag: when set to true, all queries will be answered from remote as well as local storage. When false (the default), any queries that can be answered completely from local storage will not be sent to the remote endpoint.

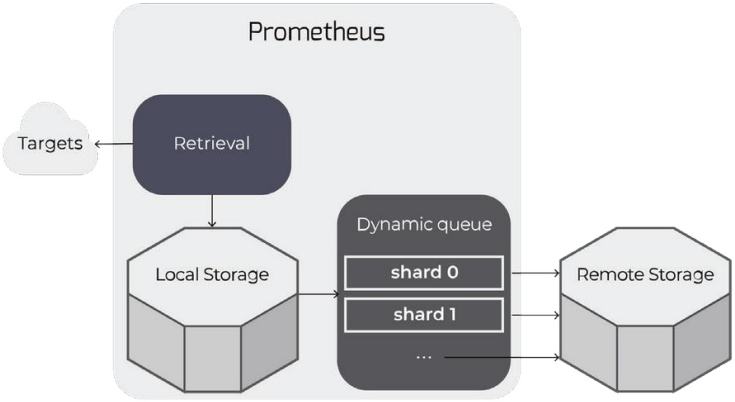
You can specify a set of `required_matchers` (label, value pairs) to restrict remote reads to some subset of queries. This is useful if e.g. you write only a subset of your metrics to remote storage (see below).

For more complex configurations, there are also options for request timeouts, TLS configuration, and proxy setup.

You can read from multiple remote endpoints by having one `remote_read` section for each.

11.4 Remote write

When configured, Prometheus forwards its scraped samples to one or more remote stores.



Remote writes work by "tailing" time series samples written to local storage, and queuing them up for write to remote storage.

The queue is actually a dynamically-managed set of "shards": all of the samples for any particular time series (i.e. unique metric) will end up on the same shard.

The queue automatically scales up or down the number of shards writing to remote storage to keep up with the rate of incoming data.

This allows Prometheus to manage remote storage while using only the resources necessary to do so, and with minimal configuration.

11.5 Configuration

You configure the remote storage write path in the `remote_write` section of the Prometheus configuration file.

Like for `remote_read`, the simplest configuration is just a remote storage write URL, plus an authentication method. You can use either HTTP basic or bearer token authentication.

You can use `write_relabel_configs` to relabel or restrict the metrics you write to remote storage. For example, a common use is to drop some subset of metrics:

```
1. writeRelabelConfigs:
2.  # drop all metrics of this name across all jobs
3.  - sourceLabels: ["__name__"]
4. regex: some_metric_prefix_to_drop_.*
5. action: drop
```

The queue_config section gives you some control over the dynamic queue described above. Usually, you won't need to make changes here and can rely on Prometheus' defaults.

- `capacity`: each shard is itself a queue, and this is the number of samples queued before the shard "blocks" further additions;
- `min_shards`, `max_shards`: the minimum & maximum shards the dynamic queue will use;
- `max_samples_per_send`, `batch_send_deadline`: each shard batches samples up into blocks of `max_`

`samples_per_send`, or if it can't make a batch of that size before `batch_send_deadline`, sends anyway; this latter will rarely happen on a busy Prometheus instance;
- `min_backoff`, `max_backoff`, `max_retries`: retry management; note `max_retries` is not used in the current implementation - each shard will just block and retry sends "forever".

Like for remote_read, you can also configure options for request timeouts, TLS configuration, and proxy setup.

You can write to multiple remote endpoints by having one `remote_write` section for each.

11.6 Log messages

You may see some messages from the remote storage subsystem in your logs:

- **dropped sample for series that was not explicitly dropped via relabelling**
  Because of relabelling or for some other reason, we've ended up with a series with no labels in the remote write path; we drop it.
- **Remote storage resharding from N to M**
  The dynamic queue size is changing the number of shards - either growing to keep up with the number of incoming samples vs. outgoing remote storage write rate, or shrinking because we have more shards than are necessary.

- **Currently resharding, skipping**
  The dynamic queue wants to change to a new number of shards, but a reshard is already in progress.
- **Failed to flush all samples on shutdown**
  While shutting down a dynamic queue, Prometheus was unable to flush all samples to remote storage - it's probable there was a problem with the remote storage endpoint.

11.7 Metrics exported from the remote storage subsystem

The remote storage subsystem exports lots of metrics, prefixed with `prometheus_remote_storage_` or `prometheus_wal_watcher_`. Here's a selection you might find interesting:

- `prometheus_remote_storage_samples_in_total`: Samples in to remote storage, compare to samples out for queue managers (counter);
- `prometheus_remote_storage_succeeded_samples_total`: Total number of samples successfully sent to remote storage (counter);
- `prometheus_remote_storage_pending_samples`: The number of samples pending in the queue's shards to be sent to the remote storage (gauge);
- `prometheus_remote_storage_shards`: The number of shards used for parallel sending to the remote storage (gauge);
- `prometheus_remote_storage_sent_batch_duration_seconds`: Duration of sample batch send calls to the remote storage (histogram).

Example 1: Monitoring a Python Web App with Prometheus

12.1 Introduction

We eat lots of our own dogfood at MetricFire - which means we monitor our services with a dedicated cluster running the same software.

This has worked out really well for us over the years: as our own customer, we quickly spot issues in our various ingestion, storage and rendering services. It also drives the service status transparency our users love.

Recently we've been working on a new Prometheus offering. Eating the right dogfood here means integrating Prometheus monitoring with our (mostly Python) backend stack.

This chapter describes how we've done that in one instance, with a fully worked example of monitoring a simple Flask application running under uWSGI + nginx. We'll also discuss why it remains surprisingly involved to get this right.

12.2 A little history

Prometheus' ancestor and main inspiration is Google's Borgmon.

In its native environment, Borgmon relies on ubiquitous and straightforward service discovery: monitored services are managed by Borg, so it's easy to find e.g. all jobs running on a cluster for a particular user; or for more complex deployments, all sub-tasks that together make up a job.

Each of these might become a single target for Borgmon to scrape data from via `/varz` endpoints, analogous to Prometheus' `/metrics`. Each is typically a multi-threaded server written in C++, Java, Go, or (less commonly) Python.

Prometheus inherits many of Borgmon's assumptions about its environment. In particular, client libraries assume that metrics come from various libraries and subsystems, in multiple threads of execution, running in a shared address space. On the server side, Prometheus assumes that one target is one (probably) multi-threaded program.

12.3 Why did it have to be snakes?

These assumptions break in many non-Google deployments, particularly in the Python world. Here it is common (e.g. using Django or Flask) to run under a WSGI application server that spreads requests across multiple workers, each of which is a **process** rather than a thread.

In a naive deployment of the Prometheus Python client for a Flask app running under uWSGI, each request from the Prometheus server to `/metrics` can hit a different worker process, each of which exports its own counters, histograms, etc. The resulting monitoring data is garbage.

For example, each scrape of a specific counter will return the value for one worker rather than the whole job: the value jumps all over the place and tells you nothing useful about the application as a whole.

12.4 A solution

Amit Saha discusses the same problems and various solutions in a detailed writeup. We follow option #2: the Prometheus Python client includes a multiprocess mode intended to handle this situation, with gunicorn being the motivating example of an application server.

This works by sharing a directory of mmap()'d dictionaries across all the processes in an application. Each process then does the maths to return a shared view of the whole application's metrics when it is scraped by Prometheus.

This has some "headline" disadvantages listed in the docs: no per-process Python metrics for free, lack of full support for certain metric types, a slightly complicated Gauge type, etc.

It's also difficult to configure end-to-end. Here's what's necessary & how we achieved each part in our environment; hopefully this

full example will help anyone doing similar work in the future.

1. The shared directory must be passed to the process as an environment variable, `prometheus_multiproc_dir`.

   No problem: we use uWSGI's env option to pass it in: see uwsgi.ini.

2. The client's shared directory must be cleared across application restarts.

   This was a little tricky to figure out. We use one of uWSGI's hardcoded hooks, `exec-asap`, to exec a shell script right after reading the configuration file and before doing anything else. See uwsgi.ini.

   Our script removes & recreates the Prometheus client's shared data directory.

   In order to be sure of the right permissions, we run uwsgi under supervisor as `root` and drop privs within uwsgi.

3. The application must set up the Python client's multiprocess mode.

   This is mostly a matter of following the docs, which we did via Saha's post: see metrics.py.

   Note that this includes some neat middleware exporting Prometheus metrics for response status and latency.

4. uWSGI must set up the application environment so that applications load after `fork()`.

   By default, uWSGI attempts to save memory by loading the application and `then fork()` 'ing. This indeed has copy-on-write advantages and might save a significant amount of memory.

   However, it appears to interfere with the operation of the client's multiprocess mode - possibly because there's some locking prior to fork() this way?

   uWSGI's lazy-apps option allows us to load the application after forking, which gives us a cleaner environment.

So altogether, this results in a working `/metrics` endpoint for our Flask app running under uWSGI. You can try out the full worked example in our pandoras_flask demo.

Note that in our demo we expose the metrics endpoint on a different port to the app proper - this makes it easy to allow access for our monitoring without users being able to hit it.

In our deployments, we also use the uwsgi_exporter to get more stats out of uWSGI itself.

12.5 Futures

Saha's blog post lays out a series of alternatives, with pushing metrics via a local statsd as the favoured solution. That's not really a hop we prefer to take.

Ultimately, running everything under container orchestration like kubernetes would provide the native environment in which Prometheus shines, but that's a big step just to get its other advantages in an existing Python application stack.

Probably the most Promethean intermediate step is to register each sub-process separately as a scraping target. This is the approach taken by django-prometheus, though the suggested "port range" approach is a bit hacky.

In our environment, we could (and may yet) implement this idea with something like:

1. Running a webserver inside a thread in each process, listening on an ephemeral port and serving `/metrics` queries;
2. Having the webserver register and regularly refresh its address (e.g. hostname:32769) in a short-TTL `etcd` path—we use etcd already for most of our service discovery needs;
3. Using file-based service discovery in Prometheus to locate these targets and scrape them as individuals.

We think this approach is less involved than using the Python client's multiprocess mode, but it comes with its own complexities.

It's worth noting that having one target per worker contributes to something of a time series explosion. For example, in this case a single default Histogram metric to track response times from the Python client across 8 workers would produce around 140 individual time series, before multiplying by other labels we might include. That's not a problem for Prometheus to handle, but it does add up (or likely, multiply) as you scale, so be careful!

12.6 Conclusion

For now, exporting metrics to Prometheus from a standard Python web app stack is a bit involved no matter which road you take. We hope this post will help people who just want to get going with their existing nginx + uwsgi + `Flask` apps.

As we run more services under container orchestration—something we intend to do—we expect it will become easier to integrate Prometheus monitoring with them.

Example 2: HA Kubernetes Monitoring with Prometheus and Thanos

13.1 Introduction

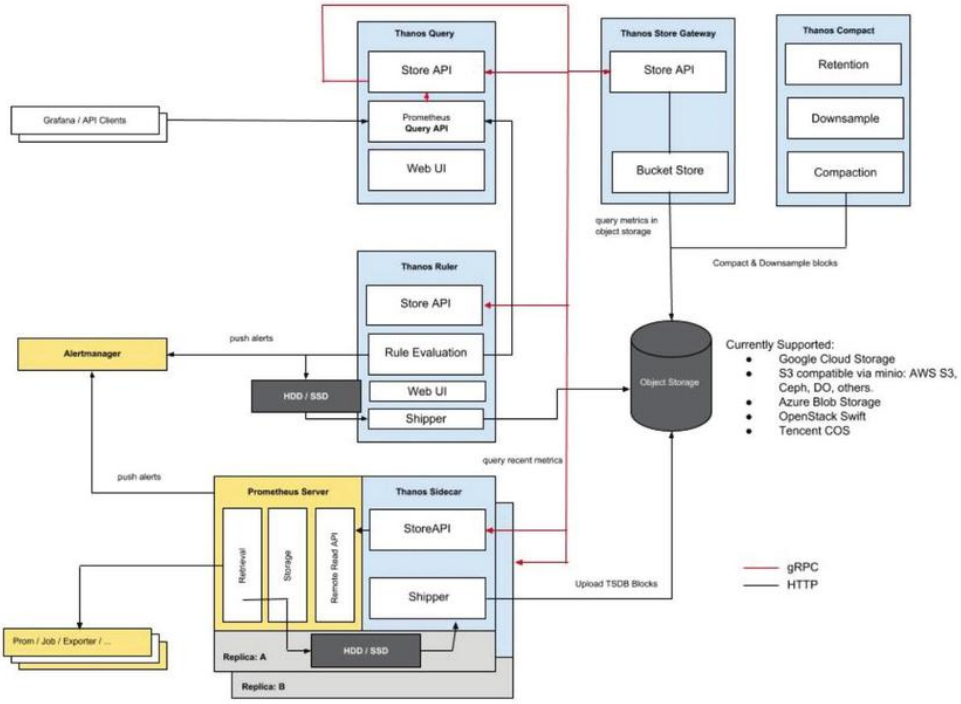
In this article, we will deploy a clustered Prometheus setup that integrates Thanos. It is resilient against node failures and ensures appropriate data archiving. The setup is also scalable. It can span multiple Kubernetes clusters under the same monitoring umbrella. Finally, we will visualize and monitor all our data in accessible and beautiful Grafana dashboards.

13.2 Why integrate Prometheus with Thanos?

Prometheus is scaled using a federated setup, and its deployments use a persistent volume for the pod. However, not all data can be aggregated using federated mechanisms. Often, you need a different tool to manage Prometheus configurations. To address these issues, we will use Thanos. Thanos allows you to create multiple instances of Prometheus, deduplicate data, and archive data in long-term storage like GCS or S3.

13.3 Thanos overview

13.3.1 Thanos architecture



The components of Thanos are sidecar, store, query, compact, and ruler. Let's take a look at what each one does.

13.3.2 Thanos Sidecar

- The main component that runs along Prometheus
- Reads and archives data on the object store
- Manages Prometheus's configuration and lifecycle

- Injects external labels into the Prometheus configuration to distinguish each Prometheus instance
- Can run queries on Prometheus servers' PromQL interfaces
- Listens in on Thanos gRPC protocol and translates queries between gRPC and REST

13.3.3 Thanos Store

- Implements the Store API on top of historical data in an object storage bucket
- Acts primarily as an API gateway and therefore does not need significant amounts of local disk space
- Joins a Thanos cluster on startup and advertises the data it can access
- Keeps a small amount of information about all remote blocks on a local disk in sync with the bucket
- This data is generally safe to delete across restarts at the cost of increased startup times

13.3.4 Thanos Query

- Listens in on HTTP and translates queries to Thanos gRPC format
- Aggregates the query result from different sources, and can read data from Sidecar and Store
- In HA setup, Thanos Query even deduplicates the result

A note on run-time duplication of HA groups: Prometheus is stateful and does not allow for replication of its database.

Therefore, it is not easy to increase high availability by running multiple Prometheus replicas.

Simple load balancing will also not work -- say your app crashes. The replica might be up, but querying it will result in a small time gap for the period during which it was down. This isn't fixed by having a second replica because it could be down at any moment, for example, during a rolling restart. These instances show how load balancing can fail.

Thanos Query pulls the data from both replicas and deduplicates those signals, filling the gaps, if any, to the Querier consumer.

13.3.5 Thanos Compact

- Applies the compaction procedure of the Prometheus 2.0 storage engine to block data in object storage
- Generally not concurrent with safe semantics and must be deployed as a singleton against a bucket
- Responsible for downsampling data: 5 minute downsampling after 40 hours and 1 hour downsampling after 10 days

13.3.6 Thanos Ruler

Thanos Ruler basically does the same thing as the querier but for Prometheus' rules. The only difference is that it can communicate with Thanos components.

13.4. Thanos implementation

Prerequisites: In order to completely understand this tutorial, the following are needed:

1. Working knowledge of Kubernetes and kubectl

2. A running Kubernetes cluster with at least 3 nodes (We will use a GKE)

3. Implementing Ingress Controller and Ingress objects (We will use Nginx Ingress Controller); although this is not mandatory, it is highly recommended in order to reduce external endpoints.

4. Creating credentials to be used by Thanos components to access object store (in this case, GCS bucket)

   a. Create 2 GCS buckets and name them as **prometheus-long-term** and **thanos-ruler**

   b. Create a service account with the role as **Storage Object Admin**

   c. Download the key file as json credentials and name it **thanos-gcs-credentials.json**

   d. Create a Kubernetes secret using the credentials, as you can see in the following snippet:

```
kubectl create secret generic thanos-gcs-credentials
--from-file=thanos-gcs-credentials.json -n monitoring
```

13.5 Deployment

Deploying Prometheus Services Accounts, Clusterrole and Clusterrolebinding: The following manifest creates the monitoring namespace, service accounts, clusterrole and clusterrolebindings needed by Prometheus.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: v1
kind: ServiceAccount
metadata:
 name: monitoring
 namespace: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1beta1
kind: ClusterRole
metadata:
  name: monitoring
  namespace: monitoring
rules:
```

```
- apiGroups: [""]
  resources:
  - nodes
  - nodes/proxy
  - services
  - endpoints
  - pods
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources:
  - configmaps
  verbs: ["get"]
- nonResourceURLs: ["/metrics"]
  verbs: ["get"]
---
apiVersion: rbac.authorization.k8s.io/v1beta1
kind: ClusterRoleBinding
metadata:
  name: monitoring
subjects:
  - kind: ServiceAccount
    name: monitoring
    namespace: monitoring
roleRef:
  kind: ClusterRole
  Name: monitoring
  apiGroup: rbac.authorization.k8s.io
---
```

Deploying Prometheus Configuration configmap: The following config map creates the Prometheus configuration file template that will be read by the Thanos sidecar component. The template will also generate the actual configuration file. The file will be consumed by the Prometheus container running in the same pod. It is extremely important to add the **external_labels** section in the config file so that the querier can deduplicate data based on it.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-server-conf
  labels:
    name: prometheus-server-conf
  namespace: monitoring
data:
  prometheus.yaml.tmpl: |-
    global:
      scrape_interval: 5s
      evaluation_interval: 5s
      external_labels:
        cluster: prometheus-ha
        # Each Prometheus has to have unique labels.
        replica: $(POD_NAME)

    rule_files:
      - /etc/prometheus/rules/*rules.yaml

    alerting:
```

```
      # We want our alerts to be deduplicated
      # from different replicas.
      alert_relabel_configs:
      - regex: replica
        action: labeldrop

      alertmanagers:
        - scheme: http
          path_prefix: /
          static_configs:
            - targets: ['alertmanager:9093']

    scrape_configs:
    - job_name: kubernetes-nodes-cadvisor
      scrape_interval: 10s
      scrape_timeout: 10s
      scheme: https
      tls_config:
        ca_file: /var/run/secrets/kubernetes.io/
serviceaccount/ca.crt
      bearer_token_file: /var/run/secrets/kubernetes.io/
serviceaccount/token
      kubernetes_sd_configs:
        - role: node
      relabel_configs:
        - action: labelmap
          regex: __meta_kubernetes_node_label_(.+)
        # Only for Kubernetes ^1.7.3.
        # See: https://github.com/prometheus/prometheus/
```

```
issues/2916
        - target_label: __address__
          replacement: kubernetes.default.svc:443
        - source_labels: [__meta_kubernetes_node_name]
          regex: (.+)
          target_label: __metrics_path__
          replacement: /api/v1/nodes/${1}/proxy/metrics/
cadvisor
      metric_relabel_configs:
        - action: replace
          source_labels: [id]
          regex: '^/machine\.slice/machine-rkt\\
x2d([^\\]+)\\.+/([^/]+)\.service$'
          target_label: rkt_container_name
          replacement: '${2}-${1}'
        - action: replace
          source_labels: [id]
          regex: '^/system\.slice/(.+)\.service$'
          target_label: systemd_service_name
          replacement: '${1}'

    - job_name: 'kubernetes-pods'
      kubernetes_sd_configs:
        - role: pod
      relabel_configs:
        - action: labelmap
          regex: __meta_kubernetes_pod_label_(.+)
        - source_labels: [__meta_kubernetes_namespace]
          action: replace
```

```
          target_label: kubernetes_namespace
        - source_labels: [__meta_kubernetes_pod_name]
          action: replace
          target_label: kubernetes_pod_name
        - source_labels: [__meta_kubernetes_pod_
annotation_prometheus_io_scrape]
          action: keep
          regex: true
        - source_labels: [__meta_kubernetes_pod_
annotation_prometheus_io_scheme]
          action: replace
          target_label: __scheme__
          regex: (https?)
        - source_labels: [__meta_kubernetes_pod_
annotation_prometheus_io_path]
          action: replace
          target_label: __metrics_path__
          regex: (.+)
        - source_labels: [__address__, __meta_kubernetes_
pod_prometheus_io_port]
          action: replace
          target_label: __address__
          regex: ([^:]+)(?::\d+)?;(\d+)
          replacement: $1:$2

    - job_name: 'kubernetes-apiservers'
      kubernetes_sd_configs:
        - role: endpoints
      scheme: https
```

```
      tls_config:
        ca_file: /var/run/secrets/kubernetes.io/
serviceaccount/ca.crt
      bearer_token_file: /var/run/secrets/kubernetes.io/
serviceaccount/token
      relabel_configs:
        - source_labels: [__meta_kubernetes_namespace, __
meta_kubernetes_service_name, __meta_kubernetes_endpoint_
port_name]
          action: keep
          regex: default;kubernetes;https

    - job_name: 'kubernetes-service-endpoints'
      kubernetes_sd_configs:
        - role: endpoints
      relabel_configs:
        - action: labelmap
          regex: __meta_kubernetes_service_label_(.+)
        - source_labels: [__meta_kubernetes_namespace]
          action: replace
          target_label: kubernetes_namespace
        - source_labels: [__meta_kubernetes_service_name]
          action: replace
          target_label: kubernetes_name
        - source_labels: [__meta_kubernetes_service_
annotation_prometheus_io_scrape]
          action: keep
          regex: true
        - source_labels: [__meta_kubernetes_service_
```

```
annotation_prometheus_io_scheme]
          action: replace
          target_label: __scheme__
          regex: (https?)
        - source_labels: [__meta_kubernetes_service_
annotation_prometheus_io_path]
          action: replace
          target_label: __metrics_path__
          regex: (.+)
        - source_labels: [__address__, __meta_kubernetes_
service_annotation_prometheus_io_port]
          action: replace
          target_label: __address__
          regex: (.+)(?::\d+);(\d+)
          replacement: $1:$2
```

Deploying Prometheus Rules configmap: this will create alert rules that will be relayed to Alertmanager for delivery.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-rules
  labels:
    name: prometheus-rules
  namespace: monitoring
data:
  alert-rules.yaml: |-
    groups:
```

```
      - name: Deployment
        rules:
        - alert: Deployment at 0 Replicas
          annotations:
            summary: Deployment {{$labels.deployment}} in
{{$labels.namespace}} is currently having no pods running
          expr: |
            sum(kube_deployment_status_replicas{pod_
template_hash=""}) by (deployment,namespace)  < 1
          for: 1m
          labels:
            team: devops

        - alert: HPA Scaling Limited
          annotations:
            summary: HPA named {{$labels.hpa}} in
{{$labels.namespace}} namespace has reached scaling
limited state
          expr: |
(sum(kube_hpa_status_
condition{condition="ScalingLimited",status="true"}) by
(hpa,namespace)) == 1
          for: 1m
          labels:
            team: devops

        - alert: HPA at MaxCapacity
          annotations:
            summary: HPA named {{$labels.hpa}} in
```

```
{{$labels.namespace}} namespace is running at Max Capacity
          expr: |
            ((sum(kube_hpa_spec_max_replicas) by
(hpa,namespace)) - (sum(kube_hpa_status_current_replicas)
by (hpa,namespace))) == 0
          for: 1m
          labels:
            team: devops

      - name: Pods
        rules:
        - alert: Container restarted
          annotations:
            summary: Container named {{$labels.container}}
in {{$labels.pod}} in {{$labels.namespace}} was restarted
          expr: |

sum(increase(kube_pod_container_status_restarts_
total{namespace!="kube-system",pod_template_hash=""}[1m]))
by (pod,namespace,container) > 0
          for: 0m
          labels:
            team: dev

        - alert: High Memory Usage of Container
          annotations:
            summary: Container named {{$labels.container}}
in {{$labels.pod}} in {{$labels.namespace}} is using more
```

```
than 75% of Memory Limit
          expr: |
            ((( sum(container_memory_usage_
bytes{image!="",container_name!="POD", namespace!="kube-
system"}) by (namespace,container_name,pod_name)  /
sum(container_spec_memory_limit_bytes{image!="",container_
name!="POD",namespace!="kube-system"}) by
(namespace,container_name,pod_name) ) * 100 ) < +Inf ) >
75
          for: 5m
          labels:
            team: dev

        - alert: High CPU Usage of Container
          annotations:
            summary: Container named {{$labels.container}}
in {{$labels.pod}} in {{$labels.namespace}} is using more
than 75% of CPU Limit
          expr: |

((sum(irate(container_cpu_usage_seconds_
total{image!="",container_name!="POD", namespace!="kube-
system"}[30s])) by (namespace,container_name,pod_name)
/ sum(container_spec_cpu_quota{image!="",container_
name!="POD", namespace!="kube-system"} / container_
spec_cpu_period{image!="",container_name!="POD",
namespace!="kube-system"}) by (namespace,container_
name,pod_name) ) * 100)  > 75
          for: 5m
```

```
          labels:
            team: dev


      - name: Nodes
        rules:
        - alert: High Node Memory Usage
          annotations:
            summary: Node {{$labels.kubernetes_io_
hostname}} has more than 80% memory used. Plan Capcity
          expr: |
            (sum (container_memory_working_set_
bytes{id="/",container_name!="POD"}) by (kubernetes_io_
hostname) / sum (machine_memory_bytes{}) by (kubernetes_
io_hostname) * 100) > 80
          for: 5m
          labels:
            team: devops


        - alert: High Node CPU Usage
          annotations:
            summary: Node {{$labels.kubernetes_io_
hostname}} has more than 80% allocatable cpu used. Plan
Capacity.
          expr: |
            (sum(rate(container_cpu_usage_seconds_
total{id="/", container_name!="POD"}[1m])) by (kubernetes_
io_hostname) / sum(machine_cpu_cores) by (kubernetes_io_
hostname)  * 100) > 80
          for: 5m
          labels:
```

```
            team: devops
        - alert: High Node Disk Usage
          annotations:
            summary: Node {{$labels.kubernetes_io_
hostname}} has more than 85% disk used. Plan Capacity.
          expr: |

(sum(container_fs_usage_bytes{device=~"^/dev/[sv]d[a-z]
[1-9]$",id="/",container_name!="POD"}) by (kubernetes_
io_hostname) / sum(container_fs_limit_bytes{container_
name!="POD",device=~"^/dev/[sv]d[a-z][1-9]$",id="/"}) by
(kubernetes_io_hostname)) * 100 > 85
          for: 5m
          labels:
            team: devops
```

Deploying Prometheus Stateful Set

```
apiVersion: storage.k8s.io/v1beta1
kind: StorageClass
metadata:
  name: fast
  namespace: monitoring
provisioner: kubernetes.io/gce-pd
allowVolumeExpansion: true
---
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
```

```
  name: prometheus
  namespace: monitoring
spec:
  replicas: 3
  serviceName: prometheus-service
  template:
    metadata:
      labels:
        app: prometheus
        thanos-store-api: "true"
    spec:
      serviceAccountName: monitoring
      containers:
        - name: prometheus
          image: prom/prometheus:v2.4.3
          args:
            - "--config.file=/etc/prometheus-shared/
prometheus.yaml"
            - "--storage.tsdb.path=/prometheus/"
            - "--web.enable-lifecycle"
            - "--storage.tsdb.no-lockfile"
            - "--storage.tsdb.min-block-duration=2h"
            - "--storage.tsdb.max-block-duration=2h"
          ports:
            - name: prometheus
              containerPort: 9090
          volumeMounts:
            - name: prometheus-storage
              mountPath: /prometheus/
```

```
            - name: prometheus-config-shared
              mountPath: /etc/prometheus-shared/
            - name: prometheus-rules
              mountPath: /etc/prometheus/rules
        - name: thanos
          image: quay.io/thanos/thanos:v0.8.0
          args:
            - "sidecar"
            - "--log.level=debug"
            - "--tsdb.path=/prometheus"
            - "--prometheus.url=http://127.0.0.1:9090"
            - "--objstore.config={type: GCS, config:
{bucket: prometheus-long-term}}"
            - "--reloader.config-file=/etc/prometheus/
prometheus.yaml.tmpl"
            - "--reloader.config-envsubst-file=/etc/
prometheus-shared/prometheus.yaml"
            - "--reloader.rule-dir=/etc/prometheus/rules/"
          env:
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name : GOOGLE_APPLICATION_CREDENTIALS
              value: /etc/secret/thanos-gcs-credentials.json
          ports:
            - name: http-sidecar
              containerPort: 10902
            - name: grpc
```

```
        containerPort: 10901
    livenessProbe:
        httpGet:
          port: 10902
          path: /-/healthy
    readinessProbe:
      httpGet:
        port: 10902
        path: /-/ready
    volumeMounts:
      - name: prometheus-storage
        mountPath: /prometheus
      - name: prometheus-config-shared
        mountPath: /etc/prometheus-shared/
      - name: prometheus-config
        mountPath: /etc/prometheus
      - name: prometheus-rules
        mountPath: /etc/prometheus/rules
      - name: thanos-gcs-credentials
        mountPath: /etc/secret
        readOnly: false
securityContext:
  fsGroup: 2000
  runAsNonRoot: true
  runAsUser: 1000
volumes:
  - name: prometheus-config
    configMap:
      defaultMode: 420
```

```
          name: prometheus-server-conf
      - name: prometheus-config-shared
        emptyDir: {}
      - name: prometheus-rules
        configMap:
          name: prometheus-rules
      - name: thanos-gcs-credentials
        secret:
          secretName: thanos-gcs-credentials
volumeClaimTemplates:
- metadata:
    name: prometheus-storage
    namespace: monitoring
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: fast
    resources:
      requests:
        storage: 20Gi
```

It is important to understand the following about the above manifest:

1. Prometheus is deployed as a stateful set with three replicas. Each replica provisions its own persistent volume dynamically.
2. Prometheus configuration is generated by the Thanos Sidecar container using the template file created above.
3. Thanos handles data compaction and therefore we need to set **--storage.tsdb.min-block-duration=2h** and **--storage.tsdb.max-block-duration=2h**

4. Prometheus stateful set is labeled as **thanos-store-api: "true"** so that each pod gets discovered by the headless service (we will show you how to do that next). This headless service will be used by **Thanos Query** to query data across all the Prometheus instances.
5. We apply the same label to the **Thanos Store** and **Thanos Ruler** component so that they are also discovered by the querier and can be used for querying metrics.
6. The GCS bucket credentials path is provided using the **GOOGLE_APPLICATION_CREDENTIALS** environment variable. The configuration file is mounted to that from the secret created as a part of the prerequisites.

Deploying Prometheus Services

```
apiVersion: v1
kind: Service
metadata:
  name: prometheus-0-service
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
  namespace: monitoring
  labels:
    name: prometheus
spec:
  selector:
    statefulset.kubernetes.io/pod-name: prometheus-0
  ports:
```

```
    - name: prometheus
      port: 8080
      targetPort: prometheus
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus-1-service
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
  namespace: monitoring
  labels:
    name: prometheus
spec:
  selector:
    statefulset.kubernetes.io/pod-name: prometheus-1
  ports:
    - name: prometheus
      port: 8080
      targetPort: prometheus
---
apiVersion: v1
kind: Service
metadata:
  name: prometheus-2-service
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
```

```
  namespace: monitoring
  labels:
    name: prometheus
spec:
  selector:
    statefulset.kubernetes.io/pod-name: prometheus-2
  ports:
    - name: prometheus
      port: 8080
      targetPort: prometheus
---
#This service creates a srv record for querier to find
about store-api's
apiVersion: v1
kind: Service
metadata:
  name: thanos-store-gateway
  namespace: monitoring
spec:
  type: ClusterIP
  clusterIP: None
  ports:
    - name: grpc
      port: 10901
      targetPort: grpc
  selector:
    thanos-store-api: "true"
```

We create different services for each Prometheus pod in the stateful set. These are not strictly necessary, but are created only

for debugging purposes. The purpose of thanos-store-gateway headless service has been explained above. Next, we will expose the Prometheus services using an ingress object.

Deploying Thanos Query: this is one of the main components of Thanos deployment. Note the following

1. The container argument **--store=dnssrv+thanos-store-gateway:10901** helps discover all the components from which metric data should be queried.
2. The service **thanos-querier** provides a web interface to run PromQL queries. It also has the option to deduplicate data across various Prometheus clusters.
3. From here, we provide Grafana as a datasource for all the dashboards.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: thanos-querier
  namespace: monitoring
  labels:
    app: thanos-querier
spec:
  replicas: 1
```

```
selector:
  matchLabels:
    app: thanos-querier
template:
  metadata:
    labels:
    app: thanos-querier
  spec:
    containers:
    - name: thanos
      image: quay.io/thanos/thanos:v0.8.0
      args:
      - query
      - --log.level=debug
      - --query.replica-label=replica
      - --store=dnssrv+thanos-store-gateway:10901
      ports:
      - name: http
        containerPort: 10902
      - name: grpc
        containerPort: 10901
      livenessProbe:
        httpGet:
          port: http
          path: /-/healthy
      readinessProbe:
        httpGet:
          port: http
          path: /-/ready
```

```
---
apiVersion: v1
kind: Service
metadata:
  labels:
    app: thanos-querier
  name: thanos-querier
  namespace: monitoring
spec:
  ports:
  - port: 9090
    protocol: TCP
    targetPort: http
    name: http
  selector:
    app: thanos-querier
```

Deploying Thanos Store Gateway: this will create the store component which serves metrics from the object storage to the querier.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
```

```
  name: thanos-store-gateway
  namespace: monitoring
  labels:
    app: thanos-store-gateway
spec:
  replicas: 1
  selector:
    matchLabels:
      app: thanos-store-gateway
  serviceName: thanos-store-gateway
  template:
    metadata:
      labels:
        app: thanos-store-gateway
        thanos-store-api: "true"
    spec:
      containers:
        - name: thanos
          image: quay.io/thanos/thanos:v0.8.0
          args:
          - "store"
          - "--log.level=debug"
          - "--data-dir=/data"
          - "--objstore.config={type: GCS, config: {bucket:
prometheus-long-term}}"
          - "--index-cache-size=500MB"
          - "--chunk-pool-size=500MB"
          env:
            - name : GOOGLE_APPLICATION_CREDENTIALS
```

```
                value: /etc/secret/thanos-gcs-credentials.json
            ports:
            - name: http
              containerPort: 10902
            - name: grpc
              containerPort: 10901
            livenessProbe:
              httpGet:
                port: 10902
                path: /-/healthy
            readinessProbe:
              httpGet:
                port: 10902
                path: /-/ready
            volumeMounts:
              - name: thanos-gcs-credentials
                mountPath: /etc/secret
                readOnly: false
      volumes:
        - name: thanos-gcs-credentials
          secret:
            secretName: thanos-gcs-credentials
---
```

Deploying Thanos Compact

```
apiVersion: v1
kind: Namespace
metadata:
```

```
  name: monitoring
---
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
  name: thanos-compactor
  namespace: monitoring
  labels:
    app: thanos-compactor
spec:
  replicas: 1
  selector:
    matchLabels:
      app: thanos-compactor
  serviceName: thanos-compactor
  template:
    metadata:
      labels:
        app: thanos-compactor
    spec:
      containers:
        - name: thanos
          image: quay.io/thanos/thanos:v0.8.0
          args:
            - "compact"
            - "--log.level=debug"
            - "--data-dir=/data"
            - "--objstore.config={type: GCS, config:
{bucket: prometheus-long-term}}"
```

```
            - "--wait"
          env:
            - name : GOOGLE_APPLICATION_CREDENTIALS
              value: /etc/secret/thanos-gcs-credentials.json
          ports:
            - name: http
              containerPort: 10902
          livenessProbe:
            httpGet:
              port: 10902
              path: /-/healthy
          readinessProbe:
            httpGet:
              port: 10902
              path: /-/ready
          volumeMounts:
            - name: thanos-gcs-credentials
              mountPath: /etc/secret
              readOnly: false
      volumes:
        - name: thanos-gcs-credentials
          secret:
            secretName: thanos-gcs-credentials
```

Deploying Thanos Ruler

```
apiVersion: v1
kind: Namespace
metadata:
```

```
  name: monitoring
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: thanos-ruler-rules
  namespace: monitoring
data:
  alert_down_services.rules.yaml: |
    groups:
    - name: metamonitoring
      rules:
      - alert: PrometheusReplicaDown
        annotations:
          message: Prometheus replica in cluster
{{$labels.cluster}} has disappeared from Prometheus target
discovery.
        expr: |
          sum(up{cluster="prometheus-ha",
instance=~".*:9090", job="kubernetes-service-endpoints"})
by (job,cluster) < 3
        for: 15s
        labels:
          severity: critical
---
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
  labels:
```

```
    app: thanos-ruler
  name: thanos-ruler
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: thanos-ruler
  serviceName: thanos-ruler
  template:
    metadata:
      labels:
        app: thanos-ruler
        thanos-store-api: "true"
    spec:
      containers:
        - name: thanos
          image: quay.io/thanos/thanos:v0.8.0
          args:
            - rule
            - --log.level=debug
            - --data-dir=/data
            - --eval-interval=15s
            - --rule-file=/etc/thanos-ruler/*.rules.yaml
            - --alertmanagers.url=http://alertmanager:9093
            - --query=thanos-querier:9090
            - "--objstore.config={type: GCS, config:
{bucket: thanos-ruler}}"
            - --label=ruler_cluster="prometheus-ha"
```

```
        - --label=replica="$(POD_NAME)"
      env:
        - name : GOOGLE_APPLICATION_CREDENTIALS
          value: /etc/secret/thanos-gcs-credentials.json
        - name: POD_NAME
          valueFrom:
            fieldRef:
              fieldPath: metadata.name
      ports:
        - name: http
          containerPort: 10902
        - name: grpc
          containerPort: 10901
      livenessProbe:
        httpGet:
          port: http
          path: /-/healthy
      readinessProbe:
        httpGet:
          port: http
          path: /-/ready
      volumeMounts:
        - mountPath: /etc/thanos-ruler
          name: config
        - name: thanos-gcs-credentials
          mountPath: /etc/secret
          readOnly: false
  volumes:
    - configMap:
```

```
            name: thanos-ruler-rules
          name: config
        - name: thanos-gcs-credentials
          secret:
            secretName: thanos-gcs-credentials
---
apiVersion: v1
kind: Service
metadata:
  labels:
    app: thanos-ruler
  name: thanos-ruler
  namespace: monitoring
spec:
  ports:
    - port: 9090
      protocol: TCP
      targetPort: http
      name: http
  selector:
    app: thanos-ruler
```

If you go to the interactive shell in the same namespace as our workloads to check which pods thanos-store-gateway resolves, you will see something like this:

```
root@my-shell-95cb5df57-4q6w8:/# nslookup thanos-store-
gateway
Server: 10.63.240.10
```

```
Address:        10.63.240.10#53


Name:   thanos-store-gateway.monitoring.svc.cluster.local
Address: 10.60.25.2
Name:   thanos-store-gateway.monitoring.svc.cluster.local
Address: 10.60.25.4
Name:   thanos-store-gateway.monitoring.svc.cluster.local
Address: 10.60.30.2
Name:   thanos-store-gateway.monitoring.svc.cluster.local
Address: 10.60.30.8
Name:   thanos-store-gateway.monitoring.svc.cluster.local
Address: 10.60.31.2


root@my-shell-95cb5df57-4q6w8:/# exit
```

The IPs returned above correspond to our Prometheus pods, thanos-store and thanos-ruler. This can be verified as:

```
$ kubectl get pods -o wide -l thanos-store-api="true"
NAME                     READY   STATUS    RESTARTS   AGE    IP           NODE                                    NOMINATED NODE   READINESS
GATES
prometheus-0             2/2     Running   0          100m   10.60.31.2   gke-demo-1-pool-1-649cbe02-jdnv         <none>           <none>
prometheus-1             2/2     Running   0          14h    10.60.30.2   gke-demo-1-pool-1-7533d618-kxkd         <none>           <none>
prometheus-2             2/2     Running   0          31h    10.60.25.2   gke-demo-1-pool-1-4e9889dd-27gc         <none>           <none>
thanos-ruler-0           1/1     Running   0          100m   10.60.30.8   gke-demo-1-pool-1-7533d618-kxkd         <none>           <none>
thanos-store-gateway-0   1/1     Running   0          14h    10.60.25.4   gke-demo-1-pool-1-4e9889dd-27gc         <none>           <none>
```

Deploying Alertmanager: This will create our alertmanager deployment. It will deliver all the alerts generated as per Prometheus Rules.

```
apiVersion: v1
kind: Namespace
metadata:
```

```
  name: monitoring
---
kind: ConfigMap
apiVersion: v1
metadata:
  name: alertmanager
  namespace: monitoring
data:
  config.yml: |-
    global:
      resolve_timeout: 5m
      slack_api_url: "<your_slack_hook>"
      victorops_api_url: "<your_victorops_hook>"

    templates:
    - '/etc/alertmanager-templates/*.tmpl'
    route:
      group_by: ['alertname', 'cluster', 'service']
      group_wait: 10s
      group_interval: 1m
      repeat_interval: 5m
      receiver: default
      routes:
      - match:
          team: devops
        receiver: devops
        continue: true
      - match:
          team: dev
```

```
        receiver: dev
        continue: true

    receivers:
    - name: 'default'

    - name: 'devops'
      victorops_configs:
      - api_key: '<YOUR_API_KEY>'
        routing_key: 'devops'
        message_type: 'CRITICAL'
        entity_display_name: '{{ .CommonLabels.alertname
}}'
        state_message: 'Alert: {{ .CommonLabels.alertname
}}. Summary:{{ .CommonAnnotations.summary }}. RawData: {{
.CommonLabels }}'
      slack_configs:
      - channel: '#k8-alerts'
        send_resolved: true

    - name: 'dev'
      victorops_configs:
      - api_key: '<YOUR_API_KEY>'
        routing_key: 'dev'
        message_type: 'CRITICAL'
        entity_display_name: '{{ .CommonLabels.alertname
}}'
        state_message: 'Alert: {{ .CommonLabels.alertname
}}. Summary:{{ .CommonAnnotations.summary }}. RawData: {{
```

```
.CommonLabels }}'
      slack_configs:
      - channel: '#k8-alerts'
        send_resolved: true

---
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  name: alertmanager
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: alertmanager
  template:
    metadata:
      name: alertmanager
      labels:
        app: alertmanager
    spec:
      containers:
      - name: alertmanager
        image: prom/alertmanager:v0.15.3
        args:
          - '--config.file=/etc/alertmanager/config.yml'
          - '--storage.path=/alertmanager'
        ports:
```

```
        - name: alertmanager
          containerPort: 9093
        volumeMounts:
        - name: config-volume
          mountPath: /etc/alertmanager
        - name: alertmanager
          mountPath: /alertmanager
      volumes:
      - name: config-volume
        configMap:
          name: alertmanager
      - name: alertmanager
        emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  annotations:
    prometheus.io/scrape: 'true'
    prometheus.io/path: '/metrics'
  labels:
    name: alertmanager
  name: alertmanager
  namespace: monitoring
spec:
  selector:
    app: alertmanager
  ports:
  - name: alertmanager
```

```
    protocol: TCP
    port: 9093
    targetPort: 9093
```

Deploying Kubestate Metrics: Kubestate metrics deployment is needed to relay some important container metrics. These metrics are not natively exposed by the kubelet and are not directly available to Prometheus.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: rbac.authorization.k8s.io/v1
# kubernetes versions before 1.8.0 should use rbac.
authorization.k8s.io/v1beta1
kind: ClusterRoleBinding
metadata:
  name: kube-state-metrics
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: kube-state-metrics
subjects:
- kind: ServiceAccount
  name: kube-state-metrics
  namespace: monitoring
---
```

```
apiVersion: rbac.authorization.k8s.io/v1
# kubernetes versions before 1.8.0 should use rbac.
authorization.k8s.io/v1beta1
kind: ClusterRole
metadata:
  name: kube-state-metrics
rules:
- apiGroups: [""]
  resources:
  - configmaps
  - secrets
  - nodes
  - pods
  - services
  - resourcequotas
  - replicationcontrollers
  - limitranges
  - persistentvolumeclaims
  - persistentvolumes
  - namespaces
  - endpoints
  verbs: ["list", "watch"]
- apiGroups: ["extensions"]
  resources:
  - daemonsets
  - deployments
  - replicasets
  verbs: ["list", "watch"]
- apiGroups: ["apps"]
```

```
  resources:
  - statefulsets
  verbs: ["list", "watch"]
- apiGroups: ["batch"]
  resources:
  - cronjobs
  - jobs
  verbs: ["list", "watch"]
- apiGroups: ["autoscaling"]
  resources:
  - horizontalpodautoscalers
  verbs: ["list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
# kubernetes versions before 1.8.0 should use rbac.
authorization.k8s.io/v1beta1
kind: RoleBinding
metadata:
  name: kube-state-metrics
  namespace: monitoring
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: kube-state-metrics-resizer
subjects:
- kind: ServiceAccount
  name: kube-state-metrics
  namespace: monitoring
---
```

```
apiVersion: rbac.authorization.k8s.io/v1
# kubernetes versions before 1.8.0 should use rbac.
authorization.k8s.io/v1beta1
kind: Role
metadata:
  namespace: monitoring
  name: kube-state-metrics-resizer
rules:
- apiGroups: [""]
  resources:
  - pods
  verbs: ["get"]
- apiGroups: ["extensions"]
  resources:
  - deployments
  resourceNames: ["kube-state-metrics"]
  verbs: ["get", "update"]
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: kube-state-metrics
  namespace: monitoring
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kube-state-metrics
  namespace: monitoring
```

```
spec:
  selector:
    matchLabels:
      k8s-app: kube-state-metrics
  replicas: 1
  template:
    metadata:
      labels:
        k8s-app: kube-state-metrics
    spec:
      serviceAccountName: kube-state-metrics
      containers:
      - name: kube-state-metrics
        image: quay.io/mxinden/kube-state-metrics:v1.4.0-
gzip.3
        ports:
        - name: http-metrics
          containerPort: 8080
        - name: telemetry
          containerPort: 8081
        readinessProbe:
          httpGet:
            path: /healthz
            port: 8080
          initialDelaySeconds: 5
          timeoutSeconds: 5
      - name: addon-resizer
        image: k8s.gcr.io/addon-resizer:1.8.3
        resources:
```

```
          limits:
            cpu: 150m
            memory: 50Mi
          requests:
            cpu: 150m
            memory: 50Mi
        env:
          - name: MY_POD_NAME
            valueFrom:
              fieldRef:
                fieldPath: metadata.name
          - name: MY_POD_NAMESPACE
            valueFrom:
              fieldRef:
                fieldPath: metadata.namespace
        command:
          - /pod_nanny
          - --container=kube-state-metrics
          - --cpu=100m
          - --extra-cpu=1m
          - --memory=100Mi
          - --extra-memory=2Mi
          - --threshold=5
          - --deployment=kube-state-metrics
---
apiVersion: v1
kind: Service
metadata:
  name: kube-state-metrics
```

```
  namespace: monitoring
  labels:
    k8s-app: kube-state-metrics
  annotations:
    prometheus.io/scrape: 'true'
spec:
  ports:
  - name: http-metrics
    port: 8080
    targetPort: http-metrics
    protocol: TCP
  - name: telemetry
    port: 8081
    targetPort: telemetry
    protocol: TCP
  selector:
    k8s-app: kube-state-metrics
```

Deploying Node-exporter Daemonset: Node-exporter daemonset runs a node-exporter pod on each node. It exposes very important node metrics that can be pulled by Prometheus instances.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: extensions/v1beta1
kind: DaemonSet
```

```
metadata:
  name: node-exporter
  namespace: monitoring
  labels:
    name: node-exporter
spec:
  template:
    metadata:
      labels:
        name: node-exporter
      annotations:
         prometheus.io/scrape: "true"
         prometheus.io/port: "9100"
    spec:
      hostPID: true
      hostIPC: true
      hostNetwork: true
      containers:
        - name: node-exporter
          image: prom/node-exporter:v0.16.0
          securityContext:
            privileged: true
          args:
            - --path.procfs=/host/proc
            - --path.sysfs=/host/sys
          ports:
            - containerPort: 9100
              protocol: TCP
          resources:
```

```
        limits:
          cpu: 100m
          memory: 100Mi
        requests:
          cpu: 10m
          memory: 100Mi
      volumeMounts:
        - name: dev
          mountPath: /host/dev
        - name: proc
          mountPath: /host/proc
        - name: sys
          mountPath: /host/sys
        - name: rootfs
          mountPath: /rootfs
  volumes:
    - name: proc
      hostPath:
        path: /proc
    - name: dev
      hostPath:
        path: /dev
    - name: sys
      hostPath:
        path: /sys
    - name: rootfs
      hostPath:
        path: /
```

Deploying Grafana This will create our Grafana deployment and Service which will be exposed using our ingress object. We should add thanos-querier as the datasource for our Grafana deployment. In order to do so:

1. Click on **Add DataSource**
2. Set **Name: DS_PROMETHEUS**
3. Set **Type: Prometheus**
4. Set **URL:** http://thanos-querier:9090
5. Save and Test. You can now build your custom dashboards or simply import dashboards from grafana.net. Dashboard #315 and #1471 are a very good place to start.

```
apiVersion: v1
kind: Namespace
metadata:
  name: monitoring
---
apiVersion: storage.k8s.io/v1beta1
kind: StorageClass
metadata:
  name: fast
  namespace: monitoring
provisioner: kubernetes.io/gce-pd
allowVolumeExpansion: true
---
apiVersion: apps/v1beta1
kind: StatefulSet
metadata:
```

```
  name: grafana
  namespace: monitoring
spec:
  replicas: 1
  serviceName: grafana
  template:
    metadata:
      labels:
        task: monitoring
        k8s-app: grafana
    spec:
      containers:
      - name: grafana
        image: k8s.gcr.io/heapster-grafana-amd64:v5.0.4
        ports:
        - containerPort: 3000
          protocol: TCP
        volumeMounts:
        - mountPath: /etc/ssl/certs
          name: ca-certificates
          readOnly: true
        - mountPath: /var
          name: grafana-storage
        env:
        - name: GF_SERVER_HTTP_PORT
          value: "3000"
          # The following env variables are required to
make Grafana accessible via
          # the kubernetes api-server proxy. On production
```

```
clusters, we recommend
          # removing these env variables, setup auth for
grafana, and expose the grafana
          # service using a LoadBalancer or a public IP.
        - name: GF_AUTH_BASIC_ENABLED
          value: "false"
        - name: GF_AUTH_ANONYMOUS_ENABLED
          value: "true"
        - name: GF_AUTH_ANONYMOUS_ORG_ROLE
          value: Admin
        - name: GF_SERVER_ROOT_URL
          # If you're only using the API Server proxy, set
this value instead:
          # value: /api/v1/namespaces/kube-system/
services/monitoring-grafana/proxy
          value: /
      volumes:
      - name: ca-certificates
        hostPath:
          path: /etc/ssl/certs
  volumeClaimTemplates:
  - metadata:
      name: grafana-storage
      namespace: monitoring
    spec:
      accessModes: [ "ReadWriteOnce" ]
      storageClassName: fast
      resources:
        requests:
```

```
          storage: 5Gi
---
apiVersion: v1
kind: Service
metadata:
  labels:
    kubernetes.io/cluster-service: 'true'
    kubernetes.io/name: grafana
  name: grafana
  namespace: monitoring
spec:
  ports:
  - port: 3000
    targetPort: 3000
  selector:
    k8s-app: grafana
```

Deploying the Ingress Object: This is the final piece in the puzzle. This will help expose all our services outside the Kubernetes cluster and help us access them.

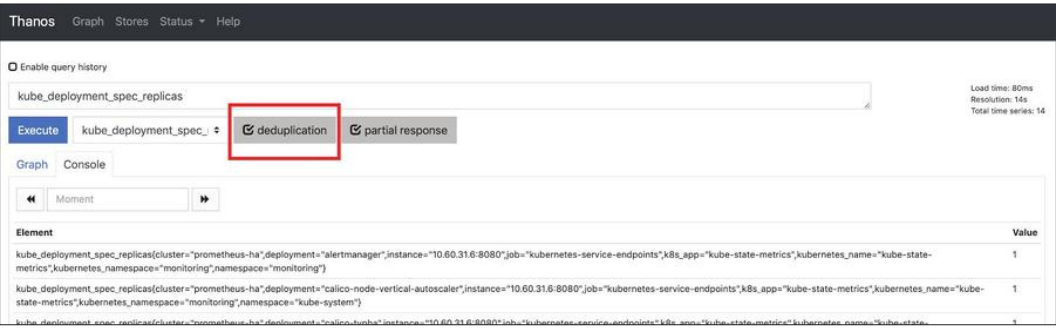
Make sure you replace **<yourdomain>** with your own domain name. You can point the ingress-controller's service to.

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: monitoring-ingress
  namespace: monitoring
```

```
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  rules:
  - host: grafana.<yourdomain>.com
    http:
      paths:
      - path: /
        backend:
          serviceName: grafana
          servicePort: 3000
  - host: prometheus-0.<yourdomain>.com
    http:
      paths:
      - path: /
        backend:
          serviceName: prometheus-0-service
          servicePort: 8080
  - host: prometheus-1.<yourdomain>.com
    http:
      paths:
      - path: /
        backend:
          serviceName: prometheus-1-service
          servicePort: 8080
  - host: prometheus-2.<yourdomain>.com
    http:
      paths:
      - path: /
```

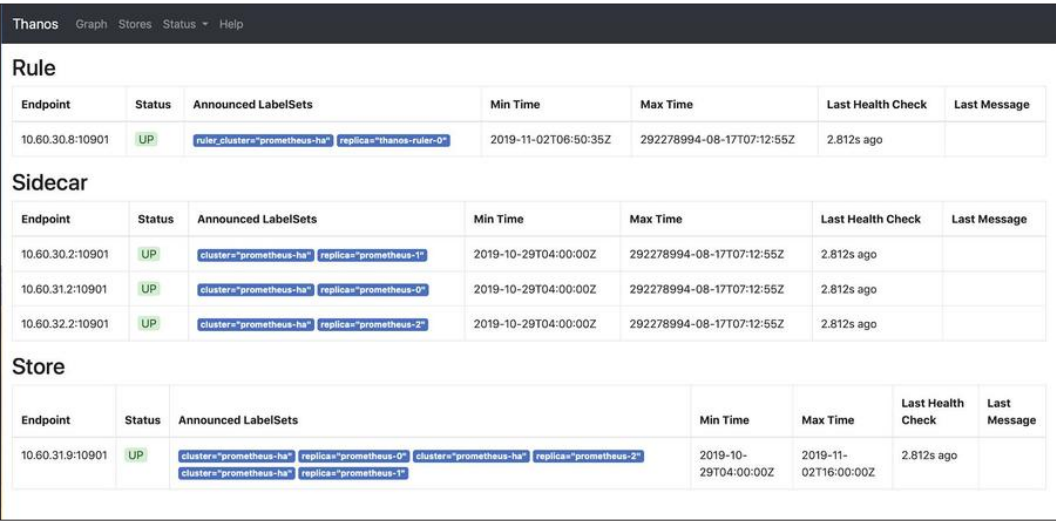
```
      backend:
        serviceName: prometheus-2-service
        servicePort: 8080
- host: alertmanager.<yourdomain>.com
  http:
    paths:
    - path: /
      backend:
        serviceName: alertmanager
        servicePort: 9093
- host: thanos-querier.<yourdomain>.com
  http:
    paths:
    - path: /
      backend:
        serviceName: thanos-querier
        servicePort: 9090
- host: thanos-ruler.<yourdomain>.com
  http:
    paths:
    - path: /
      backend:
        serviceName: thanos-ruler
        servicePort: 9090
```

You should now be able to access Thanos Querier at http://thanos-querier.<yourdomain>.com . It will look something like this:



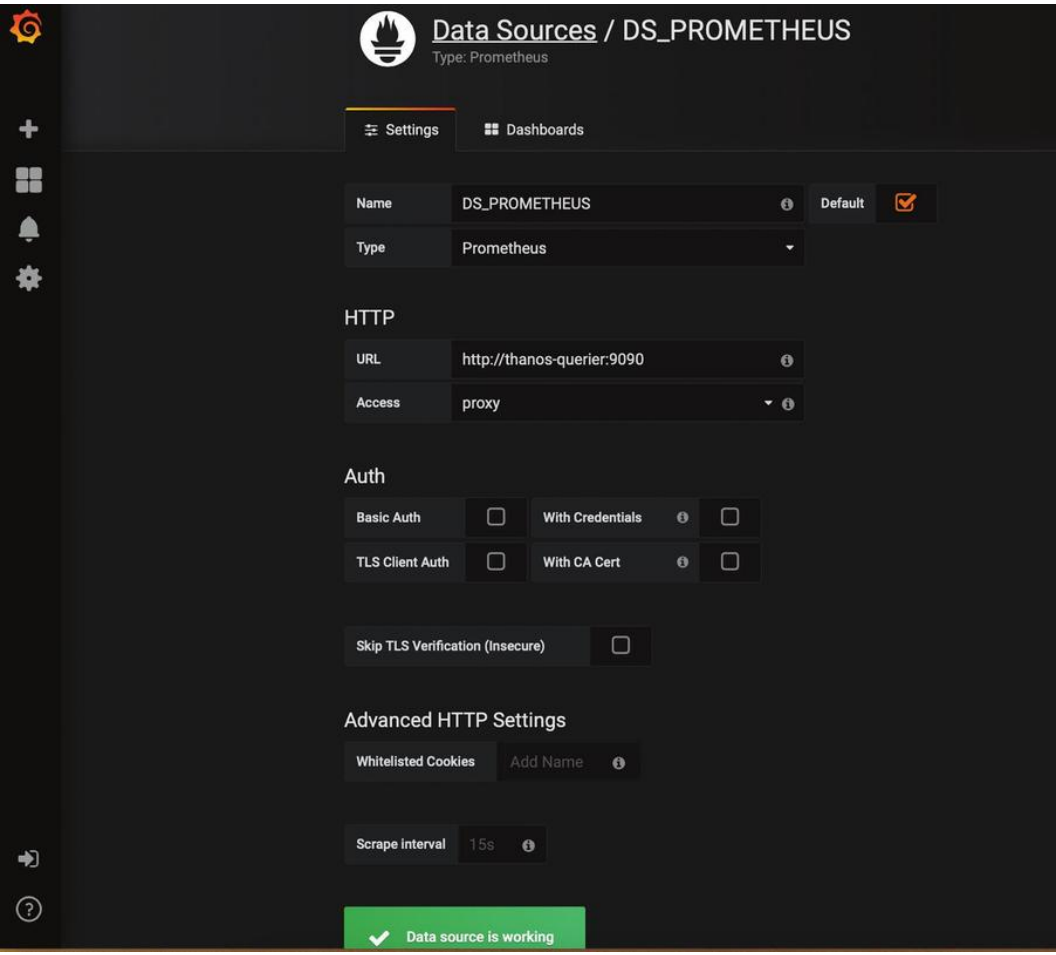
Make sure deduplication is selected.

If you click on **Stores**, you will be able to see all the active endpoints discovered by **thanos-store-gateway**.

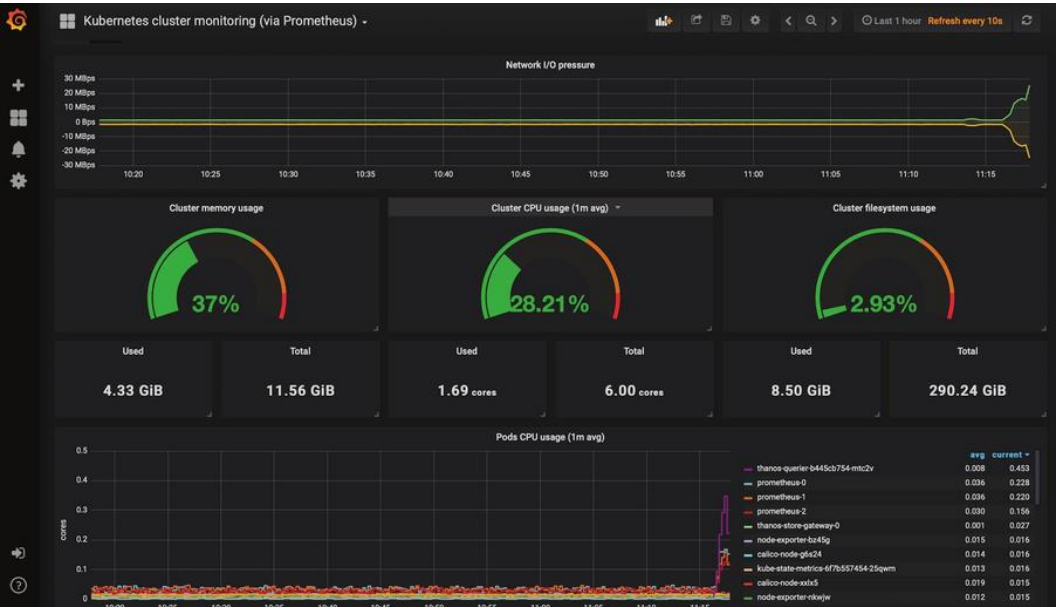


13.6. Grafana dashboards

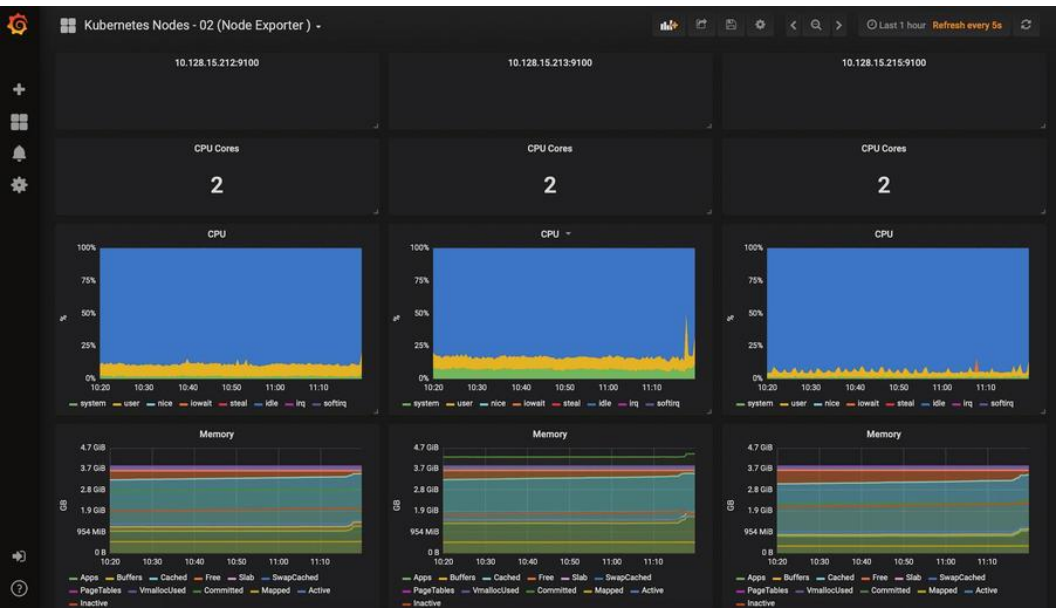
Finally, you add Thanos Querier as the datasource in Grafana and start creating dashboards.



Kubernetes Cluster Monitoring Dashboard:



Kubernetes Node Monitoring Dashboard:



13.7. Conclusion

Integrating Thanos with Prometheus allows you to scale Prometheus horizontally. Since Thanos Querier can pull metrics from other querier instances, you can pull metrics across clusters and visualize them in Grafana dashboards. Thanos lets us archive metric data in an object store that provides infinite storage for our monitoring system. It also serves metrics from the object storage itself. A major operating cost for this setup can be attributed to the object storage (S3 or GCS). This can be reduced if we apply appropriate retention policies to them.

Today's setup requires quite a bit of configuration on your part. The manifests provided above have been tested in a production environment and should make the process easy for you. Feel free to reach out should you have any questions around them. If you decide that you don't want to do the configuration yourself, we have a hosted Prometheus offering where you can offload it to us and we will manage it for you. Try a free trial, or book a demo to talk to us directly.

Example 3: Monitoring Redis Clusters with Prometheus

14.1 Introduction

This article will outline what Redis database monitoring is and how to set up a Redis database monitoring system with MetricFire. Then we'll show what the final graphs and dashboards look like when displayed on Grafana. We will be using MetricFire's hosted Prometheus and Grafana to power the monitoring, and we'll use a simulated Redis DB to generate the data for the Grafana dashboards.

Here's what you'll be able to do by the end of this example: make a beautiful Prometheus-driven Grafana dashboard monitoring your Redis Cluster.

14.2 What are Redis DB and Redis Clusters?

A Redis Database is an in-memory Data Structure Store which organizes data into key-value pairs which can be used as a database, cache, or message broker. Redis DB is open-source, and there are various hosted services offered. A Redis data structure is efficient both in terms of performance and ease of use. Redis DBs are usually used for data that needs to be retrieved quickly, such as a password that is connected to a single username, or for data that is transient and can be deleted shortly afterwards. The simple command-line interface reduces developmental effort and the in-memory component reduces latency and increases throughput.

A Redis cluster is an implementation of Redis DB that allows data to be automatically sharded across multiple Redis nodes. Clusters also provide a level of redundancy and availability during partitioning, meaning data can be communicated/transmitted when a node is recovering or failing. Redis Clusters also run on a master-slave model which protects data in the event of a “master” node failure.

14.3 How does MetricFire monitor Redis?

Each Redis cluster has a metrics_exporter component that listens on port 8070, and acts as a Prometheus endpoint from which Prometheus can get metrics. Monitoring Redis metrics with Prometheus causes little to no load to the database. Redis will push the required metrics to the Prometheus endpoint where users can scrape Prometheus for the available Redis metrics, avoiding scraping Redis each time a metric is queried. You can monitor the total number of keys in a Redis cluster, the current number of commands processed, memory usage, and total Redis connections. In addition, you can monitor cluster-wide data, individual node data, or single database data.

If you are using hosted Prometheus by MetricFire, it works in exactly the same way. MetricFire scrapes the Redis DB endpoint for metrics information, and displays it automatically in the Grafana dashboard.

14.4 How do you set up Redis Cluster Monitoring with MetricFire?

a. Install Prometheus and Redis in MetricFire UI

- Go to Add-Ons menu on the left-hand side of the MetricFire UI.
- Find Your Prometheus API Key.
- Edit the prometheus.yml file to include remote_write and remote_read sections with the API Key as bearer_token.

b. Edit Prometheus Configuration to include Redis Enterprise Job

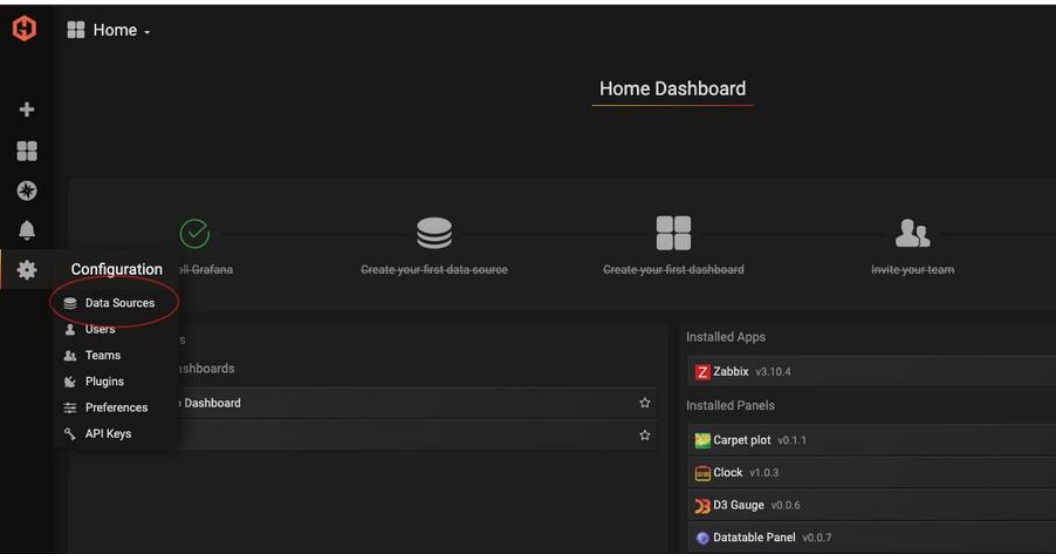
According to docs.redislabs.com, copy the Prometheus configuration from the above step into ./prometheus/prometheus.yml in your current folder. The cluster name can be either the fully-qualified domain name or the IP address.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

# Attach these labels to any time series or alerts when communicating with
# external systems (federation, remote storage, Alertmanager).
  external_labels:
    monitor: "prometheus-stack-monitor"

# Load and evaluate rules in this file every 'evaluation_interval' seconds.
#rule_files:
# - "first.rules"
# - "second.rules"

scrape_configs:
# scrape Prometheus itself
  - job_name: prometheus
    scrape_interval: 10s
    scrape_timeout: 5s
    static_configs:
      - targets: ["localhost:9090"]
```

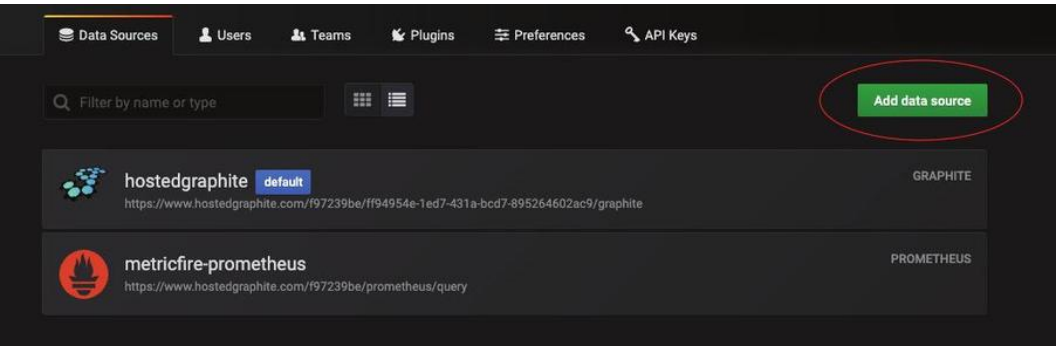
```
# scrape Redis Enterprise
  - job_name: redis-enterprise
    scrape_interval: 30s
    scrape_timeout: 30s
    metrics_path: /
    scheme: https
    tls_config:
      insecure_skip_verify: true
    static_configs:
      - targets: ["<cluster_name>:8070"]
```

c. Add Data Source to Grafana Dashboard in the MetricFire UI.

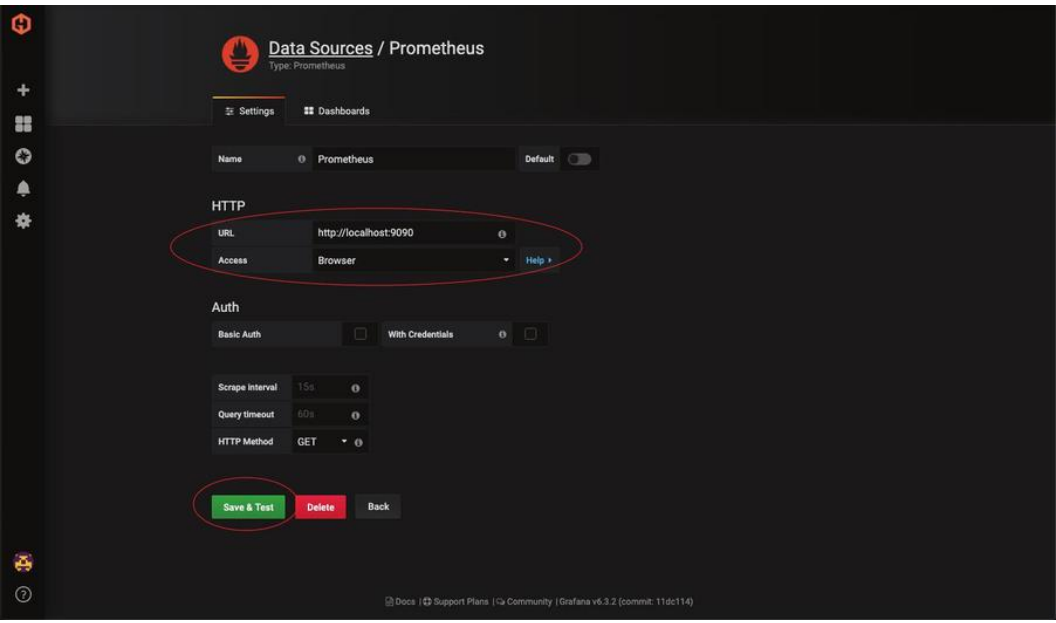
In the MetricFire UI, go to Dashboards on the left side menu, and click Grafana. As seen below, Go to Data Source menu.



Add Prometheus as a Data Source.

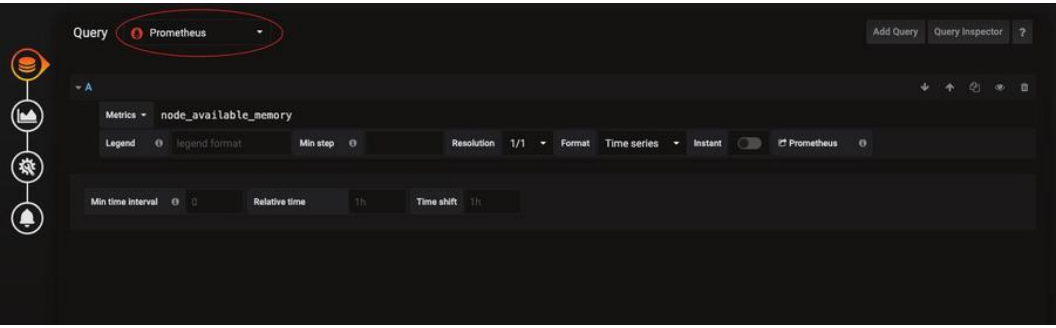


As seen below, you can see the Prometheus data source settings menu. Change the URL to http://localhost:9090. For Access, select Browser. Then, click Save & Test.



c. To view data in Grafana Dashboards, change Data Source to Prometheus

Then, change the Data Source to Prometheus to see your data shown in the Grafana Dashboard.



14.5 Example Grafana dashboards showing Redis Cluster monitoring with Prometheus

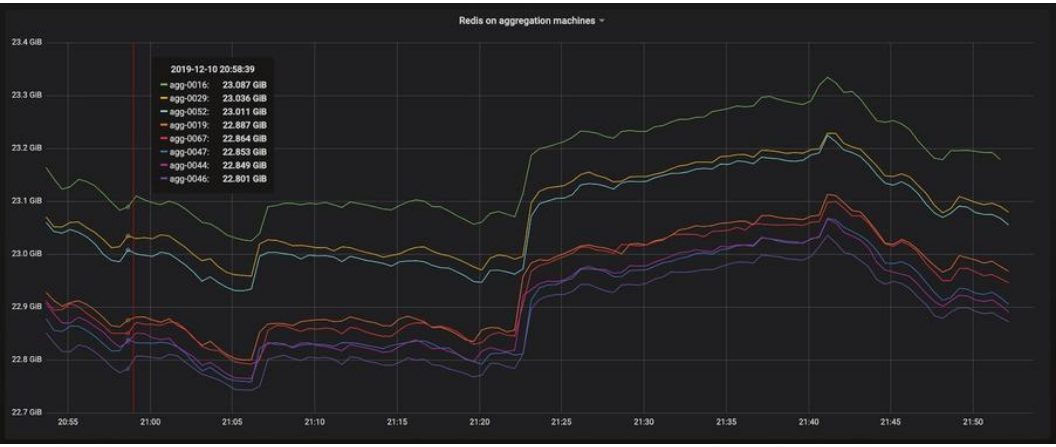
Graph 1 - Dashboard Row with four Graphs

This is an example of a row within a Grafana Dashboard. This row is made up of four panels. Grafana has the ability to group graphs, text, and tables into relevant categories so you can easily sort through different metrics within one dashboard. Organizing your panels helps with correlation and being able to quickly troubleshoot the issue.

This dashboard is showing four metrics pushed from our Redis DB. They are:

1. Redis Client view - the total number of Redis clients
2. Key view - the total number of keys in each Redis DB instance
3. Commands processed - the number of commands processed per group of machines
4. Memory - total memory usage for each different aggregation machines

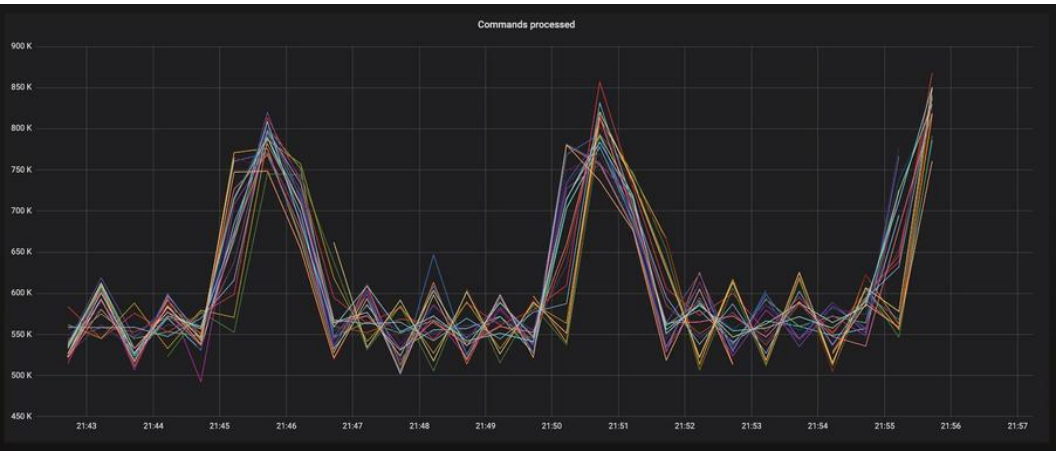
Graph 2 - Redis on Aggregation Machines



This graph shows the total memory usage for different aggregation machines. These machines are responsible for gathering data that is ingested and aggregating the data into more manageable formats. We want to monitor how much memory each resource is using. When a resource is getting close to max memory consumption, performance will start to decrease. A spike in memory usage can act as an identifier for important changes in your application and processes.

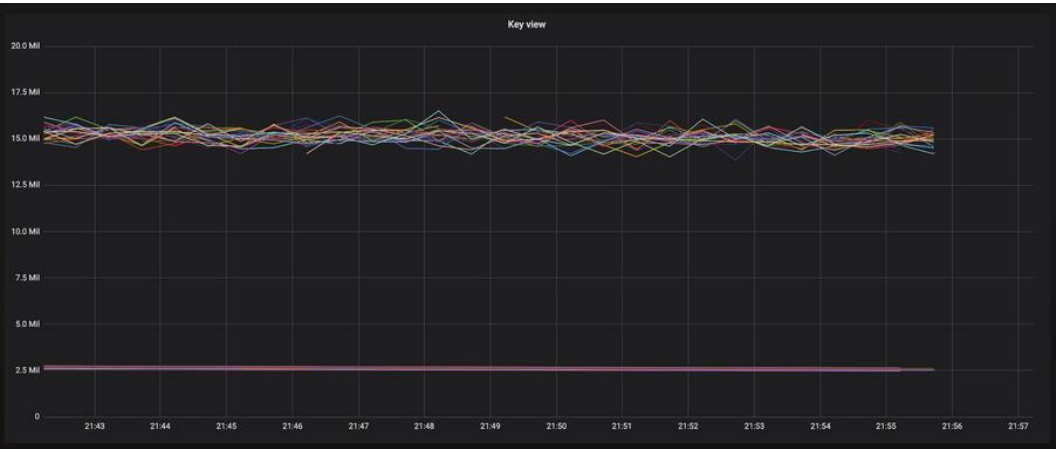
The graph is ‘stacked’ meaning the total range between lines is what the current metric is reading. This makes it easier to see the different metrics being sent when their values are all similar. This graph also has a floating legend, which helps with easy reading.

Graph 3 - Commands Processed



This is the zoomed in ‘Commands Processed’ graph from the row above. It shows the different groups of machines running a Redis DB instance and their associated number of commands processed. The ‘Commands Processed’ graph is an important metric to graph because it allows DB administrators to monitor commands passed to Redis DB. This shows us the traffic and potential stress placed on the resource.

Graph 4 - Key View



This is the zoomed in Key View graph from the dashboard row above. This is showing the total number of keys in each Redis DB instance.

Similar to the other graphs, knowing the total number of keys within an instance gives administrators greater insight into each Redis DB. If you are using Redis DB as a distributed caching store, then a graph like this will be useful to ensure each instance is being properly balanced and utilized. If an instance is showing a significant drop in keys then this is an indicator to look into this issue further.

14.6 Key Metrics for Redis DB

There are a lot of metrics that are automatically pushed from Redis DB. Take a look at a few below, and you can find a full list on the Redis website.

- Bdb_avg_latency - Average latency of operations on the database in microseconds
- Bdb_conns - Number of client connections to database
- Bdb_ingress_bytes - Rates if incoming network traffic to DB in bytes/second
- Bdb_no_of_keys - Number of keys in database
- Node_conns - Number of clients connected to endpoints on nodes
- Node_cpu_user - CPU time portion spent by users-pace process
- Node_free_memory - Free memory in a node in bytes
- Node_up - If a node is part of the cluster and is connected
- Redis_up - Shard is up and running

Example 4: Prometheus Metrics Based Autoscaling in Kubernetes

15.1 Introduction

One of the major advantages of using Kubernetes for container orchestration is that it makes it really easy to scale our application horizontally and account for increased load. Natively, horizontal pod autoscalers can scale the deployment based on CPU and Memory usage but in more complex scenarios we would want to account for other metrics before making scaling decisions.

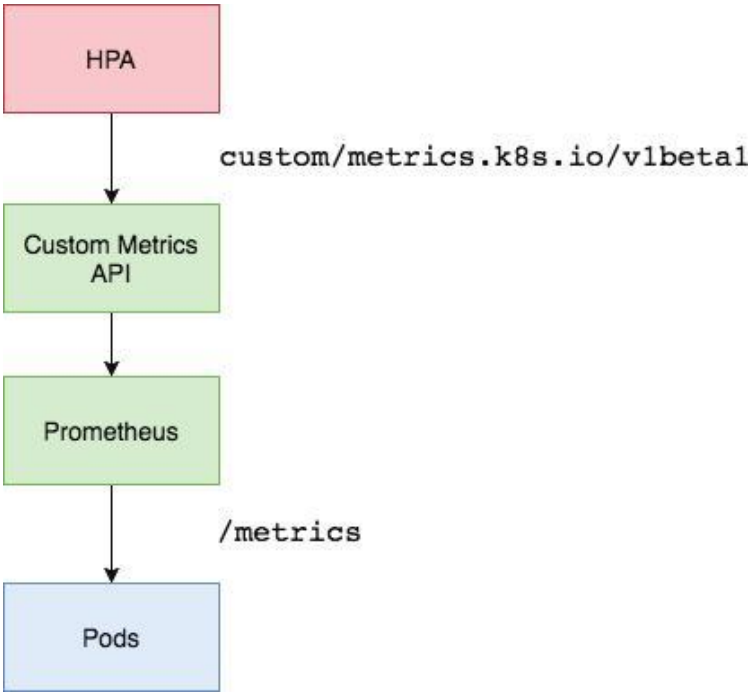
Enter Prometheus Adapter. Prometheus is the standard tool for monitoring deployed workloads and the Kubernetes cluster itself. Prometheus Adapter helps us to leverage the metrics collected by Prometheus and use them to make scaling decisions. These metrics are exposed by an API service and can be readily used by our Horizontal Pod Autoscaling object.

15.2 Deployment

15.2.1 Architecture overview

We will be using Prometheus Adapter to pull custom metrics from our Prometheus installation and then let the horizontal pod autoscaler use it to scale the pods up or down. The Prometheus

Adapter will be running as deployment exposed using a service in our cluster. Generally, a single replica of the Adapter is enough for small to medium sized clusters. However, if you have a very large cluster then you can run multiple replicas of Prometheus Adapter distributed across nodes using Node Affinities and Pod-AntiAffinity properties.



15.2.2 Prerequisites

- Running a Kubernetes cluster with at-least 3 nodes. We will be using a GKE cluster for this tutorial.
- Basic knowledge about horizontal pod autoscaling.
- Prometheus deployed in-cluster or accessible using an endpoint.

We will be using a Prometheus-Thanos Highly Available deployment. More about it can be read here.

15.2.3 Deploying the sample application

Let's first deploy a sample app over which we will be testing our Prometheus metrics autoscaling. We can use the manifest below to do it:

```
apiVersion: v1
kind: Namespace
metadata:
  name: nginx
---
apiVersion: extensions/v1beta1
kind: Deployment
metadata:
  namespace: nginx
  name: nginx-deployment
spec:
  replicas: 1
  template:
    metadata:
      annotations:
        prometheus.io/path: "/status/format/prometheus"
        prometheus.io/scrape: "true"
        prometheus.io/port: "80"
      labels:
        app: nginx-server
```

```
    spec:
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                - key: app
                  operator: In
                  values:
                  - nginx-server
              topologyKey: kubernetes.io/hostname
      containers:
      - name: nginx-demo
        image: vaibhavthakur/nginx-vts:1.0
        imagePullPolicy: Always
        resources:
          limits:
            cpu: 2500m
          requests:
            cpu: 2000m
        ports:
        - containerPort: 80
          name: http
---
apiVersion: v1
kind: Service
metadata:
```

```
  namespace: nginx
  name: nginx-service
spec:
  ports:
  - port: 80
    targetPort: 80
    name: http
  selector:
    app: nginx-server
  type: LoadBalancer
```

This will create a namespace named nginx and deploy a sample **nginx** application in it. The application can be accessed using the service and also exposes nginx vts metrics at the endpoint **`/status/format/prometheus`** over port 80. For the sake of our setup we have created a dns entry for the ExternalIP which maps to **nginx.gotham.com**.

```
root$ kubectl get deploy
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment   1/1     1            1           43d

root$ kubectl get pods
NAME                              READY STATUS  RESTARTS AGEf
nginx-deployment-65d8df7488-c578v 1/1   Running 0        9h

root$ kubectl get svc
NAME          TYPE      CLUSTER-IP    EXTERNAL-IP PORT(S) AGE
nginx-service ClusterIP 10.63.253.154 35.232.67.34 80/TCP 43d
```

```
root$ kubectl describe deploy nginx-deployment
Name:                   nginx-deployment
Namespace:              nginx
CreationTimestamp:      Tue, 08 Oct 2019 11:47:36 -0700
Labels:                 app=nginx-server
Annotations:            deployment.kubernetes.io/revision: 1
                        kubectl.kubernetes.io/last-applied-
configuration:

{"apiVersion":"extensions/v1beta1","kind":"Deployment","me
tadata":{"annotations":{},"name":"nginx-deployment","names
pace":"nginx"},"spec":...
Selector:               app=nginx-server
Replicas:               1 desired | 1 updated | 1 total |
1 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  1 max unavailable, 1 max surge
Pod Template:
  Labels:       app=nginx-server
  Annotations:  prometheus.io/path: /status/format/
prometheus
                prometheus.io/port: 80
                prometheus.io/scrape: true
  Containers:
   nginx-demo:
    Image:      vaibhavthakur/nginx-vts:v1.0
    Port:       80/TCP
    Host Port:  0/TCP
```

```
    Limits:
      cpu:  250m
    Requests:
      cpu:        200m
    Environment:  <none>
    Mounts:       <none>
  Volumes:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
OldReplicaSets:  <none>
NewReplicaSet:   nginx-deployment-65d8df7488 (1/1 replicas
created)
Events:          <none>


root$ curl nginx.gotham.com
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
```

```
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is
successfully installed and working. Further configuration
is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

These are all the metrics currently exposed by the application:

```
$ curl nginx.gotham.com/status/format/prometheus
# HELP nginx_vts_info Nginx info
# TYPE nginx_vts_info gauge
nginx_vts_info{hostname="nginx-deployment-65d8df7488-
c578v",version="1.13.12"} 1
# HELP nginx_vts_start_time_seconds Nginx start time
# TYPE nginx_vts_start_time_seconds gauge
nginx_vts_start_time_seconds 1574283147.043
# HELP nginx_vts_main_connections Nginx connections
# TYPE nginx_vts_main_connections gauge
nginx_vts_main_connections{status="accepted"} 215
nginx_vts_main_connections{status="active"} 4
```

```
nginx_vts_main_connections{status="handled"} 215
nginx_vts_main_connections{status="reading"} 0
nginx_vts_main_connections{status="requests"} 15577
nginx_vts_main_connections{status="waiting"} 3
nginx_vts_main_connections{status="writing"} 1
# HELP nginx_vts_main_shm_usage_bytes Shared memory [ngx_
http_vhost_traffic_status] info
# TYPE nginx_vts_main_shm_usage_bytes gauge
nginx_vts_main_shm_usage_bytes{shared="max_size"} 1048575
nginx_vts_main_shm_usage_bytes{shared="used_size"} 3510
nginx_vts_main_shm_usage_bytes{shared="used_node"} 1
# HELP nginx_vts_server_bytes_total The request/response
bytes
# TYPE nginx_vts_server_bytes_total counter
# HELP nginx_vts_server_requests_total The requests
counter
# TYPE nginx_vts_server_requests_total counter
# HELP nginx_vts_server_request_seconds_total The request
processing time in seconds
# TYPE nginx_vts_server_request_seconds_total counter
# HELP nginx_vts_server_request_seconds The average of
request processing times in seconds
# TYPE nginx_vts_server_request_seconds gauge
# HELP nginx_vts_server_request_duration_seconds The
histogram of request processing time
# TYPE nginx_vts_server_request_duration_seconds histogram
# HELP nginx_vts_server_cache_total The requests cache
counter
# TYPE nginx_vts_server_cache_total counter
```

```
nginx_vts_server_bytes_total{host="_",direction="in"}
3303449
nginx_vts_server_bytes_total{host="_",direction="out"}
61641572
nginx_vts_server_requests_total{host="_",code="1xx"} 0
nginx_vts_server_requests_total{host="_",code="2xx"} 15574
nginx_vts_server_requests_total{host="_",code="3xx"} 0
nginx_vts_server_requests_total{host="_",code="4xx"} 2
nginx_vts_server_requests_total{host="_",code="5xx"} 0
nginx_vts_server_requests_total{host="_",code="total"}
15576
nginx_vts_server_request_seconds_total{host="_"} 0.000
nginx_vts_server_request_seconds{host="_"} 0.000
nginx_vts_server_cache_total{host="_",status="miss"} 0
nginx_vts_server_cache_total{host="_",status="bypass"} 0
nginx_vts_server_cache_total{host="_",status="expired"} 0
nginx_vts_server_cache_total{host="_",status="stale"} 0
nginx_vts_server_cache_total{host="_",status="updating"} 0
nginx_vts_server_cache_
total{host="_",status="revalidated"} 0
nginx_vts_server_cache_total{host="_",status="hit"} 0
nginx_vts_server_cache_total{host="_",status="scarce"} 0
nginx_vts_server_bytes_total{host="*",direction="in"}
3303449
nginx_vts_server_bytes_total{host="*",direction="out"}
61641572
nginx_vts_server_requests_total{host="*",code="1xx"} 0
nginx_vts_server_requests_total{host="*",code="2xx"} 15574
nginx_vts_server_requests_total{host="*",code="3xx"} 0
```

```
nginx_vts_server_requests_total{host="*",code="4xx"} 2
nginx_vts_server_requests_total{host="*",code="5xx"} 0
nginx_vts_server_requests_total{host="*",code="total"}
15576
nginx_vts_server_request_seconds_total{host="*"} 0.000
nginx_vts_server_request_seconds{host="*"} 0.000
nginx_vts_server_cache_total{host="*",status="miss"} 0
nginx_vts_server_cache_total{host="*",status="bypass"} 0
nginx_vts_server_cache_total{host="*",status="expired"} 0
nginx_vts_server_cache_total{host="*",status="stale"} 0
nginx_vts_server_cache_total{host="*",status="updating"} 0
nginx_vts_server_cache_
total{host="*",status="revalidated"} 0
nginx_vts_server_cache_total{host="*",status="hit"} 0
nginx_vts_server_cache_total{host="*",status="scarce"} 0
```

Among these we are particularly interested in **nginx_vts_server_requests_total**. We will be using the value of this metric to determine whether or not to scale our nginx deployment.

15.2.4 Create SSL Certs and the Kubernetes Secret for Prometheus Adapter

We can use the Makefile below to generate openssl certs and corresponding Kubernetes secret:

```
# Makefile for generating TLS certs for the Prometheus
custom metrics API adapter
```

```
SHELL=bash
UNAME := $(shell uname)
PURPOSE:=metrics
SERVICE_NAME:=custom-metrics-apiserver
ALT_NAMES:="custom-metrics-apiserver.monitoring","custom-
metrics-apiserver.monitoring.svc"
SECRET_FILE:=./cm-adapter-serving-certs.yaml

certs: gensecret rmcerts

.PHONY: gencerts
gencerts:
   @echo Generating TLS certs
  @docker pull cfssl/cfssl
  @mkdir -p output
  @touch output/apiserver.pem
  @touch output/apiserver-key.pem
  @openssl req -x509 -sha256 -new -nodes -days 365 -newkey
rsa:2048 -keyout $(PURPOSE)-ca.key -out   $(PURPOSE)-ca.
crt -subj "/CN=ca"
   @echo '{"signing":{"default":{"expiry":"43800h","usag
es":["signing","key encipherment","'$(PURPOSE)'"]}}}' >
"$(PURPOSE)-ca-config.json"
   @echo
'{"CN":"'$(SERVICE_NAME)'","hosts":[$(ALT_NAMES)],"
key":{"algo":"rsa","size":2048}}' | docker run  -v
${HOME}:${HOME} -v ${PWD}/metrics-ca.key:/go/src/github.
com/cloudflare/cfssl/metrics-ca.key -v ${PWD}/metrics-ca.
crt:/go/src/github.com/cloudflare/cfssl/metrics-ca.crt -v
```

```
${PWD}/metrics-ca-config.json:/go/src/github.com/cloudflare/
cfssl/metrics-ca-config.json -i cfssl/cfssl gencert
-ca=metrics-ca.crt -ca-key=metrics-ca.key -config=metrics-
ca-config.json - | docker run --entrypoint=cfssljson -v
${HOME}:${HOME} -v ${PWD}/output:/go/src/github.com/
cloudflare/cfssl/output -i cfssl/cfssl -bare output/
apiserver

.PHONY: gensecret
gensecret: gencerts
  @echo Generating $(SECRET_FILE)
  @echo "apiVersion: v1" > $(SECRET_FILE)
  @echo "kind: Secret" >> $(SECRET_FILE)
  @echo "metadata:" >> $(SECRET_FILE)
  @echo " name: cm-adapter-serving-certs" >> $(SECRET_
FILE)
  @echo " namespace: monitoring" >> $(SECRET_FILE)
  @echo "data:" >> $(SECRET_FILE)
ifeq ($(UNAME), Darwin)
  @echo " serving.crt: $$(cat output/apiserver.pem |
base64)" >> $(SECRET_FILE)
  @echo " serving.key: $$(cat output/apiserver-key.pem |
base64)" >> $(SECRET_FILE)
endif
ifeq ($(UNAME), Linux)
  @echo " serving.crt: $$(cat output/apiserver.pem |
base64 -w 0)" >> $(SECRET_FILE)
  @echo " serving.key: $$(cat output/apiserver-key.pem |
base64 -w 0)" >> $(SECRET_FILE)
```

```
endif

.PHONY: rmcerts
rmcerts:
  @rm -f apiserver-key.pem apiserver.csr apiserver -pem
  @rm -f metrics-ca-config.json metrics-ca.crt metrics-ca.key

.PHONY: deploy-secret
deploy-secret:
  kubectl create -f ./cm-adapter-serving-certs.yaml
```

Once you have created the make file, just run the following command:

```
make certs
```

and it will create ssl certificates and the corresponding Kubernetes secret for you. Make sure that monitoring namespace exists before you create the secret. This secret will be using the Prometheus Adapter which we will deploy next.

15.2.5 Create Prometheus Adapter ConfigMap

Use the manifest below to create the Prometheus Adapter configmap:

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: adapter-config
  namespace: monitoring
data:
  config.yaml: |
    rules:
    - seriesQuery: 'nginx_vts_server_requests_total'
      resources:
        overrides:
          kubernetes_namespace:
            resource: namespace
          kubernetes_pod_name:
            resource: pod
      name:
        matches: "^(.*)_total"
        as: "${1}_per_second"
      metricsQuery: (sum(rate(<<.Series>>{<<.
LabelMatchers>>}[1m])) by (<<.GroupBy>>))
```

This config map only specifies a single metric. However, we can always add more metrics. You can refer to this link to add more metrics. It is highly recommended to fetch only those metrics which we need for the horizontal pod autoscaler. This helps in debugging and also these add-ons generate very verbose logs which get ingested by our logging backend. Fetching metrics which are not needed will not only load the service but also spam the logging backend with un-necessary logs. If you want to learn in detail about the config map, please read here.

15.2.6 Create Prometheus Adapter Deployment

Use the following manifest to deploy Prometheus Adapter:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: custom-metrics-apiserver
  name: custom-metrics-apiserver
  namespace: monitoring
spec:
  replicas: 1
  selector:
    matchLabels:
      app: custom-metrics-apiserver
  template:
    metadata:
      labels:
        app: custom-metrics-apiserver
      name: custom-metrics-apiserver
    spec:
      serviceAccountName: monitoring
      containers:
      - name: custom-metrics-apiserver
        image: quay.io/coreos/k8s-prometheus-adapter-
amd64:v0.4.1
        args:
        - /adapter
```

```
        - --secure-port=6443
        - --tls-cert-file=/var/run/serving-cert/serving.crt
        - --tls-private-key-file=/var/run/serving-cert/
serving.key
        - --logtostderr=true
        - --prometheus-url=http://thanos-querier.
monitoring:9090/
        - --metrics-relist-interval=30s
        - --v=10
        - --config=/etc/adapter/config.yaml
        ports:
        - containerPort: 6443
        volumeMounts:
        - mountPath: /var/run/serving-cert
          name: volume-serving-cert
          readOnly: true
        - mountPath: /etc/adapter/
          name: config
          readOnly: true
      volumes:
      - name: volume-serving-cert
        secret:
          secretName: cm-adapter-serving-certs
      - name: config
        configMap:
          name: adapter-config
```

This will create our deployment which will spawn the Prometheus Adapter pod to pull metrics from Prometheus. It should be noted that we have set the argument **--prometheus-url=**http://thanos-

querier.monitoring:9090/. This is because we have deployed a Thanos backed Prometheus cluster in the monitoring namespace in the same Kubernetes cluster as Prometheus Adapter. You can change this argument to point to your Prometheus deployment.

If you notice the logs for this container, you can see that it is fetching the metric defined in the config file:

```
I1122 00:26:53.228394       1 api.go:74] GET http://tha-
nos-querier.monitoring:9090/api/v1/series?match%5B%5D=ng-
inx_vts_server_requests_total&start=1574381213.217 200 OK
I1122 00:26:53.234234       1 api.go:93] Response Body:
{"status":"success","data":
[{"__name__":"nginx_vts_server_requests_total","app":"ng-
inx-server","cluster":"prometheus-
ha","code":"1xx","host":"*","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"1xx","host":"*","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
```

ha","code":"1xx","host":"_","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"1xx","host":"_","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"2xx","host":"*","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"2xx","host":"*","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"2xx","host":"_","instance":"10.60.64.39:80","-

job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"2xx","host":"_","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"3xx","host":"*","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"3xx","host":"*","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"3xx","host":"_","instance":"10.60.64.39:80","-
job":"kubernetes-

pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"3xx","host":"_","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"4xx","host":"*","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"4xx","host":"*","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"4xx","host":"_","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_

name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"4xx","host":"_","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"5xx","host":"*","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"5xx","host":"*","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"5xx","host":"_","instance":"10.60.64.39:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-

sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"5xx","host":"_","instance":"10.60.64.8:80","-
job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"total","host":"*","in-
stance":"10.60.64.39:80","job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"total","host":"*","in-
stance":"10.60.64.8:80","job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
mwzxg","pod_template_hash":"65d8df7488"},{"__name__":"ng-
inx_vts_server_requests_total","app":"nginx-
server","cluster":"prometheus-
ha","code":"total","host":"_","in-
stance":"10.60.64.39:80","job":"kubernetes-
pods","kubernetes_namespace":"nginx","kubernetes_pod_
name":"nginx-deployment-65d8df7488-
sbp95","pod_template_hash":"65d8df7488"},{"__name__":"ng-

```
inx_vts_server_requests_total",”app”:”nginx-
server”,”cluster”:”prometheus-
ha”,”code”:”total”,”host”:”_”,”in-
stance”:”10.60.64.8:80”,”job”:”kubernetes-
pods”,”kubernetes_namespace”:”nginx”,”kubernetes_pod_
name”:”nginx-deployment-65d8df7488-
mwzxg”,”pod_template_hash”:”65d8df7488”}]}
```

15.2.7 Create Prometheus Adapter API Service

The manifest below will create an API service so that our Prometheus Adapter is accessible by Kubernetes API and thus metrics can be fetched by our Horizontal Pod Autoscaler.

```
apiVersion: v1
kind: Service
metadata:
  name: custom-metrics-apiserver
  namespace: monitoring
spec:
  ports:
  - port: 443
    targetPort: 6443
  selector:
    app: custom-metrics-apiserver
---
apiVersion: apiregistration.k8s.io/v1beta1
kind: APIService
metadata:
  name: v1beta1.custom.metrics.k8s.io
```

```
spec:
  service:
    name: custom-metrics-apiserver
    namespace: monitoring
  group: custom.metrics.k8s.io
  version: v1beta1
  insecureSkipTLSVerify: true
  groupPriorityMinimum: 100
  versionPriority: 100
```

15.3. Testing the setup

Let's check all of the custom metrics that are available:

```
root$ kubectl get --raw "/apis/custom.metrics.k8s.io/
v1beta1" | jq .


{
  "kind": "APIResourceList",
  "apiVersion": "v1",
  "groupVersion": "custom.metrics.k8s.io/v1beta1",
  "resources": [
    {
      "name": "pods/nginx_vts_server_requests_per_second",
      "singularName": "",
      "namespaced": true,
      "kind": "MetricValueList",
      "verbs": [
        "get"
       ]
```

```
    },
    {
      "name": "namespaces/nginx_vts_server_requests_per_
second",
      "singularName": "",
      "namespaced": false,
      "kind": "MetricValueList",
      "verbs": [
        "get"
      ]
    }
  ]
}
```

We can see that **nginx_vts_server_requests_per_second** metric is available.

Now, let's check the current value of this metric:

```
root$ kubectl get --raw "/apis/custom.metrics.k8s.io/
v1beta1/namespaces/nginx/pods/*/nginx_vts_server_requests_
per_second" | jq .

{
  "kind": "MetricValueList",
  "apiVersion": "custom.metrics.k8s.io/v1beta1",
  "metadata": {
    "selfLink": "/apis/custom.metrics.k8s.io/v1beta1/
namespaces/nginx/pods/%2A/nginx_vts_server_requests_per_
```

```
second"
  },
  "items": [
    {
      "describedObject": {
        "kind": "Pod",
        "namespace": "nginx",
        "name": "nginx-deployment-65d8df7488-v575j",
        "apiVersion": "/v1"
      },
      "metricName": "nginx_vts_server_requests_per_
second",
      "timestamp": "2019-11-19T18:38:21Z",
      "value": "1236m"
    }
  ]
}
```

Create an HPA which will utilize these metrics. We can use the manifest below to do it:

```
apiVersion: autoscaling/v2beta1
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-custom-hpa
  namespace: nginx
spec:
  scaleTargetRef:
    apiVersion: extensions/v1beta1
```

```
    kind: Deployment
    name: nginx-deployment
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Pods
    pods:
      metricName: nginx_vts_server_requests_per_second
      targetAverageValue: 4000m
```

Once you have applied this manifest, you can check the current status of HPA as follows:

```
root$ kubectl describe hpa
Name:                nginx-custom-hpa
Namespace:           nginx
Labels:              <none>
Annotations:         autoscaling.alpha.kubernetes.io/
metrics:

[{"type":"Pods","pods":{"metricName":"nginx_vts_server_
requests_per_second","targetAverageValue":"4"}}]
                     kubectl.kubernetes.io/last-applied-
configuration:
                       {"apiVersion":"autoscaling/v2beta1",
"kind":"HorizontalPodAutoscaler","metadata":{"annotations"
:{},"name":"nginx-custom-hpa","namespace":"n...
CreationTimestamp:   Thu, 21 Nov 2019 11:11:05 -0800
Reference:           Deployment/nginx-deployment
```

```
Min replicas:       2
Max replicas:       10
Deployment pods:    0 current / 0 desired
Events:             <none>
```

Now, let's generate some load on our service. We will be using a utility called vegeta for this. In a separate terminal run this following command:

```
echo "GET http://nginx.gotham.com/" | vegeta attack -rate=5 -duration=0 | vegeta report
```

If you simultaneously monitor the nginx pods and the horizontal pod autoscaler, you should see something like this:

```
root$ kubectl get -w pods
NAME                                READY STATUS    RESTARTS AGE
nginx-deployment-65d8df7488-mwzxg   1/1   Running   0        9h
nginx-deployment-65d8df7488-sbp95   1/1   Running   0        4m9s
NAME                                 AGE
nginx-deployment-65d8df7488-pwjzm    0s
nginx-deployment-65d8df7488-pwjzm    0s
nginx-deployment-65d8df7488-pwjzm    0s
nginx-deployment-65d8df7488-pwjzm    2s
nginx-deployment-65d8df7488-pwjzm    4s
nginx-deployment-65d8df7488-jvbvp    0s
nginx-deployment-65d8df7488-jvbvp    0s
nginx-deployment-65d8df7488-jvbvp    1s
nginx-deployment-65d8df7488-jvbvp    4s
```

```
nginx-deployment-65d8df7488-jvbvp   7s
nginx-deployment-65d8df7488-skjkm   0s
nginx-deployment-65d8df7488-skjkm   0s
nginx-deployment-65d8df7488-jh5vw   0s
nginx-deployment-65d8df7488-skjkm   0s
nginx-deployment-65d8df7488-jh5vw   0s
nginx-deployment-65d8df7488-jh5vw   1s
nginx-deployment-65d8df7488-skjkm   2s
nginx-deployment-65d8df7488-jh5vw   2s
nginx-deployment-65d8df7488-skjkm   3s
nginx-deployment-65d8df7488-jh5vw   4s

root$ kubectl get hpa
NAME              REFERENCE                     TARGETS   MINPODS
nginx-custom-hpa  Deployment/nginx-deployment  5223m/4   2
MAXPODS   REPLICAS   AGE
10        3          5m5s
```

It can be clearly seen that the horizontal pod autoscaler scaled up our pods to meet the needs, and when we interrupt the vegeta command we can see the vegeta report. It clearly shows that all our requests were served by the application.

```
root$ echo "GET http://nginx.gotham.com/" | vegeta attack
-rate=5 -duration=0 | vegeta report
^CRequests      [total, rate, throughput]  224, 5.02, 5.02
Duration        [total, attack, wait]      44.663806863s,
44.601823883s, 61.98298ms
Latencies       [mean, 50, 95, 99, max]    63.3879ms,
```

```
60.867241ms, 79.414139ms, 111.981619ms, 229.310088ms
Bytes In      [total, mean]                 137088, 612.00
Bytes Out     [total, mean]                 0, 0.00
Success       [ratio]                       100.00%
Status Codes  [code:count]                  200:224
Error Set:
```

15.4 Conclusion

This set-up demonstrates how we can use Prometheus Adapter to autoscale deployments based on some custom metrics. For the sake of simplicity we have only fetched one metric from our Prometheus Server. However, the Adapter configmap can be extended to fetch some or all the available metrics and use them for autoscaling.

If the Prometheus installation is outside of our Kubernetes cluster, we just need to make sure that the query end-point is accessible from the cluster and update it in the Adapter deployment manifest. We can have complex scenarios where multiple metrics can be fetched and used in combination to make scaling decisions.